

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

Факультет інформатики та обчислювальної техніки

(повне найменування інституту, факультету)

Інформатики та програмної інженерії

(повна назва кафедри)

Курсова робота

з дисципліни Компоненти програмної інженерії

на тему Веб-сервіс прогнозування погоди, шляхом агрегації даних онлайн
гідроцентрів

Виконав (ла): студент (ка) ІІІ
курсу, групи

ІІІ-31 Віжукін Ілля
Дмитрович

(прізвище, ім'я, по батькові)

(підпис)

Керівник

ст. викладач, PhD, Головченко М.М.

посада, науковий ступінь, вчене звання, прізвище, і ім'я, по батькові

(підпис)

Члени комісії

доц., PhD, Поперешняк С.В.

посада, науковий ступінь, вчене звання, прізвище, і ім'я, по батькові

(підпис)

асистент, PhD, Ахаладзе А.Е.

посада, науковий ступінь, вчене звання, прізвище, і ім'я, по батькові

(підпис)

Засвідчую, що у цій курсовій роботі немає
запозичень з праць інших авторів без
відповідних посилань.

Студент

(підпис)

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Рівень вищої освіти – перший (бакалаврський)

Спеціальність – 121 Інженерія програмного забезпечення

Освітньо-професійна програма – Інженерія програмного забезпечення
інформаційних систем

ЗАТВЕРДЖУЮ

керівник

(підпис) Максим ГОЛОВЧЕНКО
(ім'я прізвище)

“ ____ ” _____ 2025 р.

ЗАВДАННЯ
на курсову роботу студенту

Віжуткіну Іллі Дмитровичу
(прізвище, ім'я, по батькові)

1. Тема роботи Веб-сервіс прогнозування погоди, шляхом агрегації даних
онлайн гідрометцентрів

керівник роботи Головченко Максим Миколайович, PhD
(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

2. Термін подання студентом роботи « 3 » грудня 2025 року

3. Вихідні дані до роботи: технічне завдання

4. Зміст пояснювальної записки

Передпроектне обстеження предметної області, розроблення вимог до
програмного забезпечення, конструювання та розроблення програмного
забезпечення, аналіз якості та тестування програмного забезпечення, повна
інструкція користувача

5. Перелік графічного матеріалу

- 1) _____
- 2) _____
- 3) _____
- 4) _____
- 5) _____
- 6) _____

6. Консультанти розділів роботи

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання «8» жовтня 2025 року _____

Календарний план

№ з/п	Назва етапів виконання курсової роботи	Термін виконання етапів роботи	Примітка
1	Вивчення рекомендованої літератури	12.10.2025	
2	Аналіз предметної області та аналогів	20.10.2025	
3	Формулювання вимог до ПЗ	27.10.2025	
4	Проектування архітектури ПЗ	03.11.2025	
5	Розробка backend-частини	17.11.2025	
6	Розробка frontend-частини	24.11.2025	
7	Тестування ПЗ та аналіз якості	27.11.2025	
8	Написання інструкції користувача	28.11.2025	
9	Оформлення пояснювальної записки	1.12.2025	
10	Подання КР на основний захист	3.12.2025	

Студент

(підпис)

Ілля Віжуткін

(ініціали, прізвище)

Керівник

(підпис)

Максим ГОЛОВЧЕНКО

(ініціали, прізвище)

АНОТАЦІЯ

Пояснювальна записка курсової роботи складається з п'яти розділів, містить 16 таблиць, 54 рисунки та 51 джерело – загалом 89 сторінок.

Курсова робота присвячена розробці веб-сервісу для прогнозування погоди шляхом агрегації даних з різних онлайн гідрометцентрів.

Метою розробки є забезпечення швидкого отримання даних із різних метеорологічних джерел для надання більш повної та об'єктивної картини погодних умов, що дозволить зменшити вірогідність неправильного прогнозу.

У першому розділі розглянуто предметну область, проаналізовано існуючі аналоги та обґрунтовано вибір архітектурних рішень, а також описано основний бізнес-процес системи. Другий розділ присвячений розробленню вимог до програмного забезпечення, включаючи варіанти використання, системні, функціональні та нефункціональні вимоги. У третьому розділі детально описано архітектуру програмного забезпечення за допомогою діаграм C4, обґрунтовано вибір технологічного стеку та засобів розробки, а також надано опис сторонніх API та бібліотек, що використовуються. Четвертий розділ містить аналіз якості розробленого ПЗ за ключовими метриками з результатами тестування, а також опис процесу функціонального тестування з прикладами тест-кейсів та детальним контрольним прикладом. П'ятий розділ надає повну інструкцію користувача, що охоплює як процес розгортання сервісу, так і використання його основного функціоналу.

Програмне забезпечення розгорнуто на сервері Amazon EC2 за допомогою Docker Compose і доступне через веб-браузер.

КЛЮЧОВІ СЛОВА: ВЕБ-СЕРВІС, ПРОГНОЗ ПОГОДИ, АГРЕГАЦІЯ ДАНИХ, API, PYTHON, FASTAPI, REACT, DOCKER, REDIS, NGINX.

Відомість курсової роботи

№ з/п	Формат	Позначення	Найменування	Кількість листів	Примітка
1	A4		Завдання на курсову роботу	2	
2	A4	КПІ.ІІІ-3104.045440.01.91	Технічне завдання	18	
3	A4	КПІ.ІІІ-3104.045440.02.81	Пояснювальна записка	92	
4	A4	КПІ.ІІІ-3104.045440.03.12	Текст програми	36	

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

керівник

_____ Максим ГОЛОВЧЕНКО

“ ____ ” _____ 2025 р.

**ВЕБ-СЕРВІС ПРОГНОЗУВАННЯ ПОГОДИ, ШЛЯХОМ АГРЕГАЦІЇ
ДАНИХ ОНЛАЙН ГІДРОМЕТЦЕНТРІВ**

Технічне завдання

КП.ІП-3104.045440.01.91

“ПОГОДЖЕНО”

Керівник роботи:

Максим ГОЛОВЧЕНКО

Виконавець:

Ілля ВІЖУТКІН

Київ – 2025

ЗМІСТ

1	НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ	4
2	ПІДСТАВА ДЛЯ РОЗРОБКИ	5
3	ПРИЗНАЧЕННЯ РОЗРОБКИ.....	6
4	ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	7
4.1	Вимоги до функціональних характеристик.....	7
4.1.1	Отримання інформації про погодні умови:	7
4.1.2	Агрегування даних з різних джерел:	7
4.1.3	Автоматичне визначення місцезнаходження:	7
4.1.4	Отримання динамічної картини:.....	7
4.2	Вимоги до надійності	12
4.3	Умови експлуатації	12
4.3.1	Вид обслуговування	13
4.3.2	Обслуговуючий персонал.....	13
4.4	Вимоги до складу і параметрів технічних засобів	13
4.5	Вимоги до інформаційної та програмної сумісності	14
4.5.1	Вимоги до вхідних даних	14
4.5.2	Вимоги до вихідних даних	14
4.5.3	Вимоги до мови розробки	14
4.5.4	Вимоги до середовища розробки	14
4.5.5	Вимоги до представлення вихідних кодів	14
4.6	Вимоги до маркування та пакування	15
4.7	Вимоги до транспортування та зберігання.....	15
4.8	Спеціальні вимоги	15
5	ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ	16

5.1	Попередній склад програмної документації	16
5.2	Спеціальні вимоги до програмної документації	16
6	СТАДІЇ І ЕТАПИ РОЗРОБКИ.....	17
7	ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ	18

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Назва розробки: веб-сервіс прогнозування погоди, шляхом агрегації даних онлайн гідрометцентрів.

Галузь застосування:

Наведене технічне завдання поширюється на розробку веб-сервісу прогнозування погоди, шляхом агрегації даних онлайн гідрометцентрів КП.ІП-3104.045440.02.81, котра використовується для надання користувачам прогнозу з декількох джерел в єдиному інтерфейсі та призначена для використання особами, що займаються плануванням поїздок, заходів, логістикою, сільським господарством, туризмом та іншими погодозалежними активностями.

2 ПІДСТАВА ДЛЯ РОЗРОБКИ

Підставою для розробки веб-сервісу прогнозування погоди, шляхом агрегації даних онлайн гідрометцентрів, є індивідуальний навчальний план студента та РНП затверджена кафедрою інформатики та програмної інженерії Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

3 ПРИЗНАЧЕННЯ РОЗРОБКИ

Розробка призначена для автоматизації процесу отримання актуальної метеорологічної інформації з кількох онлайн джерел (гідрометцентрів) шляхом агрегації даних через API та подальшого надання цієї інформації користувачам у зручному та зрозумілому веб-інтерфейсі. Сервіс може бути використаний для планування діяльності, що залежить від погодних умов.

Метою розробки є забезпечення швидкого отримання даних із різних метеорологічних джерел для надання більш повної та об'єктивної картини погодних умов, що дозволить зменшити вірогідність неправильного прогнозу.

4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Вимоги до функціональних характеристик

Програмне забезпечення повинно забезпечувати виконання наступних основних функцій:

4.1.1 Отримання інформації про погодні умови:

- можливість вводу назви населеного пункту, із підказками, при частковому введенні, для отримання актуальних метеорологічних даних для нього;
- можливість отримання як поточних погодних умов, так і прогнозу на певний день;
- прогноз має включати температуру, стан неба, вітер, тиск, ймовірність опадів, їх кількість та інші погодні параметри.

4.1.2 Агрегування даних з різних джерел:

- система має збирати погодні дані з обраних джерел;
- результат має містити перелік використаних джерела, тобто ті, з яких вдалося отримати дані;
- можливість отримати як найбільш вірогідну оцінку погодних параметрів, так і результати окремих джерел.

4.1.3 Автоматичне визначення місцезнаходження:

- можливість автоматичного визначення місцезнаходження користувача та відображення відповідного прогнозу погоди.

4.1.4 Отримання динамічної картини:

- система має надавати графіки зміни прогнозованих метеорологічних показників;
- можливість задавати період для графіків динаміки.

Для реалізації відповідних функціональних вимог були розроблені прототипи екранних форм, які демонструють основний функціонал користувацького інтерфейсу.

На рисунку 4.1 можна побачити, що, при першому запуску веб-застосунку (у разі підтвердження користувача), автоматично надається прогноз для поточного місцязнаходження. На цьому ж рисунку зображено можливість введення іншого міста та отримання відповідних підказок.

На рисунку 4.2 видно повний результат прогнозу погоди для конкретного міста, із зазначеними джерелами та можливістю обрання певного дня, для отримання більш обширної інформації (рисунок 4.3).

На рисунку 4.4 зображено можливість обрання динаміки прогнозу погоди для конкретного параметру та заданого періоду.

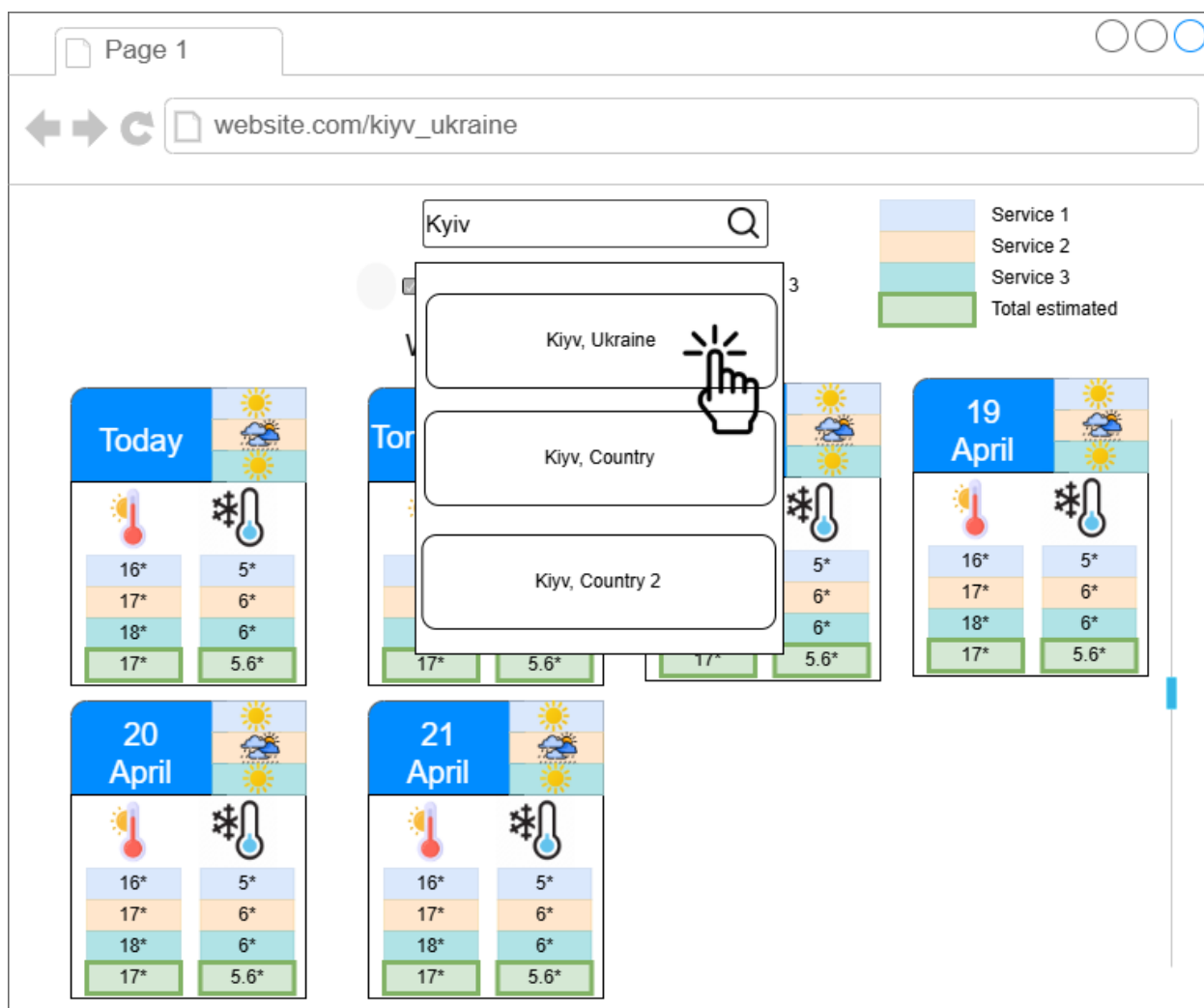


Рисунок 4.1 – Отримання початкового прогнозу та введення іншого міста

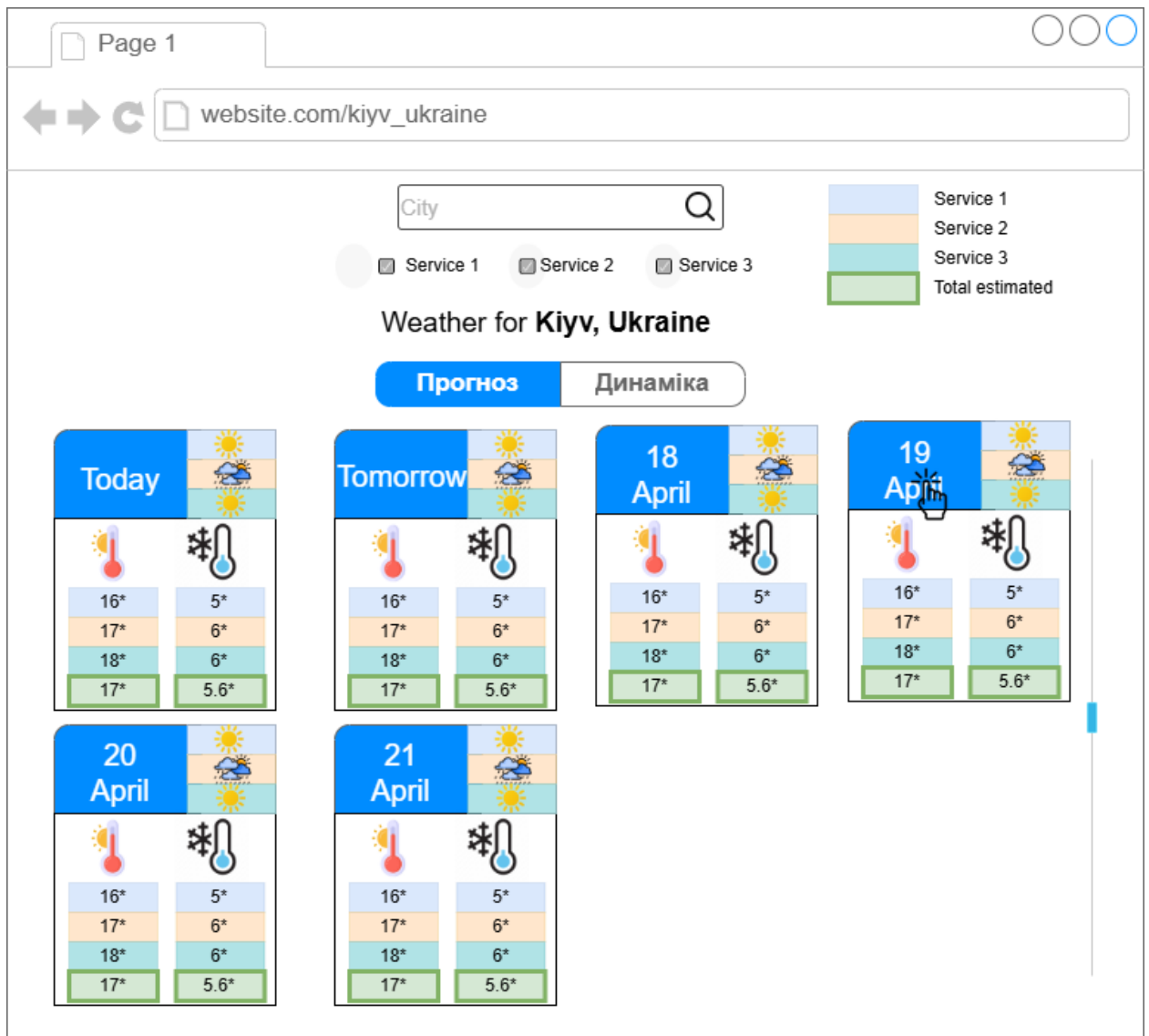


Рисунок 4.2 – Агрегований прогноз на різні дні

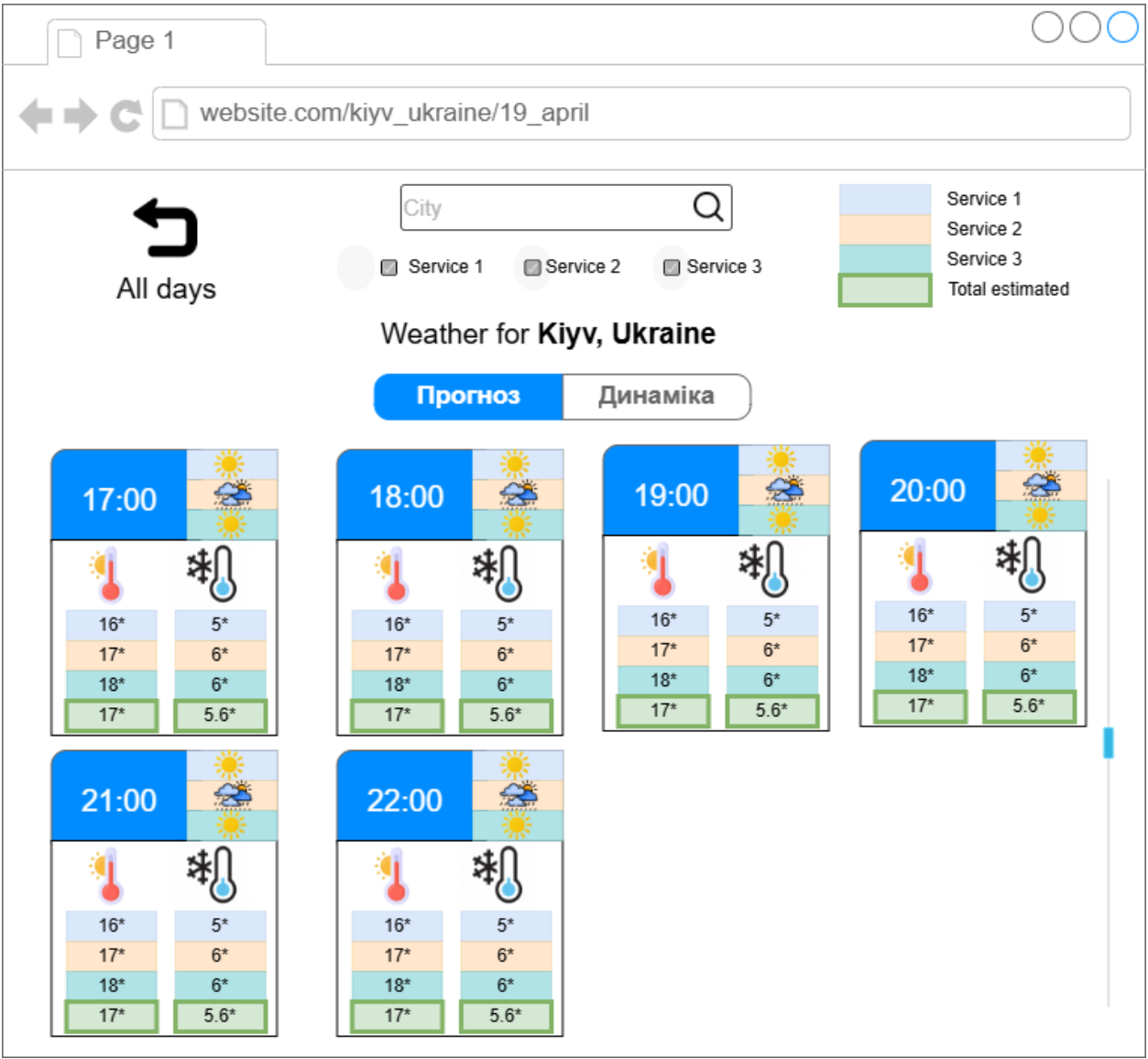


Рисунок 4.3 – Агрегований прогноз на конкретний день



Рисунок 4.4 – Динаміка прогнозу

4.2 Вимоги до надійності

- програмне забезпечення має бути доступним не менше 99% часу роботи;
- у випадку відмови одного або кількох джерел даних, система має продовжувати роботу із іншими;
- при некоректних введених даних, система має надавати зрозумілі повідомлення з описом помилки.

4.3 Умови експлуатації

Умови експлуатації згідно СанПін 2.2.2.542 – 96.

4.3.1 Вид обслуговування

Програмне забезпечення потребує оновлення підписки на відповідних метеорологічних сервісах.

4.3.2 Обслуговуючий персонал

Обслуговування може здійснювати будь-який користувач, який має необхідні дані для авторизації на відповідних метеорологічних сервісах.

4.4 Вимоги до складу і параметрів технічних засобів

Програмне забезпечення повинно функціонувати на IBM-сумісних персональних комп'ютерах або мобільних пристроях із доступом до мережі Інтернет.

Мінімальна конфігурація технічних засобів:

- тип процесору: Intel Core i3-5005U або еквівалент;
- об'єм ОЗП: 4 ГБ;
- підключення до мережі Інтернет із швидкістю від 8 Мбіт/с.

Рекомендована конфігурація технічних засобів:

- тип процесору: Intel Core i5-6500;
- об'єм ОЗП: 8 ГБ;
- підключення до мережі Інтернет із швидкістю від 80 Мбіт/с.

Для розгортання веб-сервісу на сервері з урахування обраної архітектури та технологій, що включає FastAPI, Redis, Nginx, Docker та React, необхідно забезпечити наступні технічні характеристики серверного обладнання.

Мінімальна конфігурація серверу:

- тип процесору: 1 vCPU;
- об'єм ОЗП: 1 ГБ;
- вільне місце на диску: 5 ГБ;
- операційна система: Ubuntu 20.04 або новіша;
- підключення до мережі Інтернет із швидкістю від 20 Мбіт/с.

Рекомендована конфігурація серверу:

- тип процесору: 4 vCPU;
- об'єм ОЗП: 4 ГБ;
- вільне місце на диску: 10 ГБ;
- підключення до мережі Інтернет із швидкістю від 100 Мбіт/с.

4.5 Вимоги до інформаційної та програмної сумісності

Програмне забезпечення є веборієнтованим - для коректного функціонування необхідний один із сучасних браузерів:

- Google Chrome (версія 80 або більше);
- Mozilla Firefox (версія 74 або більше);
- Edge (версія 80 або більше);
- Opera (версія 67 або більше);
- Safari (версія 14 або більше).

4.5.1 Вимоги до вхідних даних

Жодних вхідних даних не передбачено.

4.5.2 Вимоги до вихідних даних

Жодних вихідних даних не передбачено.

4.5.3 Вимоги до мови розробки

Розробка програмного забезпечення здійснюється мовами Python (backend) та Javascript (frontend).

4.5.4 Вимоги до середовища розробки

Розробка відбувається у середовищі Visual Studio Code.

4.5.5 Вимоги до представлення вихідних кодів

Вихідні коди будуть представлені у вигляді GitHub-репозиторію.

4.6 Вимоги до маркування та пакування

Вимоги до маркування та пакування не висуваються.

4.7 Вимоги до транспортування та зберігання

Вимоги до транспортування та зберігання не висуваються.

4.8 Спеціальні вимоги

Спеціальні вимоги до системи не висуваються.

5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ

5.1 Попередній склад програмної документації

У склад супроводжувальної документації повинні входити наступні документи на аркушах формату А4:

- технічне завдання;
- пояснювальна записка;
- текст програми.

5.2 Спеціальні вимоги до програмної документації

Програмні модулі, котрі розробляються, повинні бути задокументовані, тобто тексти програм повинні містити всі необхідні коментарі.

6 СТАДІЇ І ЕТАПИ РОЗРОБКИ

№	Назва етапу	Строк	Звітність
1.	Вивчення рекомендованої літератури	12.10.2025	
2.	Аналіз предметної області та аналогів	20.10.2025	
3.	Формулювання вимог до ПЗ	27.10.2025	
4.	Проектування архітектури ПЗ	03.11.2025	
5.	Розробка backend-частини	17.11.2025	
6.	Розробка frontend-частини	24.11.2025	
7.	Тестування ПЗ та аналіз якості	27.11.2025	
8.	Написання інструкції користувача	28.11.2025	
9.	Оформлення пояснювальної записки	1.12.2025	
10.	Подання КР на основний захист	3.12.2025	

7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ

Тестування розробленого програмного продукту виконується відповідно до “Розділу тестування програмного забезпечення”.

Пояснювальна записка до курсової роботи

на тему: Веб-сервіс прогнозування погоди, шляхом агрегації даних онлайн
гідрометцентрів

КПІ.ПІ-3104.045440.02.81

ЗМІСТ

ВСТУП.....	5
1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ.....	6
1.1 Аналіз предметної області.....	6
1.2 Аналіз існуючих рішень	8
1.2.1 Аналіз відомих програмних продуктів	8
1.2.2 Аналіз відомих алгоритмічних та технічних рішень	17
1.3 Опис бізнес-процесів	18
Висновки до розділу	20
2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	22
2.1 Варіанти використання програмного забезпечення.....	22
2.2 Аналіз системних вимог	25
2.3 Розроблення функціональних вимог	28
2.4 Розроблення нефункціональних вимог.....	32
2.5 Постановка задачі	34
Висновки до розділу	35
3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	37
3.1 Архітектура програмного забезпечення	37
3.2 Обґрунтування вибору засобів розробки.....	41
3.3 Конструювання програмного забезпечення	43
3.3.1 Опис сторонніх сервісів	43

3.3.2 Опис утіліт, бібліотек та іншого стороннього програмного забезпечення	56
Висновки до розділу	59
4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	60
4.1 Аналіз якості ПЗ.....	60
4.2 Опис процесів тестування	63
4.3 Опис контрольного прикладу.....	69
Висновки до розділу	75
5 ПОВНА ІНСТРУКЦІЯ КОРИСТУВАЧА.....	77
5.1 Встановлення програмного забезпечення	77
5.2 Інструкція користувача.....	78
Висновки до розділу	82
ВИСНОВКИ.....	84
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	86

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

API – Application Programming Interface, прикладний програмний інтерфейс.

REST – Representational State Transfer, передача репрезентованого стану

UI – User Interface, інтерфейс користувача.

BPMN – Business Process Model and Notation, нотація моделювання бізнес-процесів.

VPS – Virtual Private Server, віртуальний приватний сервер

Frontend – частина програми, з якою взаємодіє користувач.

Backend – серверна частина програми, що відповідає за обробку даних і логіку

ВСТУП

У сучасному світі прогнозування погоди стало дуже важливим для загальної життєдіяльності суспільства, оскільки від нього залежить безліч аспектів життя: від планування аграрних робіт до організації масових заходів. Актуальність даної роботи обумовлена необхідністю забезпечення легкого доступу до найбільш актуальних метеорологічних даних, які можуть бути отримані шляхом агрегації інформації.

На сьогоднішній день використання онлайн веб-сервісів стало дуже популярним і сервіси прогнозування погоди не є виключенням [5]. Країни по всьому світу активно впроваджують новітні технології для поліпшення точності прогнозів. Відомі наукові установи та дослідницькі організації активно розробляють нові установи та методи обробки метеорологічних даних, що дозволяє покращити не лише точність, але й швидкість отримання актуальних прогнозів.

На даний момент існує досить велика кількість платформ, які надають метеорологічну інформацію, проте взагалі не всі з них забезпечують інтеграцію даних з кількох джерел, що може призвести до менш точної інформації. У зв'язку з цим, розробка веб-сервісу, що агрегує дані з різних онлайн-центрів, є актуальною задачею.

Розроблений веб-сервіс буде мати широкий спектр застосувань, охоплюючи практично всі сфери життя людини. Наприклад, у сільському господарстві точні прогнози погоди допомагають ефективно планувати посіви та збори врожаю, зменшуючи ризики втрат. У туризмі сервіс дозволяє туристам обирати оптимальні дати для поїздок, враховуючи погодні умови, що підвищує комфорт і безпеку. У повсякденному житті актуальна інформація про погоду допомагає людям планувати проведення свого дня та організовувати різні заходи. Отже, надійні метеорологічні дані сприяють прийняттю обґрунтованих рішень та підвищують ефективність планування в різних аспектах людської діяльності.

1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Аналіз предметної області

У сучасному світі веб-застосунки стали дуже популярними, оскільки вони забезпечують зручний доступ до різноманітних сервісів. Веб-застосунок [1] визначається як програмне забезпечення, яке працює на веб-сервері і доступне для користувачів через браузер. Це дозволяє користуватися сервісом без необхідності його встановлення на локальний пристрій. Веб-застосунки охоплюють широкий спектр функцій, від простих форм для заповнення до складних систем для управління великими об'ємами даних, що робить їх важливими інструментами у різних сферах, включаючи бізнес, освіту та науку.

Прогнозування погоди, як предметна область, є складною і багатогранною [6,7]. Вона вимагає обробки та аналізу великих обсягів метеорологічних даних, зібраних з різних джерел, таких як метеостанції, супутники та радіолокаційні системи. Сучасні ІТ-технології пропонують різноманітні способи обробки даних, включаючи використання штучного інтелекту та машинного навчання для покращення точності прогнозів.

На поточному етапі розвитку технологій процес створення агрегаторів метеорологічних даних охоплює кілька важливих етапів [6,7]. Першим етапом є автоматизований збір даних з різноманітних джерел, таких як національні гідрометцентри, приватні метеостанції або супутникові системи. Далі слідує обробка та уніфікація отриманих даних, оскільки різні джерела можуть використовувати різні формати. Об'єднані дані піддаються аналізу за допомогою алгоритмів, що дозволяє виявляти можливі помилки та створювати прогнози на основі агрегованої інформації. Останнім етапом є візуалізація - представлення даних у вигляді інтерактивних графіків, карт або таблиць, доступних через веб-інтерфейс.

Типові можливості агрегаторів включають відображення погодних умов у реальному часі, довгострокові прогнози, аналіз історичних даних, сповіщення про зміни погоди, і налаштування під користувацькі запити [8].

Однак існує низка проблем, з якими стикаються сучасні агрегатори [6,7]. Відсутність стандартизації даних з різних джерел може призводити до розбіжностей у прогнозах. Затримки в оновленні інформації через складність інтеграції систем також знижують актуальність даних. Крім того, обмеження алгоритмів обробки великих обсягів даних можуть впливати на швидкість роботи під час пікових навантажень.

Шляхи покращення можуть включати вдосконалення алгоритмів агрегації, підвищення швидкості обробки даних за допомогою розподілених систем і використання додаткових методів валідації даних для покращення якості прогнозів [6].

У рамках цієї курсової роботи буде розроблено веб-сервіс, який об'єднуватиме дані з різних метеорологічних центрів. Основна мета - забезпечити користувачів більш повною та комплексною інформацією про погодні умови, об'єднавши різні джерела в одному інтерфейсі. На відміну від існуючих рішень, сервіс зосереджуватиметься на ефективній агрегації даних без розробки нових алгоритмів аналізу. Це дозволить мінімізувати залежність від окремих джерел та підвищити надійність і точність прогнозів.

1.2 Аналіз існуючих рішень

Проаналізуємо відомі на сьогодні програмні продукти та технічні рішення, що допоможуть у реалізації веб-сервісу прогнозування погоди шляхом агрегації даних різних гідрометцентрів.

1.2.1 Аналіз відомих програмних продуктів

Спочатку проаналізуємо існуючи аналоги по агрегації погоди.

Першим прикладом відомих аналогів є Weather [2]. Цей сервіс надає детальну інформацію про погоду, включаючи прогнози на кілька днів наперед та можливість перегляду мап погоди. Інтерфейс сайту (рисунки 1.1 – 1.3) досить зручний і зрозумілий, що дозволяє легко отримувати необхідну інформацію.

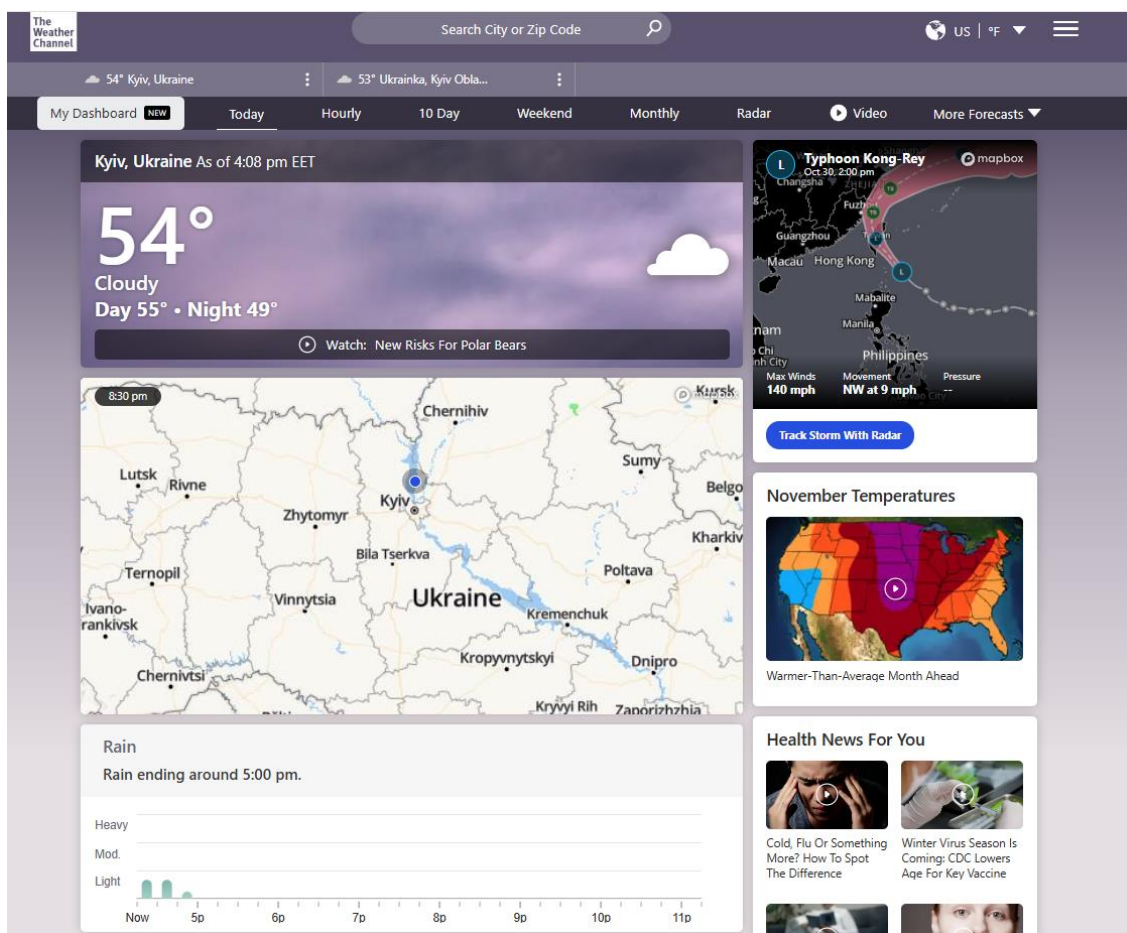


Рисунок 1.1 – UI сайту Weather

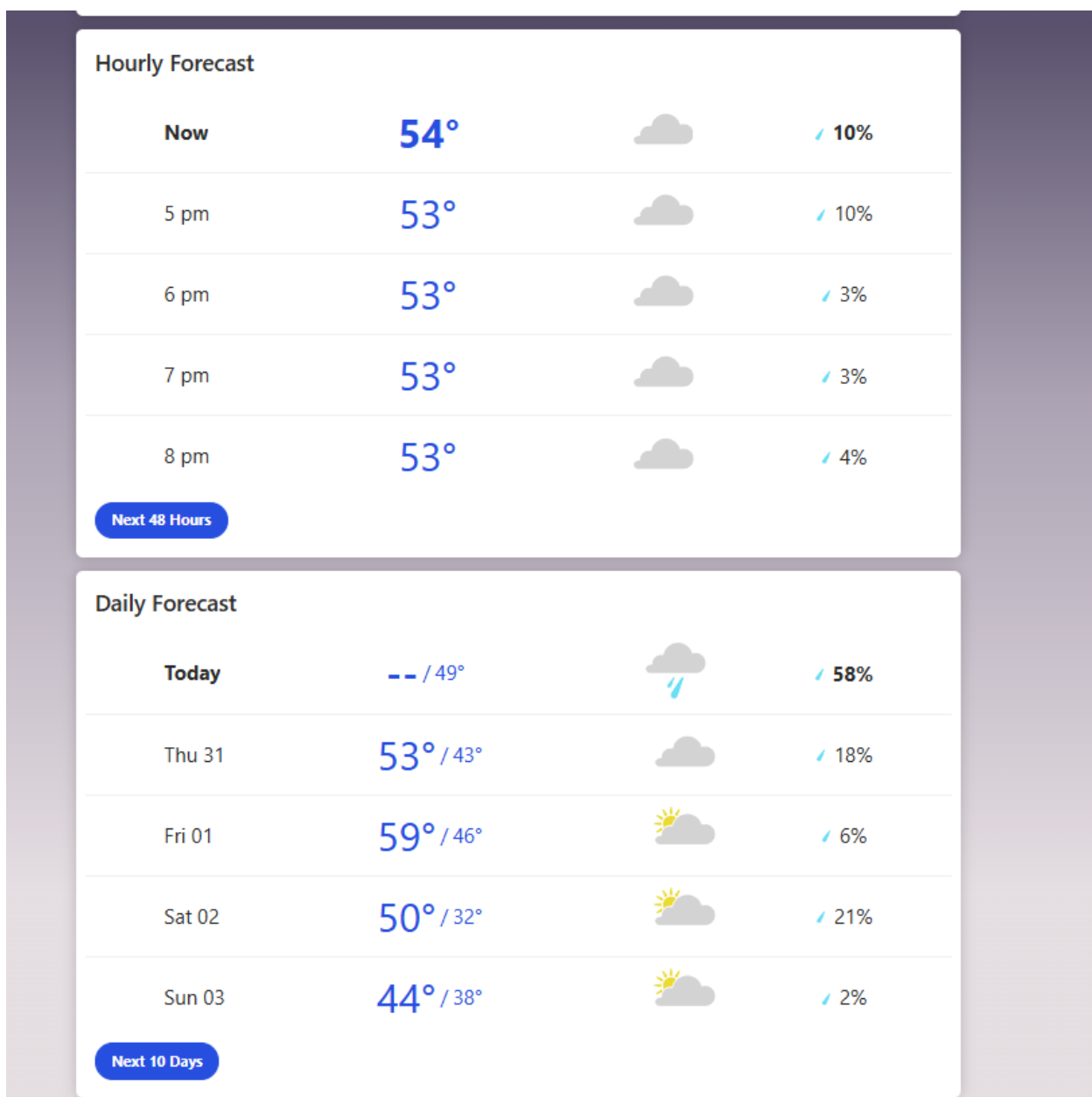


Рисунок 1.2 – Продовження UI сайту Weather

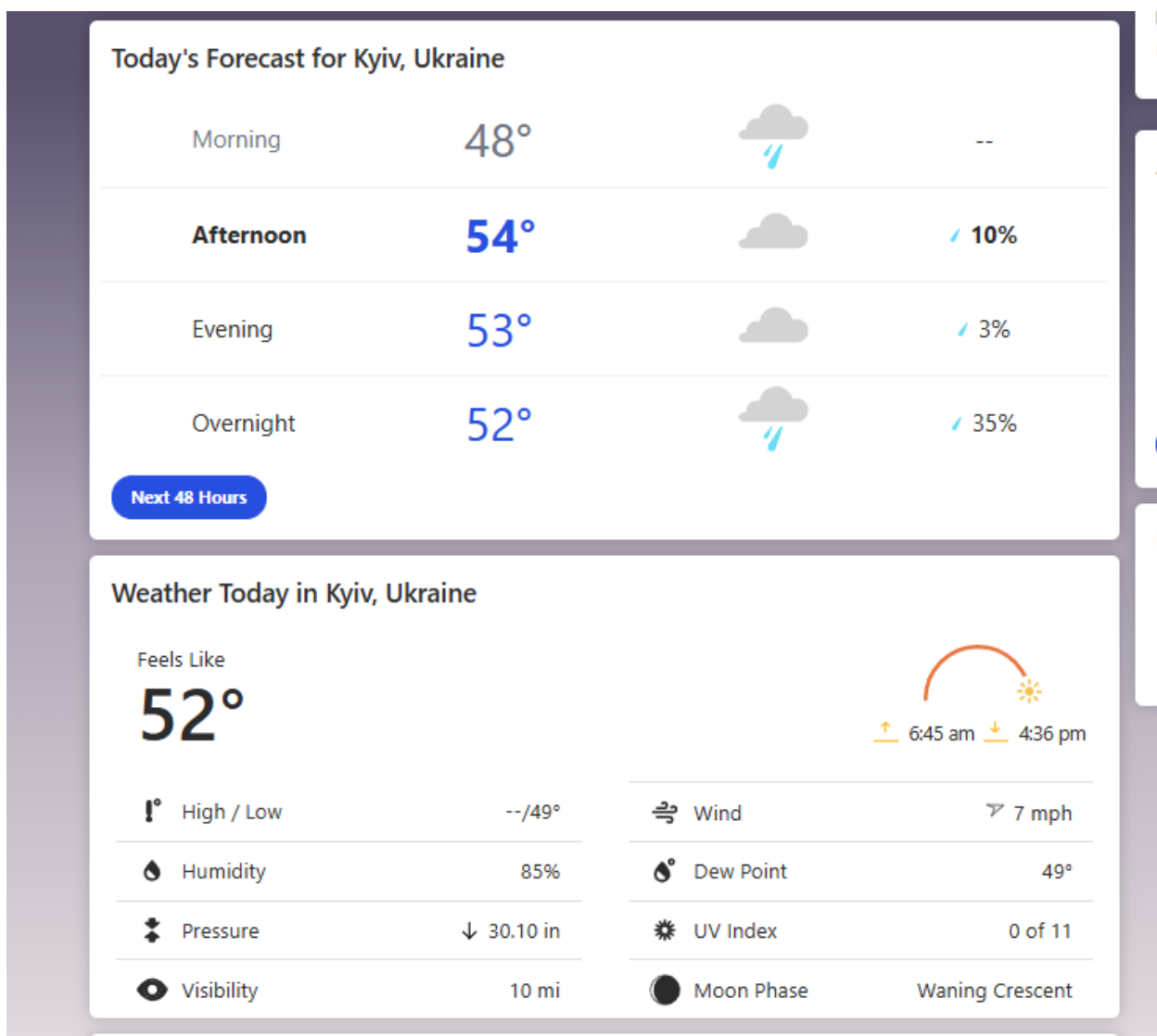


Рисунок 1.3 – Продовження UI сайту Weather

Другим продуктом є Accuweather [3]. Цей сервіс також є дуже зручним у використанні (рисунки 1.4-1.6) та надає детальні прогнози погоди, включаючи спеціалізовані прогнози для різних сфер життєдіяльності, таких як, господарство, туризм та багато іншого.

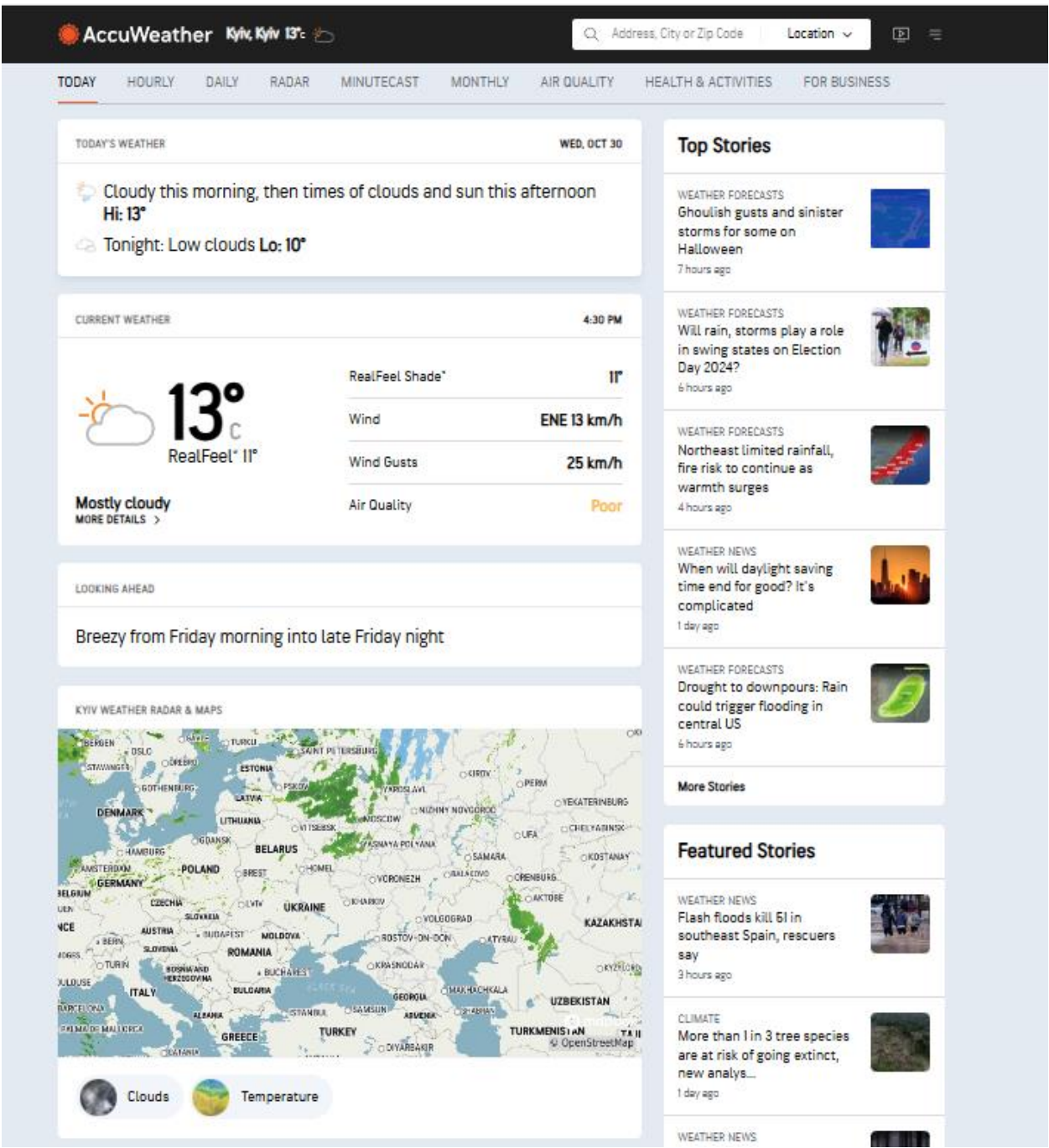


Рисунок 1.4 - UI сайту Accuweather

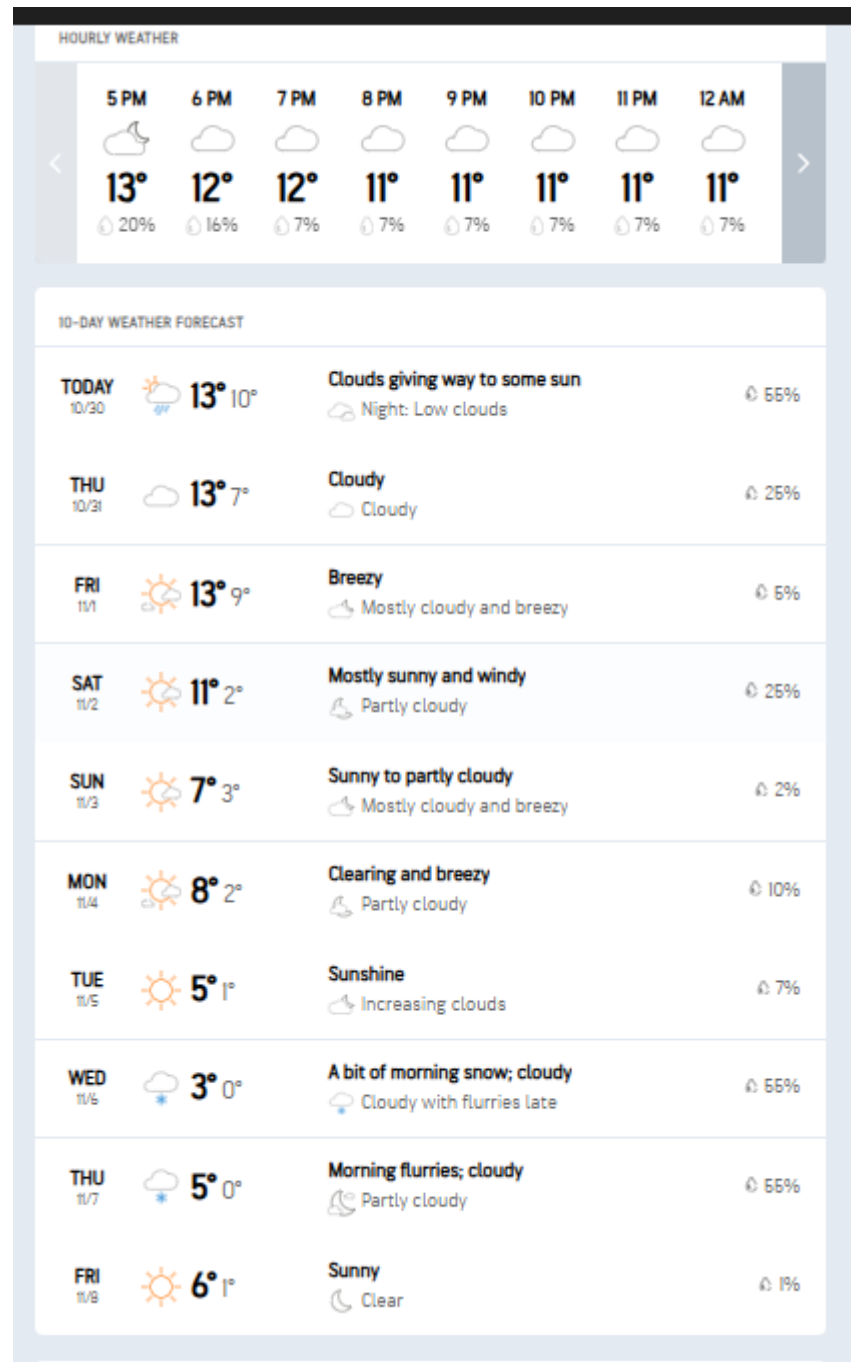


Рисунок 1.5 – Продовження UI сайту Accuweather

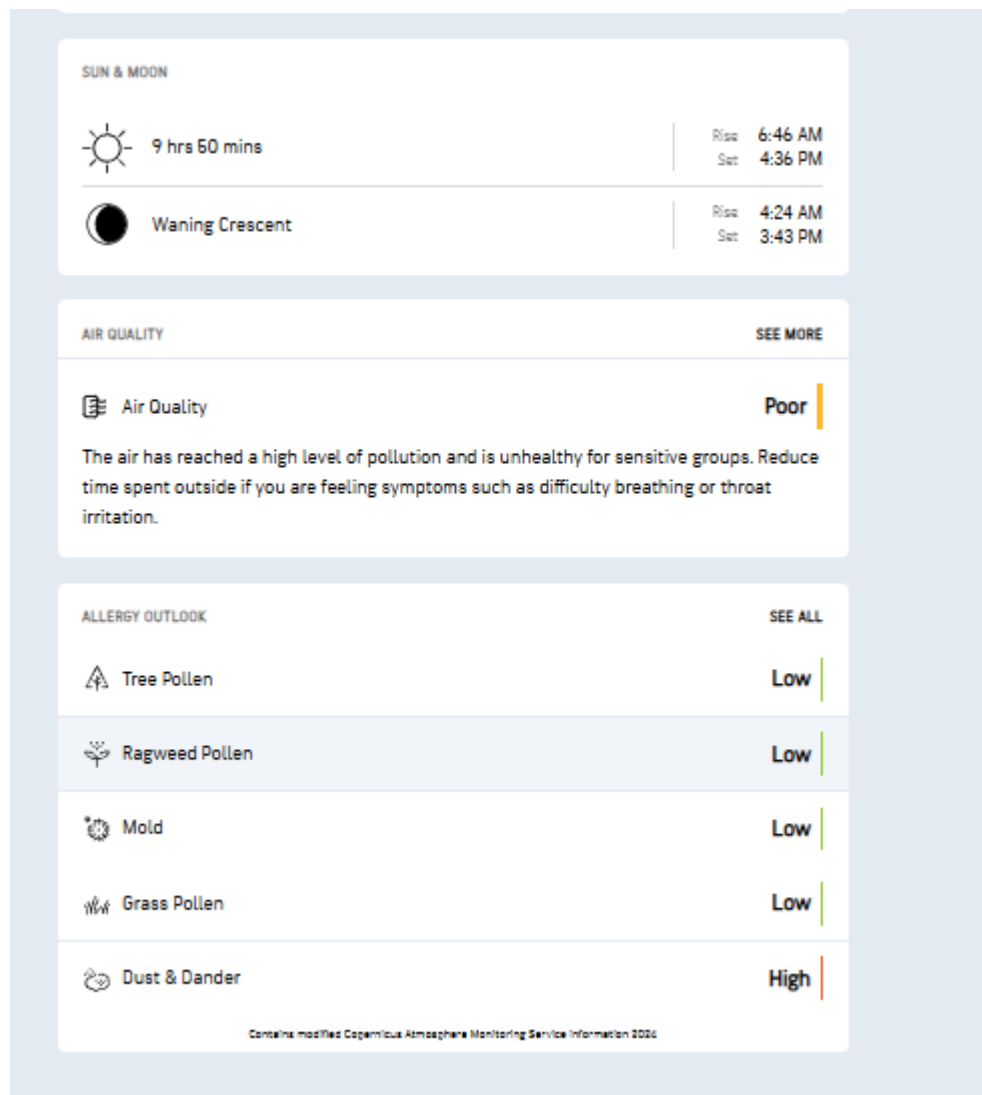


Рисунок 1.6 – Продовження UI сайту Accuweather

Третім сервісом є Worldweather [4], який є офіційним ресурсом Всесвітньої метеорологічної організації. Він акцентує увагу на актуальних погодних умовах та тенденціях. Інтерфейс користувача також досить зрозумілий (рисунок 1.7).

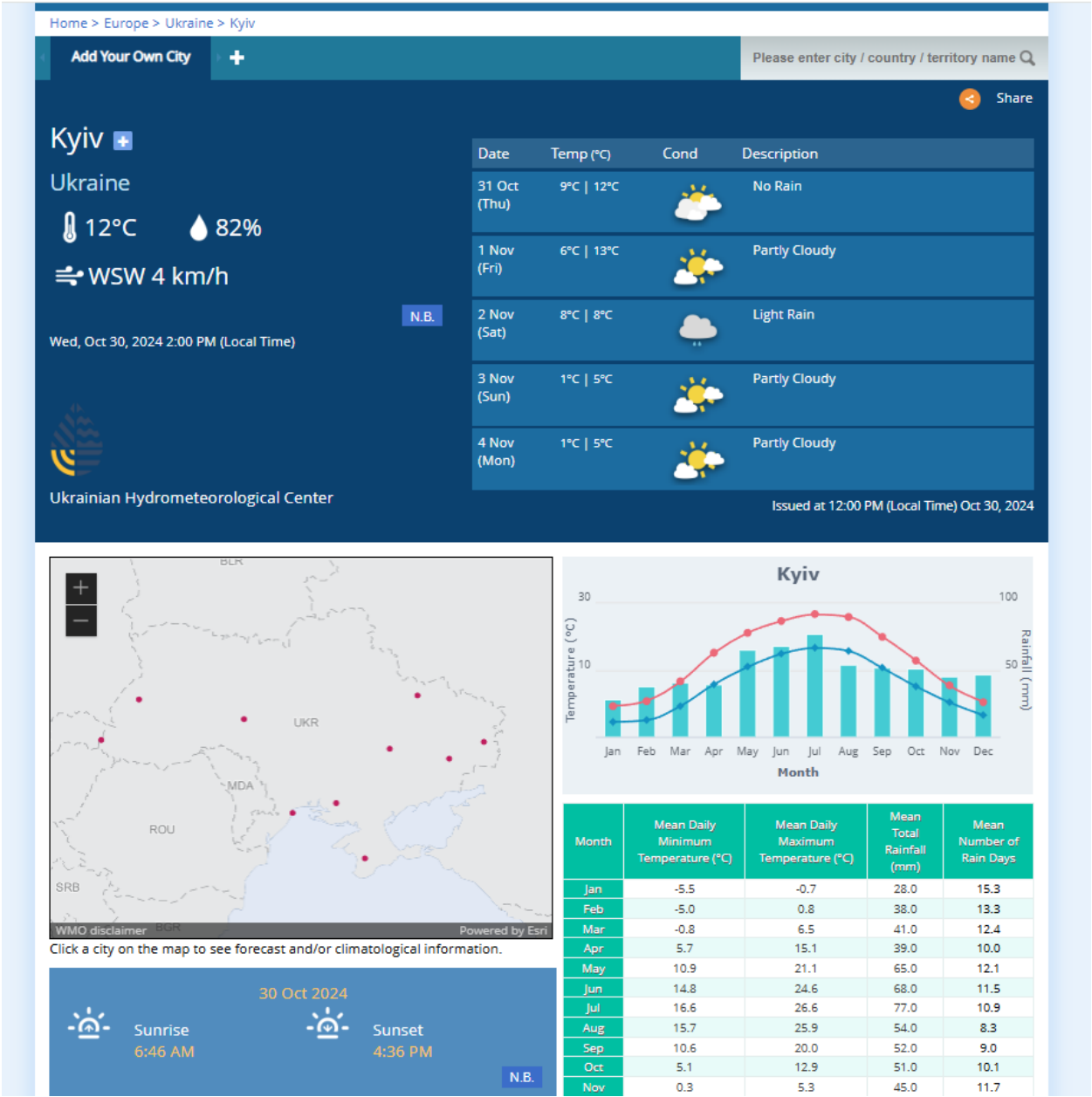


Рисунок 1.7 - UI сайту Worldweather

Більш узагальнене порівняння основних функцій сервісів наведено в таблиці 1.1.

Таблиця 1.1 – Порівняння сервісів

Функціонал	Проект в рамках курсової роботи «Веб-сервіс прогнозування погоди, шляхом агрегації даних онлайн гідрометцентрі в»	Weather	Accuweather	Worldweather	Пояснення
Агрегація даних різних центрів	Так	Ні	Ні	Ні	Проект в рамках курсової роботи об'єднує дані з різних джерел, що забезпечує ширший прогноз.

Продовження таблиці 1.1

Функціонал	Проект в рамках курсової роботи «Веб-сервіс прогнозування погоди, шляхом агрегації даних онлайн гідрометцентрі в»	Weather	Accuweather	Worldweather	Пояснення
Поточна погода	Так	Так	Так	Так	Всі сервіси надають поточну погоду.
Прогноз погоди на кілька днів	Так	Так	Так	Так	Всі сервіси надають прогноз на кілька днів.

Продовження таблиці 1.1

Функціонал	Проект в рамках курсової роботи «Веб-сервіс прогнозування погоди, шляхом агрегації даних онлайн гідрометцентрі в»	Weather	Accuweather	Worldweather	Пояснення
Прогноз погоди по годинам на кілька днів	Так	Так	Так	Ні	Один із сервісів не надає прогноз по годиннам на кілька днів.

1.2.2 Аналіз відомих алгоритмічних та технічних рішень

У процесі розробки веб-сервісу необхідно розглянути різні архітектурні підходи та типи хостингу.

Серед архітектурних моделей [9] було проаналізовано монолітну, мікросервісну, сервіс-орієнтовану та багаторівневу. Монолітна архітектура передбачає розміщення всієї логіки застосунку в одному кодовому блоку, що

спрощує розгортання, але ускладнює масштабування та підтримку. Мікросервісна архітектура розділяє застосунок на незалежні сервіси, кожен з яких відповідає за окрему функціональність, що полегшує масштабування та обслуговування, але вимагає складнішої інфраструктури. Сервіс-орієнтована архітектура фокусується на наданні функціональності через сервіси з чітко визначеними інтерфейсами, що дозволяє повторне використання та інтеграцію. Багаторівнева архітектура розділяє застосунок на рівні, такі як презентаційний, логіки застосунку та доступу до даних, що сприяє модульності та гнучкості.

Для реалізації веб-сервісу з агрегації прогнозів погоди було обрано мікросервісну архітектуру з поділом на клієнт-сервер. Це означає, що система має чіткий поділ на клієнтську частину та серверну частину, які взаємодіють через мережу за допомогою HTTP(S)-запитів. У той же час, серверна частина складається з декількох незалежних компонентів.

Щодо вибору хостингу, було розглянуто різні варіанти [10], включаючи віртуальний хостинг, VPS, хмарний хостинг та виділений сервер. Спільний хостинг є економічним варіантом, але обмежує контроль над серверним середовищем. Хмарний хостинг забезпечує високу масштабованість та доступність, але може бути дорожчим. Виділений сервер надає повний контроль, але вимагає значних ресурсів для обслуговування. Зрештою, було обрано VPS-хостинг, який забезпечує баланс між контролем, продуктивністю та вартістю. VPS дозволяє мати власне ізольоване середовище з можливістю налаштування системи, що відповідає потребам проєкту.

Таким чином, обрана архітектура та тип хостингу забезпечують ефективну реалізацію веб-сервісу з агрегації прогнозу погоди, дозволяючи досягти високої продуктивності, масштабованості та зручності в обслуговуванні.

1.3 Опис бізнес-процесів

Для опису основного бізнес-процесу отримання прогнозу погоди використовується BPMN [39] модель (рисунок 1.8).

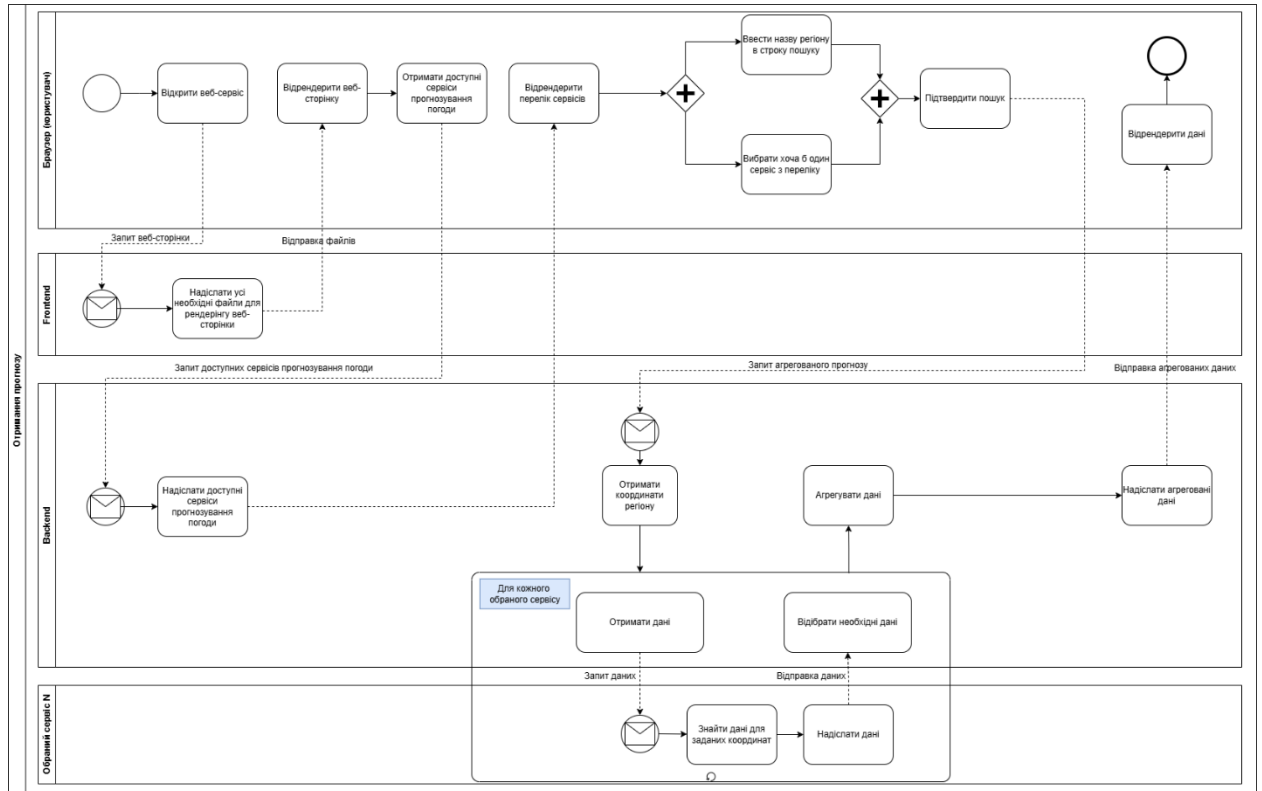


Рисунок 1.8 – Схема бізнес-процесу отримання прогнозу

Опис послідовності отримання прогнозу:

- користувач відкриває веб-сторінку сервісу (браузер робить запит до frontend-частини);
- веб-сторінка сервісу відображається на екрані користувача (браузер отримує файли від frontend-частини і рендерить їх);
- користувач отримує перелік доступних сервісів прогнозування погоди (браузер робить запит до backend-частини та рендерить відповідь);
- користувач вводить необхідне місто, обирає певні сервіси та підтверджує запит (браузер робить запит до backend-частини);
- backend-частина отримує назву міста, конвертує її в координати;

- backend-частина для кожного обраного сервісу робить запит до відповідного API, отримує дані та фільтрує тільки необхідну їй інформацію;
- backend-частина агрегує оброблені дані та відправляє користувачу (браузеру);
- прогноз відображається на екрані користувача (браузер рендерить отримані агреговані дані).

Висновки до розділу

У даному розділі було проведено детальне передпроектне дослідження предметної області, що стало основою для створення веб-сервісу з агрегації погодних даних.

Спершу було виконано аналіз предметної області, де було описано актуальність використання веб-застосунків. Було обґрунтовано необхідність у створенні додатку інтеграції різних сервісів прогнозування даних, що забезпечить більш точний і ширший прогноз погоди.

Наступним кроком був аналіз існуючих рішень, що включав огляд відомих сервісів, таких як Weather, Accuweather, Worldweather. Проаналізовано їхні можливості та недоліки з точки зору агрегації даних і представлено їх у таблиці. Це дало змогу визначити основні відмінності між сервісами та переконатися в унікальності запропонованого рішення.

Далі, для забезпечення стабільної роботи веб-сервісу, було обрано мікросервісну архітектуру із поділом на клієнтську та серверну частину, задля забезпечення масштабованості та контролю окремих сервісів. Як хостинг було обрано VPS, що надає баланс між контролем, продуктивністю та вартістю.

Крім того, у наступному розділі було описано бізнес-процеси сервісу за допомогою BPMN-моделі, що ілюструє повний цикл взаємодії користувача з додатком – від відкриття сторінки до відображення прогнозу. Ця модель

показує логіку обробки запиту користувача, вибір джерел, агрегацію та виведення результатів

Отже, проведене передпроектне обстеження дозволило сформулювати чітке уявлення про предметну область, існуючі аналоги, технічні рішення та бізнес-процеси, що стануть основою для подальшого формулювання вимог до програмного забезпечення.

2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Варіанти використання програмного забезпечення

Для опису системи прогнозування погоди, шляхом агрегації даних різних гідрометцентрів, було побудовано (рисунок 2.1) та описано (таблиця 2.1) діаграму варіантів використання в UML-нотації.

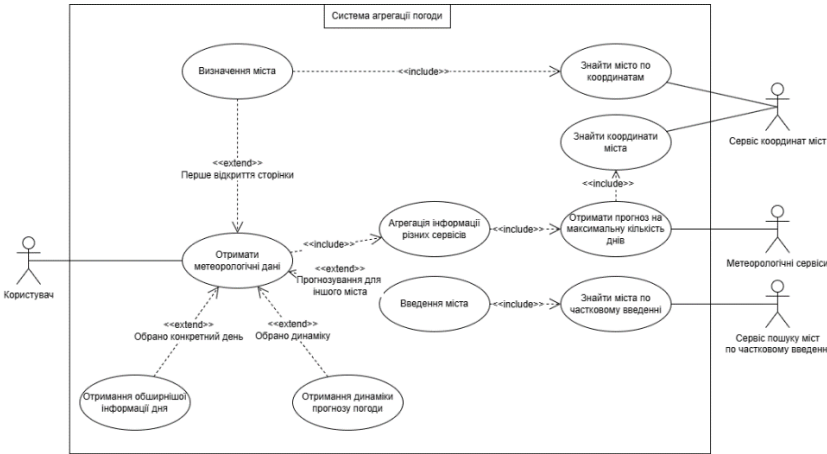


Рисунок 2.1 – Діаграма варіантів використання системи агрегації погоди

Таблиця 2.1 – Опис основного варіанту використання

Use case name	Отримати метеорологічні дані
Use case id	UC-00
Goals	– отримання поточної або прогнозованої погоди, агрегуючи дані різних сервісів; – отримання як загальної короткої поденної інформації, так і більш обшірної для конкретного дня; – передбачення можливості перегляду динаміки прогнозу погоди.

Продовження таблиці 2.1

Actors	Користувач, зовнішні метеорологічні сервіси, сервіс координат міст та сервіс пошуку міст по частковому введенні
Trigger	Користувач відкриває сторінку, підтверджуючи доступ до геолокації, або вводить місто вручну.
Pre-conditions	– доступ до Інтернету; – веб-сервіс запущено.
Flow of events	– користувач відкриває головну сторінку веб-сервісу; – система визначає або запитує місто користувача; – система надсилає запити до зовнішніх метеорологічних сервісів; – отримані дані агрегуються та форматуються; – результати виводяться у вигляді короткої інформації днів.

Продовження таблиці 2.1

Extension	– можливість автоматичного визначення міста, якщо користувач дозволить доступ; – можливість введення міста, якщо користувач не дозволив доступ до геолокації або потребує інше місто; – можливість перегляду більш обширної інформації для конкретного дня; – можливість перегляду динаміки прогнозу у вигляді графіків.
Post-Condition	Користувач бачить актуальну або прогнозовану погоду у вигляді короткої інформації для кожного дня, або більш обширної для конкретного дня, або динамічних графіків.

Для подальшого уточнення варіантів використання, також надамо унікальний ідентифікатор кожному із них (таблиця 2.2).

Таблиця 2.2 – Унікальні ідентифікатори варіантів використання

Назва	Унікальний ідентифікатор
Введення міста	UC-01
Визначення міста	UC-02
Отримання більш обширної інформації дня	UC-03

Продовження таблиці 2.2

Назва	Унікальний ідентифікатор
Отримання динаміки прогнозу погоди	UC-04
Агрегація інформації різних сервісів	UC-05
Отримати прогноз на максимальну кількість днів	UC-06
Знайти координати міста	UC-07
Знайти міста по частковому введенні	UC-08
Знайти місто по координатам	UC-09

2.2 Аналіз системних вимог

З огляду на обрані технічні рішення для реалізації веб-сервісу прогнозування погоди, до серверного середовища висуваються певні мінімальні та рекомендовані вимоги. Ці вимоги обумовлені використанням фреймворку FastAPI [15], кешуючого сховища Redis [27], системи контейнеризації Docker [24], а також зворотного серверного проксі Nginx [26], що виконує роль зворотного проксі-сервера та обслуговує статичні файли frontend-частини React [19].

Враховуючи ці компоненти, обґрунтуємо системні вимоги до серверу розгортання (таблиці 2.3-2.4).

Таблиця 2.3 – Мінімальні системні вимоги серверу

Компонент системи	Мінімальні вимоги	Обґрунтування
Процесор	1 vCPU	Мінімально необхідна кількість віртуальних процесорів для підтримки роботи компонентів, що мають високу швидкодію та не потребують значних ресурсів у базовій конфігурації.
Оперативна пам'ять	1 ГБ	Основні компоненти системи споживають незначну кількість пам'яті в стані простою, чого достатньо для прототипу.
Місце на диску	5 ГБ	Достатньо для роботи Docker, а також зберігання кешу, логів та інших даних контейнеризованих сервісів.
Операційна система	Ubuntu 20.04 або новіша	Легка, стабільна та поширена система з повною підтримкою Docker та зручністю адміністрування.

Продовження таблиці 2.3

Компонент системи	Мінімальні вимоги	Обґрунтування
Мережеве з'єднання	20 Мбіт/с	Забезпечує базову швидкість для передачі статичних файлів і обміну з зовнішніми сервісами.

Таблиця 2.4 – Рекомендовані системні вимоги серверу

Компонент системи	Рекомендовані вимоги	Обґрунтування
Процесор	4 vCPU	Оптимальна кількість віртуальних процесорів для забезпечення високої продуктивності Docker і всіх сервісів у паралельній роботі.
Оперативна пам'ять	4 ГБ	Додаткова пам'ять необхідна при активному використанні кешування в Redis та під час обробки запитів.
Місце на диску	10 ГБ	Розширене сховище для зберігання кешу, логів та інших даних.

Продовження таблиці 2.4

Мережеве з'єднання	100 Мбіт/с	Забезпечує швидку передачу даних і мінімізацію затримок при зверненні до сторонніх API.
--------------------	------------	---

2.3 Розроблення функціональних вимог

Програмне забезпечення розділене на модулі. Кожен модуль має свій певний набір функцій. В таблиці 2.5 наведено загальну модель вимог, а в таблиці 2.6 наведений опис функціональних вимог до програмного забезпечення. Матрицю трасування вимог можна побачити на рисунку 2.2.

Таблиця 2.5 – Загальна модель вимог

№	Назва	ID вимоги	Пріоритети	Ризики
1	Збір даних з різних джерел.	FR-1	Високий	Високий
2	Агрегація даних різних джерел.	FR-2	Високий	Високий
3	Отримання погодних даних за назвою населеного пункту.	FR-3	Високий	Високий
4	Перегляд узагальненого прогнозу.	FR-4	Високий	Високий
5	Перегляд прогнозу для конкретного дня.	FR-5	Високий	Високий

Продовження таблиці 2.5

№	Назва	ID вимоги	Пріоритети	Ризики
6	Візуалізація динаміки прогнозу.	FR-6	Середній	Середній
7	Можливість вибору періоду візуалізації динаміки.	FR-7	Середній	Середній
8	Можливість отримання підказок при введенні міста.	FR-8	Середній	Середній
9	Автоматичне визначення місцезнаходження.	FR-9	Низький	Низький

Таблиця 2.6 – Перелік функціональних вимог

Назва	Опис
FR-1	Збір даних з різних джерел. Автоматизувати збір даних із кожного використаного метеорологічних сервісів у певному форматі.

Продовження таблиці 2.6

Назва	Опис
FR-2	Агрегація даних різних джерел. Автоматизувати збір, агрегацію та форматування результатів прогнозу із різних джерел.
FR-3	Отримання погодних даних за назвою населеного пункту. Дозволити користувачу введення міста та обрання сервісів, використаних для агрегації.
FR-4	Перегляд узагальненого прогнозу. Можливість отримання користувачем короткої узагальненої інформації для кожного доступного дня.
FR-5	Перегляд прогнозу для конкретного дня. Можливість отримання користувачем обширного прогнозу для конкретного обраного дня.
FR-6	Візуалізація динаміки прогнозу. Можливість перегляду графіків динаміки прогнозу за різними метеорологічними параметрами.

Продовження таблиці 2.6

Назва	Опис
FR-7	Можливість вибору періоду візуалізації динаміки. Можливість отримання графіків не для усіх прогнозних днів, а для конкретного обраного періоду.
FR-8	Можливість отримання підказок при введенні міста. Можливість відображення усіх, схожих по введеній назві, міст, якщо такі існують.
FR-9	Автоматичне визначення місцезнаходження. Можливість визначення місцезнаходження користувача та автоматичного пошуку даних при першому запуску веб-сторінки та підтвердженні доступу до геолокації.

	FR-1	FR-2	FR-3	FR-4	FR-5	FR-6	FR-7	FR-8	FR-9
UC-00	+	+	+	+	+	+	+	+	+
UC-01			+					+	
UC-02									+
UC-03					+				
UC-04						+	+		
UC-05		+							
UC-06	+								
UC-07		+							
UC-08								+	
UC-09									+

Рисунок 2.2 – Матриця трасування

2.4 Розроблення нефункціональних вимог

Сервіс повинен задовільняти певним нефункціональним вимогам, тобто характеристикам, які визначають не функціональність, а властивості роботи системи. Основні нефункціональні вимоги до системи агрегації погоди наведено в таблиці 2.7.

Таблиця 2.7 – Нефункціональні вимоги системи

Тип	Вимоги
Доступність	– веб-сервіс має бути доступним не менше 99% часу; – середній час відновлення сервісу або його частини має виконуватися до 5 секунд.
Продуктивність	– коректна робота при навантаженні до 100 одночасних користувачів; – для швидкого Інтернет з'єднання середній час відповіді – 1 секунда, для повільного – до 10 секунд.
Обмеження	– система має забезпечити коректне функціонування для користувача зі швидкістю Інтернет-з'єднання не менше 8 Мбіт/с.
Масштабованість	– можливість ручного горизонтального масштабування сервісів за допомогою Docker Compose; – можливість ручного надання більших ресурсів кожному сервісу за допомогою Docker Compose.

Продовження таблиці 2.7

Тип	Вимоги
Вимоги до зручності використання	– інтерфейс користувача має бути інтуїтивним та зрозумілим, тобто не вимагати попереднього навчання; – для доступу до будь-якого функціоналу веб-застосунку має бути достатньо не більше 3 кроків.

2.5 Постановка задачі

Задача даної курсової роботи полягає в розробці веб-сервісу для прогнозування погоди шляхом агрегації даних з різних онлайн-сервісів. На вхід система приймає запит користувача з обраною локацією, а також список метеосервісів, які користувач бажає використати для отримання прогнозу. У результаті користувач отримує узагальнений прогноз погоди, що включає різні метеорологічні показники, такі як температура, вологість, швидкість вітру, тиску, погодні умови тощо.

Метою даної розробки є створення зручного інструменту для надання інтегрованих погодних прогнозів, який дозволяє ефективніше планувати діяльність, залежну від погодних умов. Застосування декількох джерел даних підвищує точність і повноту інформації, а їхнє комбінування дає змогу формувати більш достовірний прогноз.

Ключовими задачами розробки є забезпечення автоматичного визначення місцезнаходження користувача, введення та допомога вибору населеного пункту, надсилання запитів до зовнішніх API, агрегація та обробка даних, отриманих із різних джерел, а також відображення результатів у

зрозумілому та зручному для користувача вигляді із можливістю отримання динамічних графіків. Важливою є й реалізація швидкодії, стабільності та інтуїтивно зрозумілого інтерфейсу користувача.

Висновки до розділу

У даному розділі було сформовано повний набір вимог до майбутнього веб-сервісу агрегації погодних даних, що є ключовим етапом у процесі його розробки. Спочатку було представлено варіанти використання програмного забезпечення у вигляді діаграми та опису основних сценаріїв взаємодії користувача із системою. Це дозволило структурувати функціональність додатку та визначити спосіб взаємодії.

Далі проведено аналіз системних вимог, з урахуванням обраних технологій, таких як FastAPI, React, Docker, Redis, Nginx. Було чітко окреслено мінімальні та рекомендовані параметри серверного середовища, необхідні для стабільної та ефективної роботи сервісу.

Окремо було опрацьовано функціональні вимоги, які охоплюють усі ключові можливості системи. Вимоги було систематизовано у таблицях з вказаними пріоритетами та потенційними ризиками, що дозволяє ефективніше планувати реалізацію кожного компонента. Також було створено матрицю трасування для контролю відповідності реалізації заявленим вимогам.

Далі у розділі було визначено нефункціональні вимоги до системи, що забезпечують створення надійного, швидкого та інтуїтивно зрозумілого продукту, здатного відповідати очікуванням кінцевих користувачів.

Окрему увагу було приділено постановці задачі, де було узагальнено принцип роботи веб-застосунку, його мету та коротко описано вимоги для подальшої розробки.

Таким чином, у цьому розділі було повністю сформовано технічну, функціональну та нефункціональну основу програмного забезпечення, яка

стане фундаментом для подальшого проєктування архітектури, реалізації логіки та тестування веб-сервісу прогнозування погоди.

3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Архітектура програмного забезпечення

Архітектура програмного забезпечення побудована з урахуванням поділу на frontend та backend частини, взаємодіючи між собою у рамках клієнт-серверної моделі. Такий підхід дозволяє чітко розмежувати функціональність, що відповідає вимогам масштабованості, підтримованості та продуктивності системи.

Frontend реалізований як односторінковий застосунок на основі бібліотеки React, що забезпечує динамічну роботу з інтерфейсом користувача. Backend реалізовано за допомогою FastAPI і відповідає за головну логіку: допомога автоматичного визначення міста, підказки при введенні міста та агрегація погодніх даних із різних джерел, а також працює із кешуванням. Для зберігання проміжних результатів використовується Redis, що знижує навантаження та прискорює відповіді. Усі компоненти працюють в окремих контейнерах з використанням Docker, що спрощує тестування та розгортання. Nginx виконує роль зворотного проксі-сервера та забезпечує маршрутизацію запитів до відповідних сервісів.

Для представлення архітектури розроблені діаграми моделі С4. Високорівнева структура системи представлена на рівнях 1 та 2 (рисунки 3.1-3.2), більш детальна організація логіки на рівні 3 (рисунок 3.3), а на рівні 4 (рисунок 3.4) зображено модель коду backend-частини.

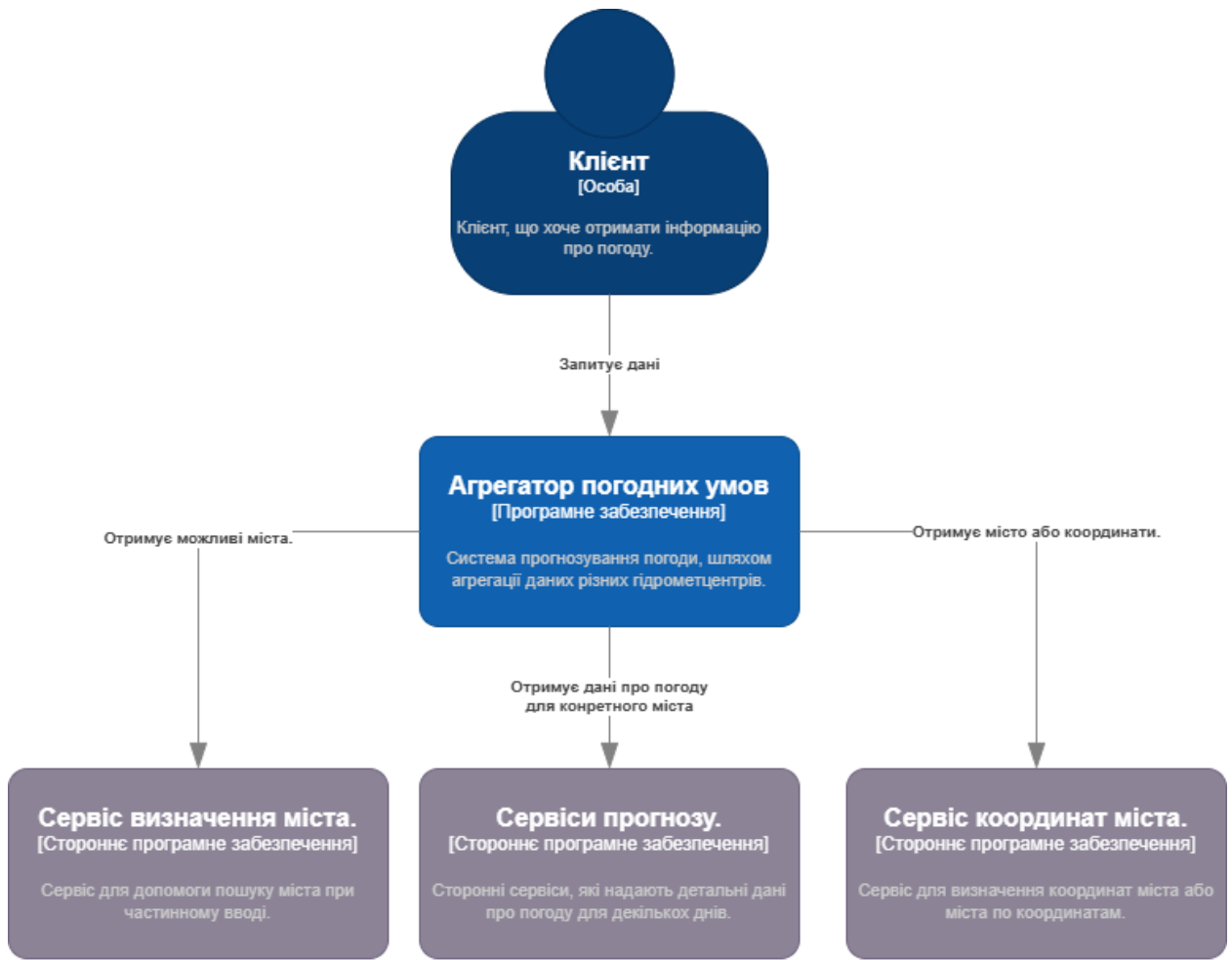


Рисунок 3.1 – Діаграма С4, рівень 1

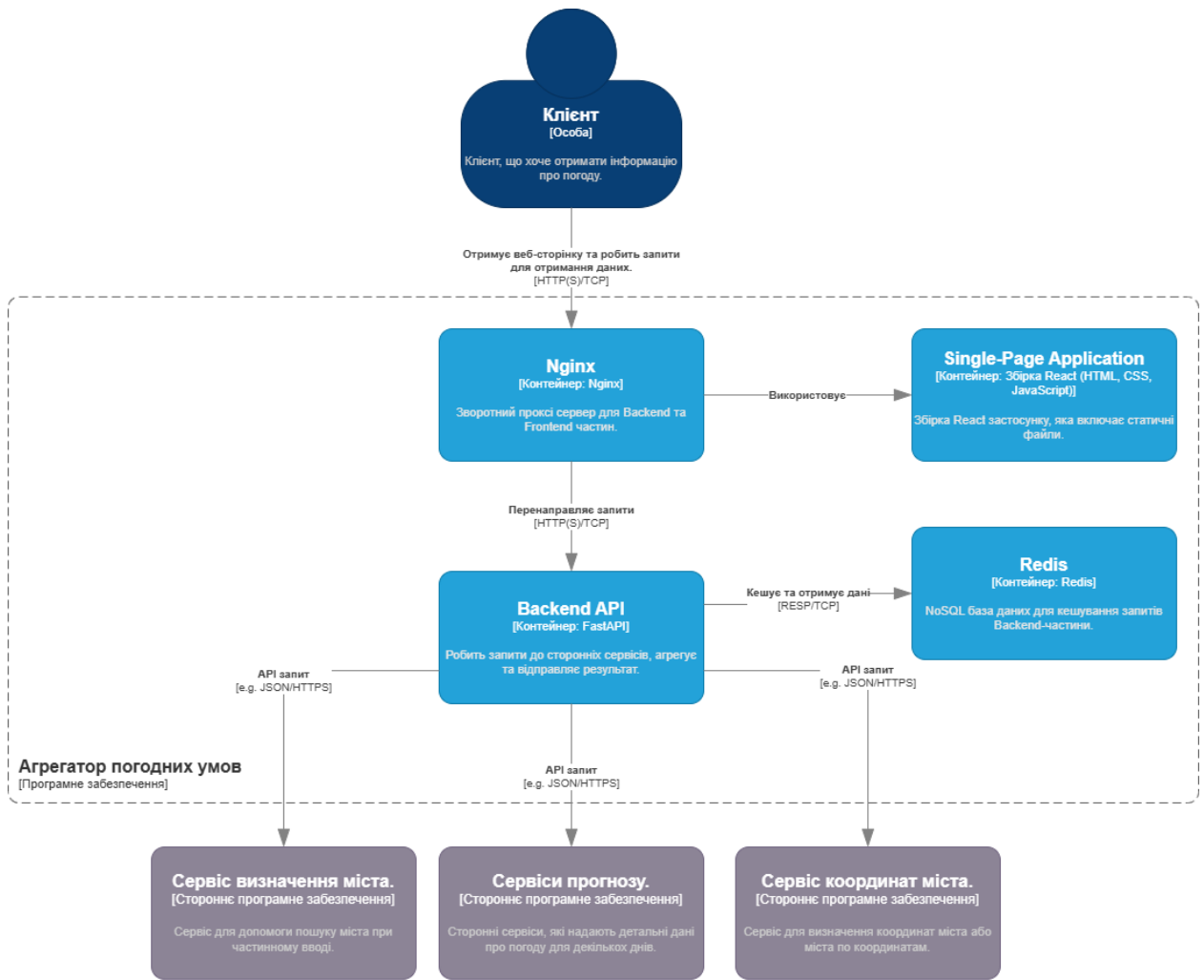


Рисунок 3.2 – Діаграма C4, рівень 2

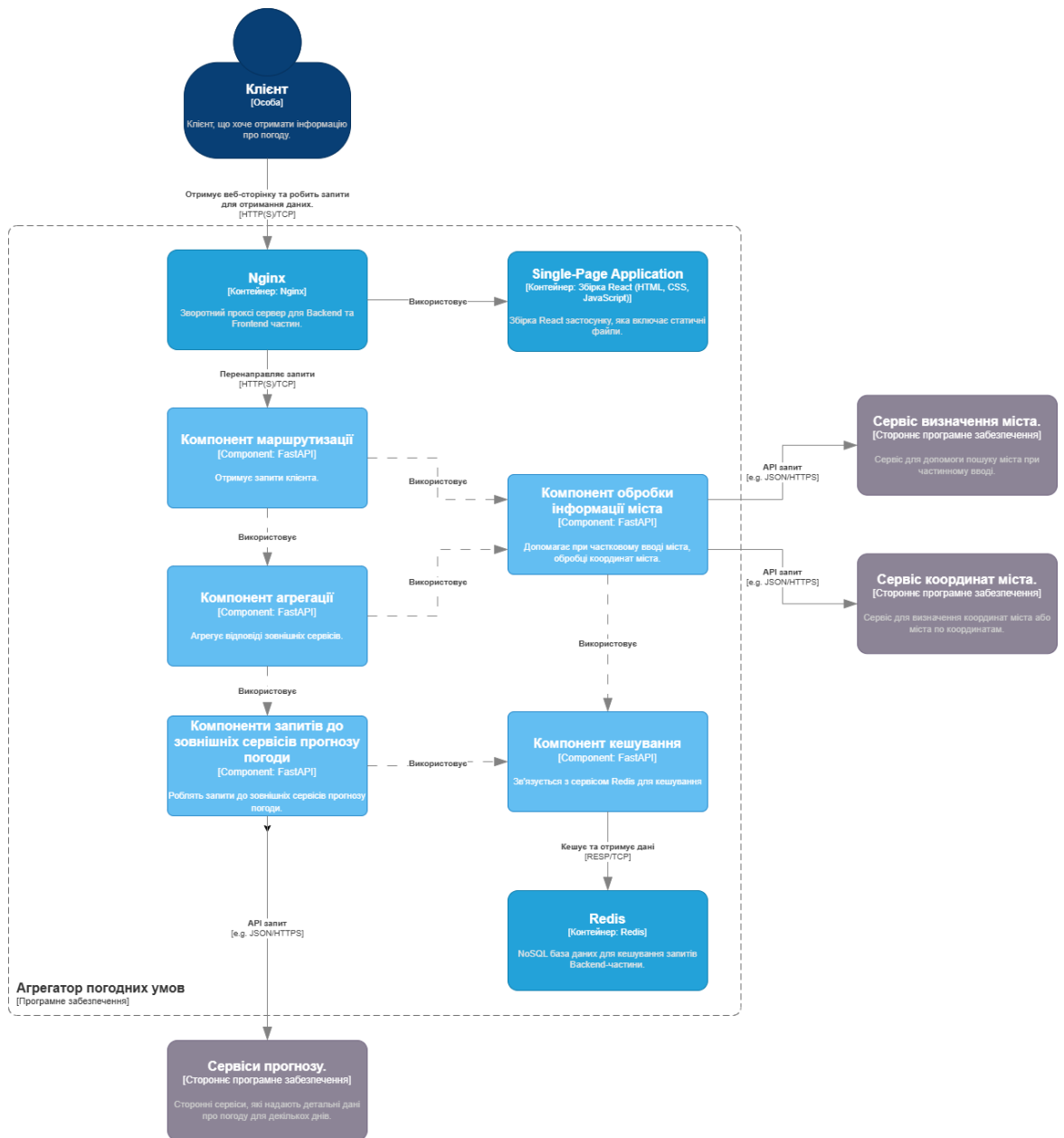


Рисунок 3.3 – Діаграма C4, рівень 3

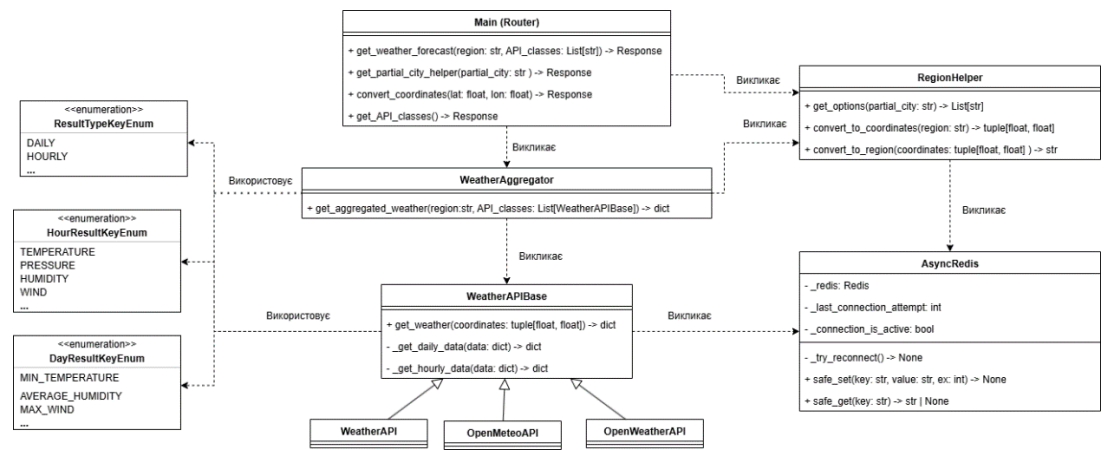


Рисунок 3.4 – Діаграма C4, рівень 4

3.2 Обґрунтування вибору засобів розробки

Для розробки веб-сервісу було обрано технології, що забезпечують високу продуктивність, зручність у розробці та масштабованість.

Основною мовою програмування для серверної частини є Python [11], оскільки він має велику різноманітність бібліотек і фреймворків для будь-яких задач. Python також підтримує асинхронне програмування, що є важливим для роботи із запитами до сторонніх сервісів. Інші варіанти, такі як Java [12], PHP [14], Go [13], хоча і є популярними для серверної частини веб-додатків, виявились не такими швидкими в плані розробки.

Для розробки веб-сервісу було обрано FastAPI [15] як основний фреймворк для Python. FastAPI дозволяє швидко створювати ефективні REST API з високою продуктивністю завдяки асинхронному підходу, що робить його оптимальним вибором для проєктів, які обробляють багато запитів одночасно. Альтернативи, такі як Flask [16] або Django [17], не забезпечують таке ж поєднання продуктивності та зручності розробки API.

Для клієнтської частини веб-сервісу був обраний JavaScript [18] з використанням бібліотеки React [19]. Це одна з найбільш популярних технологій для створення інтерактивних користувацьких інтерфейсів, що дозволяє ефективно працювати з компонентами та оновленням даних у реальному часі. Vue[20] і Angular[21] також є хорошими альтернативами, але React має більшу екосистему та кращу підтримку серед розробників, що робить його найбільш універсальним варіантом для цього проєкту. Для стилізації було обрано Tailwind [22], оскільки цей фреймворк дозволяє швидко та гнучко створювати адаптивний дизайн без потреби написання великих обсягів кастомних стилів, що є перевагою порівняно з використанням Bootstrap [23], що пропонує обмежені готові компоненти.

Для кешування даних і забезпечення високої швидкості доступу до часто використовуваної інформації був обраний Redis [27]. Він надає можливість

швидкого збереження та доступу до даних у пам'яті, що значно прискорює роботу веб-сервісу. В порівнянні з такими альтернативними, як Memcached [28] або MongoDB [29], Redis є більш популярним та швидкісним варіантом.

Для керування веб-запитами, забезпечення високої доступності та безпеки був обраний Nginx [26]. Цей веб-сервер є потужним інструментом для балансування навантаження, розподілу запитів та обробки статичних файлів. Nginx забезпечує високу продуктивність при великій кількості одночасних з'єднань. Він дозволяє оптимізувати використання серверних ресурсів, оскільки працює з асинхронною обробкою запитів, що зменшує навантаження на сервер і підвищує швидкість реакції. При порівнянні із аналогами, такими як Apache [35], Nginx сильно виграє в швидкодії та легкості налаштування.

Для контейнеризації застосунку було обрано Docker [24], оскільки він дозволяє створювати ізольовані середовища для запуску різних частин сервісу, що спрощує налаштування та розгортання проєкту на різних системах. Інші рішення, такі як Kubernetes [25], є надмірними для цього проєкту, оскільки потребують складної оркестрації контейнерів, що не є необхідним для поточного рівня складності.

Для розробки веб-сервісу було обрано середовище розробки Visual Studio Code [36]. Це популярний редактор коду, що пропонує потужний інструмент, з можливістю встановлювати безліч плагінів, для підтримки розробки на мовах Python, JavaScript та відповідних бібліотек, а також інтеграцією з Docker. Інші IDE, такі як PyCharm [37] чи WebStorm [38], можуть бути потужнішими в певних аспектах, проте для даного проєкту, з використанням різних технологій, будуть не найкращим вибором.

Вибір сервісів для агрегації прогнозів погоди був обумовлений необхідністю забезпечити функціональність без значних обмежень по кількості запитів. Було обрано три погодні сервіси: Openweathermap [30], Weatherapi [31] та Open-meteo [32], які надають подібні необхідні можливості, але мають безкоштовну версію з достатньо хорошими обмеженнями в

порівнянні із такими аналогами, як наприклад Tomorrow [33] або Weatherstack [34]. Також було обрано сервіс Geocodemap, який надає безкоштовну можливість як звичайного геокодування, так і зворотнього, а також може допомагати при частковому введенні міста.

3.3 Конструювання програмного забезпечення

3.3.1 Опис сторонніх сервісів

Функціонування веб-сервісу базується на взаємодії з кількома зовнішніми API для отримання метеорологічних даних та інформації про географічне розташування.

Сервіс Geocodemap [47] використовується для виконання операцій геокодування: перетворення назви міста на географічні координати, отримання підказок при введенні часткової назви міста та зворотнього геокодування (визначення назви міста по координатам). Це ключовий сервіс, що дозволяє уніфікувати запити до погодних API, перетворюючи назву в координати, що однаково сприймаються кожним із сервісів. Інтерфейс Geocodemap зображено на рисунку 3.5. Обов'язковим параметром API є `api_key` – унікальний ключ наданий для кожного користувача сервісу. Для зворотнього геокодування надаються також `lat` (широта) та `lon` (довгота), а в результаті отримаємо назву міста та іншу інформацію (рисунок 3.6). Для отримання можливих варіантів міст та їх координат по частковому введенні назви, необхідно передати її як параметр `q`. В результаті отримаємо варіанти можливих міст та координати для кожного з них (рисунок 3.7).

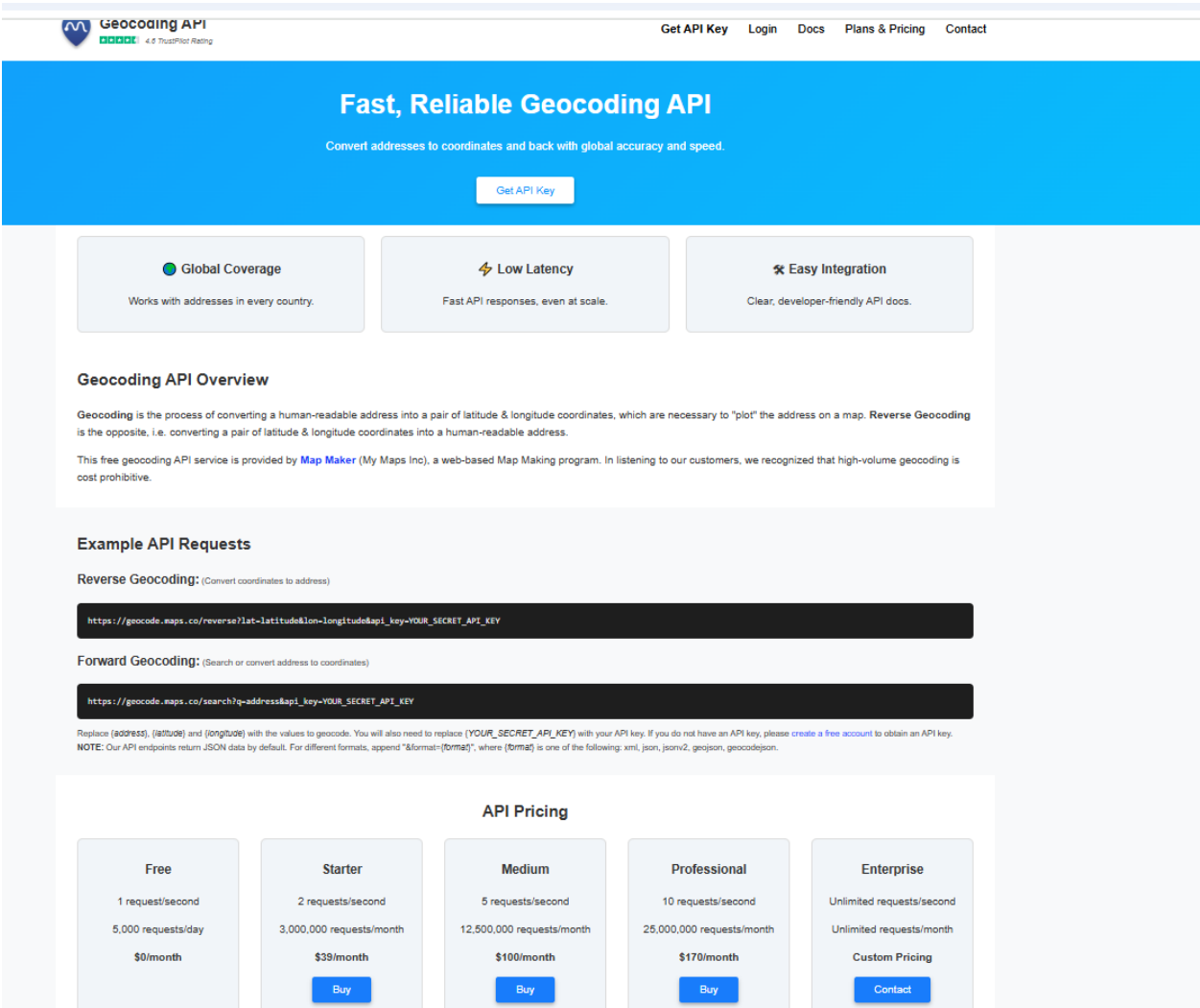


Рисунок 3.5 – Интерфейс Geocodemaps

```

{
  "place_id": 211659005,
  "licence": "Data © OpenStreetMap contributors, ODbL 1.0. http://osm.org/copyright",
  "osm_type": "relation",
  "osm_id": 5513390,
  "lat": "49.7238365",
  "lon": "69.56605900811016",
  "class": "boundary",
  "type": "administrative",
  "place_rank": 12,
  "importance": 0.435997717445363,
  "address_type": "county",
  "name": "Нұра ауданы",
  "display_name": "Нұра ауданы, Қарағанды облысы, Қазақстан",
  "address": {
    "county": "Нұра ауданы",
    "state": "Қарағанды облысы",
    "ISO3166-2-lvl4": "KZ-35",
    "country": "Қазақстан",
    "country_code": "kz"
  },
  "boundingbox": [
    "48.6435968",
    "50.8040763",
    "67.3991195",
    "72.0296315"
  ]
}

```

Рисунок 3.6 – Результат зворотнього геокодування

```
[
  {
    "place_id": 188468739,
    "licence": "Data © OpenStreetMap contributors, ODbL 1.0. http://osm.org/copyright",
    "osm_type": "relation",
    "osm_id": 421866,
    "lat": "50.4500336",
    "lon": "30.5241361",
    "class": "boundary",
    "type": "administrative",
    "place_rank": 8,
    "importance": 0.785673452883558,
    "addresstype": "city",
    "name": "Київ",
    "display_name": "Київ, Україна",
    "boundingbox": [
      "50.2132422",
      "50.5908142",
      "30.2361453",
      "30.8263593"
    ]
  },
  {
    "place_id": 188459944,
    "licence": "Data © OpenStreetMap contributors, ODbL 1.0. http://osm.org/copyright",
    "osm_type": "relation",
    "osm_id": 71248,
    "lat": "50.178595",
    "lon": "30.4924884",
    "class": "boundary",
    "type": "administrative",
    "place_rank": 8,
    "importance": 0.670323449338236,
    "addresstype": "state",
    "name": "Київська область",
    "display_name": "Київська область, Україна",
    "boundingbox": [
      "49.1787440",
      "51.5531668",
      "29.2675489",
      "32.1610264"
    ]
  }
]
```

Рисунок 3.7 – Результат геокодування по частковому введенні

Наступним сервісом є Openweathermap – одним із найбільш популярних сервісів, що надає широкий спектр метеорологічних даних. У проєкті використовується його кінцева точка «One Call API 3.0», яка за одним запитом надає поточну погоду, погодинний прогноз на 48 годин та денний прогноз на 8 днів. Інтерфейс сервісу представлений на рисунку 3.8. Обов'язковими параметрами для отримання прогнозу погоди є `appid`, що є унікальним ключом, наданим для кожного користувача, `lat` (широта) та `lon` (довгота). Приклад результату наведено на рисунках 3.9-3.10.

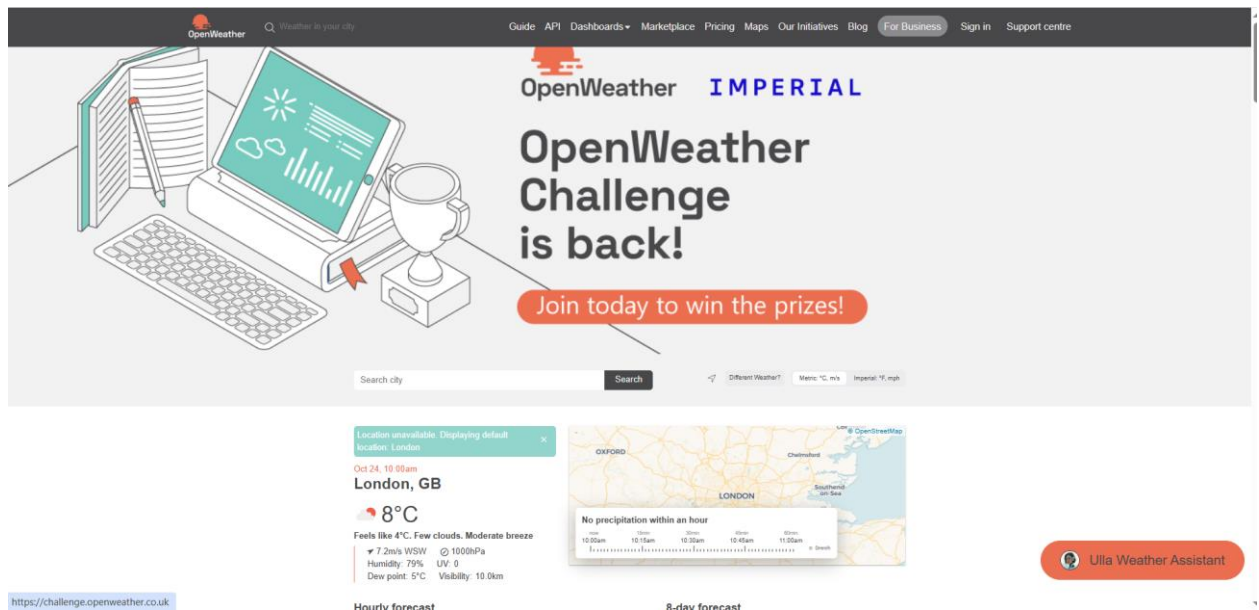


Рисунок 3.8 – Интерфейс Openweathermap.

```

{
  "lat":33.44,
  "lon":-94.04,
  "timezone":"America/Chicago",
  "timezone_offset":-18000,
  "current":{
    "dt":1684929490,
    "sunrise":1684926645,
    "sunset":1684977332,
    "temp":292.55,
    "feels_like":292.87,
    "pressure":1014,
    "humidity":89,
    "dew_point":290.69,
    "uvi":0.16,
    "clouds":53,
    "visibility":10000,
    "wind_speed":3.13,
    "wind_deg":93,
    "wind_gust":6.71,
    "weather":[
      {
        "id":803,
        "main":"Clouds",
        "description":"broken clouds",
        "icon":"04d"
      }
    ]
  },
  "minutely":{
    {
      "dt":1684929540,
      "precipitation":0
    },
    ...
  ],
  "hourly":{
    {
      "dt":1684926000,
      "temp":292.01,
      "feels_like":292.33,
      "pressure":1014,
      "humidity":91,
      "dew_point":290.51,
      "uvi":0,
      "clouds":54,
      "visibility":10000,
      "wind_speed":2.58,
      "wind_deg":86,
      "wind_gust":5.88,
      "weather":[
        {
          "id":803,
          "main":"Clouds",
          "description":"broken clouds",
          "icon":"04n"
        }
      ],
      "pop":0.15
    },
    ...
  ],
  ...
}

```

Рисунок 3.9 – Результат прогнозу Openweathermap (частина 1)

```

    ],
    "daily": [
      {
        "dt": 1684951200,
        "sunrise": 1684926645,
        "sunset": 1684977332,
        "moonrise": 1684941060,
        "moonset": 1684905480,
        "moon_phase": 0.16,
        "summary": "Expect a day of partly cloudy with rain",
        "temp": {
          "day": 299.03,
          "min": 290.69,
          "max": 300.35,
          "night": 291.45,
          "eve": 297.51,
          "morn": 292.55
        },
        "feels_like": {
          "day": 299.21,
          "night": 291.37,
          "eve": 297.86,
          "morn": 292.87
        },
        "pressure": 1016,
        "humidity": 59,
        "dew_point": 290.48,
        "wind_speed": 3.98,
        "wind_deg": 76,
        "wind_gust": 8.92,
        "weather": [
          {
            "id": 500,
            "main": "Rain",
            "description": "light rain",
            "icon": "10d"
          }
        ],
        "clouds": 92,
        "pop": 0.47,
        "rain": 0.15,
        "uvi": 9.23
      },
      ...
    ],
    "alerts": [
      {
        "sender_name": "NWS Philadelphia - Mount Holly (New Jersey, Delaware, Southeastern Pennsylvania)",
        "event": "Small Craft Advisory",
        "start": 1684952747,
        "end": 1684988747,
        "description": "...SMALL CRAFT ADVISORY REMAINS IN EFFECT FROM 5 PM THIS\nAFTERNOON TO 3 AM EST FRIDAY...\n* WHAT...North winds 15 to 20 kt with gusts up to 25 kt and seas\n3 to 5 ft expected.\n* WHERE...Coastal waters from Little Egg Inlet to Great Egg\nInlet NJ out 20 nm, Coastal waters from Great Egg Inlet to\nCape May NJ out 20 nm and Coastal waters from Manasquan Inlet\nto Little Egg Inlet NJ out 20 nm.\n* WHEN...From 5 PM this afternoon to 3 AM EST Friday.\n* IMPACTS...Conditions will be hazardous to small craft.",
        "tags": [
          ]
        },
        ...
      ]
    ]
  }

```

Рисунок 3.10 - Результат прогнозу Openweathermap (частина 2)

Weatherapi – це ще один комплексний сервіс, що надає доступ до поточних погодних умов та прогнозів. У веб-застосунку використовується кінцева точка «Forecast API», яка дозволяє отримати детальний прогноз на термін до 14 днів, включаючи погодинні дані. Інтерфейс сервісу представлено на рисунку 3.11. Основними параметрами знову є унікальний ключ – key, а також q, що відповідає або за координати, або за назву. Приклад результату представлено на рисунках 3.12-3.14.

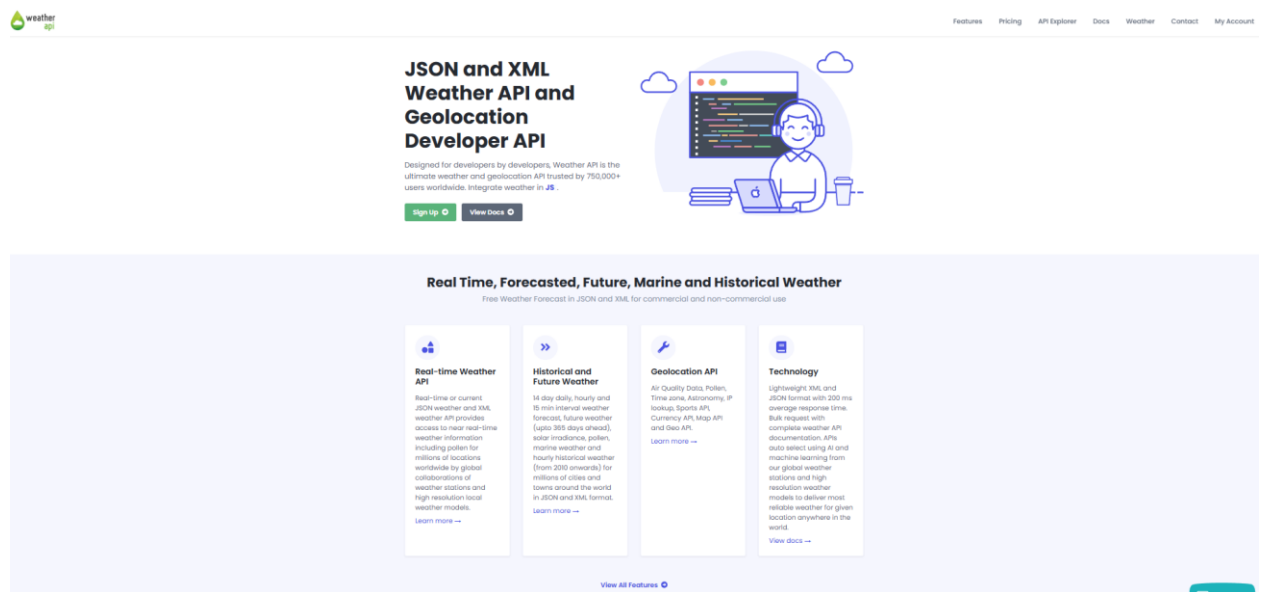


Рисунок 3.11 – Інтерфейс Weatherapi

```

{
  "location": {
    "name": "London",
    "region": "City of London, Greater London",
    "country": "United Kingdom",
    "lat": 51.5171,
    "lon": -0.1062,
    "tz_id": "Europe/London",
    "localtime_epoch": 1761296263,
    "localtime": "2025-10-24 09:57"
  },
  "current": {
    "last_updated_epoch": 1761295500,
    "last_updated": "2025-10-24 09:45",
    "temp_c": 7.1,
    "temp_f": 44.8,
    "is_day": 1,
    "condition": {
      "text": "Partly Cloudy",
      "icon": "///cdn.weatherapi.com/weather/64x64/day/116.png",
      "code": 1003
    },
    "wind_mph": 11.6,
    "wind_kph": 18.7,
    "wind_degree": 243,
    "wind_dir": "WSW",
    "pressure_mb": 1000.0,
    "pressure_in": 29.53,
    "precip_mm": 0.0,
    "precip_in": 0.0,
    "humidity": 81,
    "cloud": 0,
    "feelslike_c": 3.9,
    "feelslike_f": 38.9,
    "windchill_c": 6.0,
    "windchill_f": 42.8,
    "heatindex_c": 8.8,
    "heatindex_f": 47.9,
    "dewpoint_c": 3.9,
    "dewpoint_f": 38.9,
    "vis_km": 10.0,
    "vis_miles": 6.0,
    "uv": 0.2,
    "gust_mph": 15.4,
    "gust_kph": 24.7
  },
}

```

Рисунок 3.12 – Результат прогнозу Weatherapi (частина 1)

```

"forecast": {
  "forecastday": [
    {
      "date": "2025-10-24",
      "date_epoch": 1761264000,
      "day": {
        "maxtemp_c": 12.6,
        "maxtemp_f": 54.7,
        "mintemp_c": 6.7,
        "mintemp_f": 44.1,
        "avgtemp_c": 9.4,
        "avgtemp_f": 48.9,
        "maxwind_mph": 15.0,
        "maxwind_kph": 24.1,
        "totalprecip_mm": 1.34,
        "totalprecip_in": 0.05,
        "totalsnow_cm": 0.0,
        "avgvis_km": 9.6,
        "avgvis_miles": 5.0,
        "avghumidity": 73,
        "daily_will_it_rain": 1,
        "daily_chance_of_rain": 93,
        "daily_will_it_snow": 0,
        "daily_chance_of_snow": 0,
        "condition": {
          "text": "Patchy rain nearby",
          "icon": "//cdn.weatherapi.com/weather/64x64/day/176.png",
          "code": 1063
        },
        "uv": 0.3
      },
      "astro": {
        "sunrise": "07:41 AM",
        "sunset": "05:48 PM",
        "moonrise": "11:17 AM",
        "moonset": "06:26 PM",
        "moon_phase": "Waxing Crescent",
        "moon_illumination": 6,
        "is_moon_up": 0,
        "is_sun_up": 0
      },
      "hour": [

```

Рисунок 3.13 – Результат прогнозу Weatherapi (частина 2)

```

    "hour": [
      {
        "time_epoch": 1761260400,
        "time": "2025-10-24 00:00",
        "temp_c": 7.4,
        "temp_f": 45.4,
        "is_day": 0,
        "condition": {
          "text": "Clear ",
          "icon": "//cdn.weatherapi.com/weather/64x64/night/113.png",
          "code": 1000
        },
        "wind_mph": 13.6,
        "wind_kph": 22.0,
        "wind_degree": 266,
        "wind_dir": "W",
        "pressure_mb": 999.0,
        "pressure_in": 29.51,
        "precip_mm": 0.0,
        "precip_in": 0.0,
        "snow_cm": 0.0,
        "humidity": 75,
        "cloud": 7,
        "feelslike_c": 3.9,
        "feelslike_f": 39.1,
        "windchill_c": 3.9,
        "windchill_f": 39.1,
        "heatindex_c": 7.4,
        "heatindex_f": 45.4,
        "dewpoint_c": 3.4,
        "dewpoint_f": 38.1,
        "will_it_rain": 0,
        "chance_of_rain": 0,
        "will_it_snow": 0,
        "chance_of_snow": 0,
        "vis_km": 10.0,
        "vis_miles": 6.0,
        "gust_mph": 20.2,
        "gust_kph": 32.5,
        "uv": 0}
    ]
  }
}

```

Рисунок 3.14 – Результат прогнозу Weatherapi (частина 3)

Останнім сервісом є Open-meteo – повністю безкоштовний погодніи сервіс з відкритими даними, що надає високу гнучкість у налаштуванні

запитів. На відміну від інших сервісів, він дозволяє користувачу самому обирати, які саме метеорологічні параметри включати у відповідь, що дозволяє оптимізувати обсяг отриманих даних. Інтерфейс сервісу представлено на рисунку 3.15. Сервіс не потребує API-ключа, тому необхідними є тільки параметри lat (широта) та lon (довгота). Приклад результату наведено на рисунках 3.16-3.17.

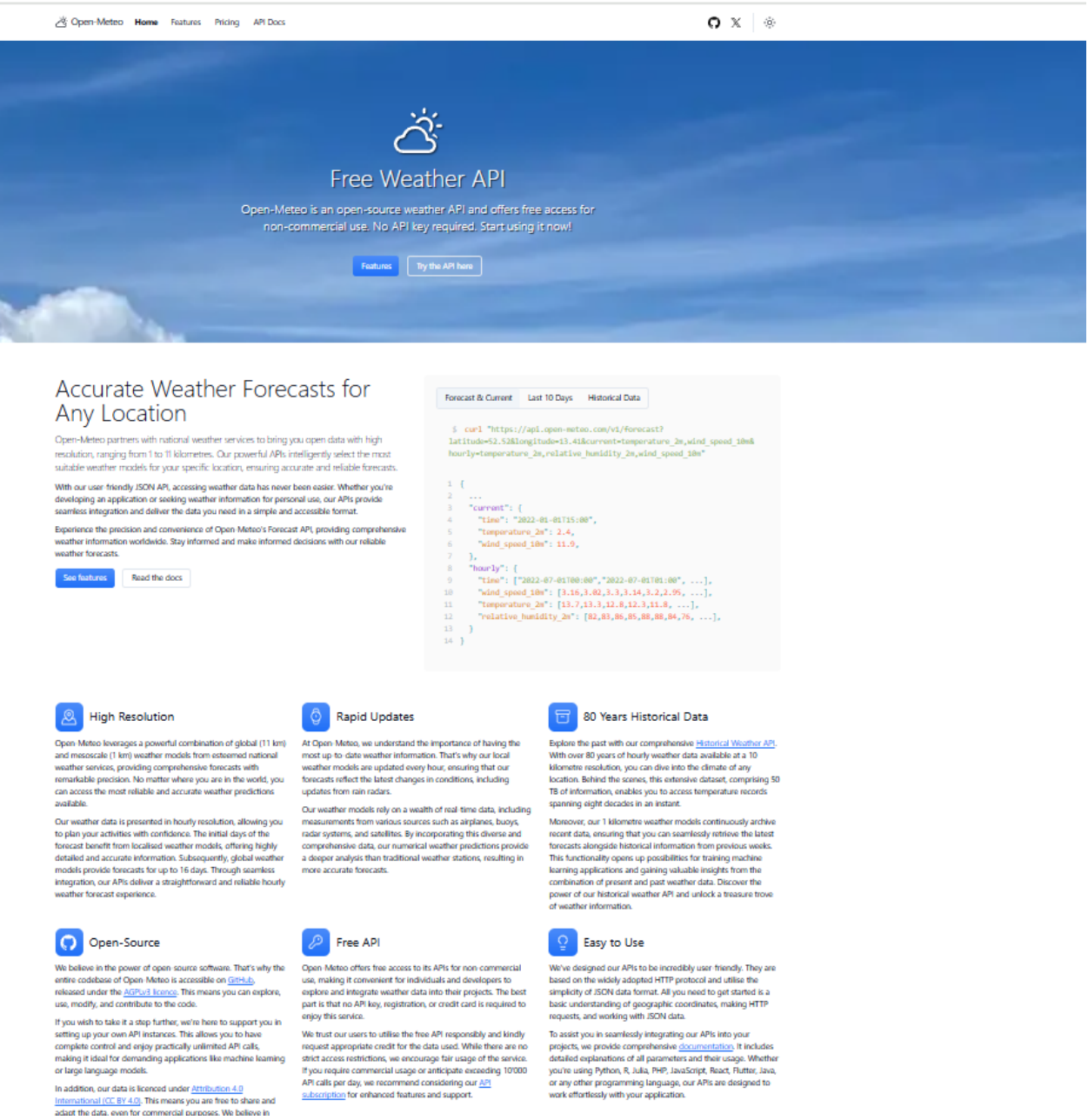


Рисунок 3.15 – Інтерфейс Open-meteo

```

{
  "latitude": 49.9375,
  "longitude": 50,
  "generationtime_ms": 0.352740287780762,
  "utc_offset_seconds": 18000,
  "timezone": "Asia/Oral",
  "timezone_abbreviation": "GMT+5",
  "elevation": 6,
  "hourly_units": {
    "time": "iso8601",
    "pressure_msl": "hPa",
    "weather_code": "wmo code",
    "temperature_2m": "°C",
    "relative_humidity_2m": "%",
    "wind_speed_10m": "km/h"
  },
  "hourly": {
    "time": [
      "2025-10-24T00:00",
      "2025-10-24T01:00",
      "2025-10-24T02:00",
      "2025-10-24T03:00",
      "2025-10-24T04:00",
      "2025-10-24T05:00",
      "2025-10-24T06:00",
      "2025-10-24T07:00",
      "2025-10-24T08:00",
      "2025-10-24T09:00",
      "2025-10-24T10:00",
      "2025-10-24T11:00",
      "2025-10-24T12:00",
      "2025-10-24T13:00",
      "2025-10-24T14:00",
      "2025-10-24T15:00",
      "2025-10-24T16:00",
      "2025-10-24T17:00",
      "2025-10-24T18:00",
      "2025-10-24T19:00",
      "2025-10-24T20:00",
      "2025-10-24T21:00",
      "2025-10-24T22:00",
      "2025-10-24T23:00",
      "2025-10-25T00:00",
      "2025-10-25T01:00",
      "2025-10-25T02:00",
      "2025-10-25T03:00",
      "2025-10-25T04:00",
      "2025-10-25T05:00",
      "2025-10-25T06:00",
      "2025-10-25T07:00",
      "2025-10-25T08:00",
      "2025-10-25T09:00",
      "2025-10-25T10:00",
      "2025-10-25T11:00",
      "2025-10-25T12:00",
      "2025-10-25T13:00",
      "2025-10-25T14:00",
      "2025-10-25T15:00",
      "2025-10-25T16:00",
      "2025-10-25T17:00",
      "2025-10-25T18:00",
      "2025-10-25T19:00",
      "2025-10-25T20:00",
      "2025-10-25T21:00"
    ]
  }
}

```

Рисунок 3.16 - Результат прогнозу Open-meteo (частина 1)

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Рисунок 3.17 - Результат прогнозу Open-meteo (частина 2)

3.3.2 Опис утіліт, бібліотек та іншого стороннього програмного забезпечення

Опис основних утиліт, бібліотек та іншого стороннього програмного забезпечення, що використовується у розробці наведено в таблиці 3.1.

Таблиця 3.1 – Опис утиліт, бібліотек та іншого стороннього ПЗ

Частина застосунку	№ п/п	Назва	Опис застосування
Backend	1	FastAPI	Асинхронний веб-фреймворк для створення REST API на мові Python. Використовується для обробки HTTP-запитів від клієнтської частини.
	2	Httpx [41]	Асинхронна бібліотека для виконання HTTP-запитів до зовнішніх сервісів.

Продовження таблиці 3.1

Частина застосунку	№ п/п	Назва	Опис застосування
Backend	3	Redis [42]	Бібліотека, що включає асинхронний модуль для взаємодії з in-memory сховищем Redis, що використовується для реалізації механізму кешування.
Frontend	4	React	Бібліотека JavaScript для створення динамічних та інтерактивних користувацьких інтерфейсів.
	5	React Router [43]	Бібліотека для реалізації маршрутизації в односторінковому додатку, що дозволяє навігацію між сторінками прогнозу, динаміки тощо.
	6	Recharts [44]	Бібліотека для створення діаграм та графіків, що використовується для відображення динаміки погодних показників.
	7	Tailwind CSS	CSS-фреймворк для швидкої та гнучкої стилізації інтерфейсу користувача без написання додаткового коду в CSS-файлах.

Продовження таблиці 3.1

Частина застосунку	№ п/п	Назва	Опис застосування
Розгортання	8	Docker (з Docker Compose)	Платформа для контейнеризації, що дозволяє ізолювати та запускати frontend, backend, redis у вигляді окремих сервісів, які запускаються однаково на будь-якій машині із встановленим Docker, що спрощує розгортання.
	9	Nginx	Веб-сервер, що використовується як зворотній проксі для маршрутизації запитів до backend-частини та для віддачі статичних файлів frontend-частини у робочому середовищі
Середовище розробки та тестування	10	VS Code	Головне середовище розробки програмного забезпечення.
	11	Postman [45]	Програмне забезпечення для тестування запитів до кінцевих точок backend-частини.
	12	SonarQube [46]	Програмне забезпечення для тестування якості коду проєкту.

Висновки до розділу

У даному розділі було детально описано процес конструювання та розробки програмного забезпечення для веб-сервісу прогнозування погоди, що є ключовим етапом реалізації проєкту.

Спочатку було представлено архітектуру програмного забезпечення, що базується на клієнт-серверній моделі з чітким поділом на frontend та backend частини. Для візуалізації архітектури на різних рівнях абстракції було використано діаграми моделі C4, які ілюструють як загальну структуру системи та її взаємодію із зовнішніми сервісами (рівні 1-2), так і внутрішню компонентну будову та модель коду backend-частини (рівні 3-4).

Далі було наведено обґрунтування вибору технологічного стеку. Для серверної частини було обрано Python з асинхронним фреймворком FastAPI, що забезпечує високу продуктивність. Клієнтська частина реалізована на JavaScript з використанням бібліотеки React для створення динамічних інтерфейсів. Для кешування даних було інтегровано Redis, а для розгортання та керування інфраструктурою – Docker та Nginx.

У рамках конструювання програмного забезпечення було надано детальний опис сторонніх сервісів, що є джерелами даних для системи: Geocodemaps, Openweathermap, Weatherapi та Open-meteo. Для кожного сервісу було описано його призначення, вхідні та вихідні дані, що дозволяє зрозуміти принципи інтеграції. На завершення було систематизовано опис ключових бібліотек, утиліт та іншого стороннього ПЗ, що було використано у проєкті.

Таким чином, у цьому розділі було представлено повну картину технічної реалізації проєкту: від архітектури до конкретних інструментів. Обрані рішення дозволили створити надійний, масштабований та продуктивний веб-сервіс, що повністю відповідає функціональним та нефункціональним вимогам, визначеним у попередньому розділі.

4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Аналіз якості ПЗ

Аналіз якості програмного забезпечення є комплексним процесом, що оцінює відповідність продукту як вимогам користувача, так і стандартам розробки.

Метриками для оцінки якості ПЗ обрано наступні:

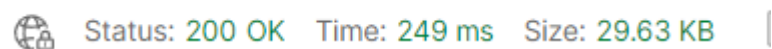
- продуктивність (швидкість відповіді при звичайному режимі та під навантаженням);
- доступність (час безвідмовної роботи та швидкість відновлення, при відмові будь-якого з необхідних сервісів);
- зручність використання (інтуїтивність інтерфейсу);
- підтримуваність (якість коду).

Для перевірки продуктивності backend-частини застосуємо сервіс Postman. Спочатку протестуємо середній час відгуку без навантаження та отримаємо результати, які приблизно дорівнюють 1100 та 250 мс без заповненого кешу та із ним відповідно (рисунки 4.1-4.2). Далі протестуємо середній час відгуку та кількість помилок для 100 користувачів одночасно (рисунок 4.3) та отримаємо досить хороші результати, де кількість помилок, хоч й на початку досить велика, проте спадає пізніше й доходить до 0%. Усі помилки були викликані через обмеження безкоштовної версії API.



Status: 200 OK Time: 1128 ms Size: 29.63 KB

Рисунок 4.1 – Швидкість відповіді без кешу



Status: 200 OK Time: 249 ms Size: 29.63 KB

Рисунок 4.2 – Швидкість відповіді з кешем

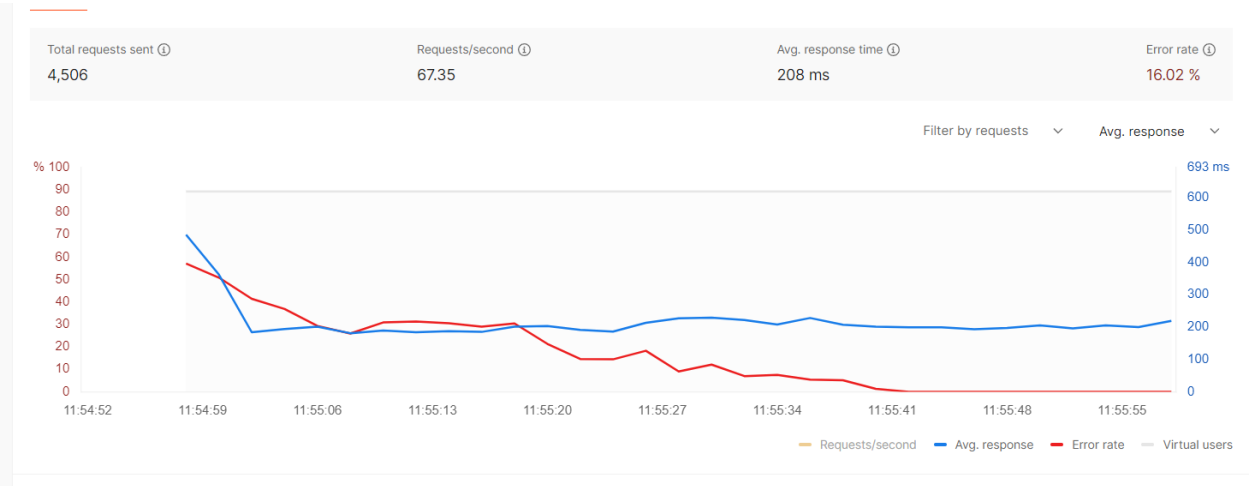


Рисунок 4.3 – Швидкість та кількість помилок при роботі під навантаженням

Тепер перевіримо frontend-частину як на швидкість, так і на зручність виокристання. Почнемо із аналізу швидкості та зручності, використовуючи вбудований механізм в браузер – Lighthouse. Після тестування отримано дуже хороші результати (рисунок 4.4). Також будь-який функціонал може бути досягненим за три кроки, що є досить зручним, а завантаження при повільному Інтернет з’єднанні в середньому займає 7.5 секунд (рисунок 4.5), що є достатнім результатом.

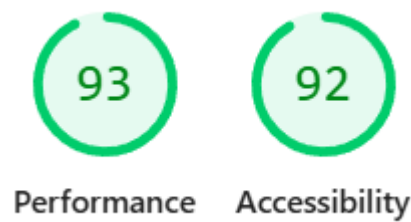


Рисунок 4.4 – Результати Lighthouse

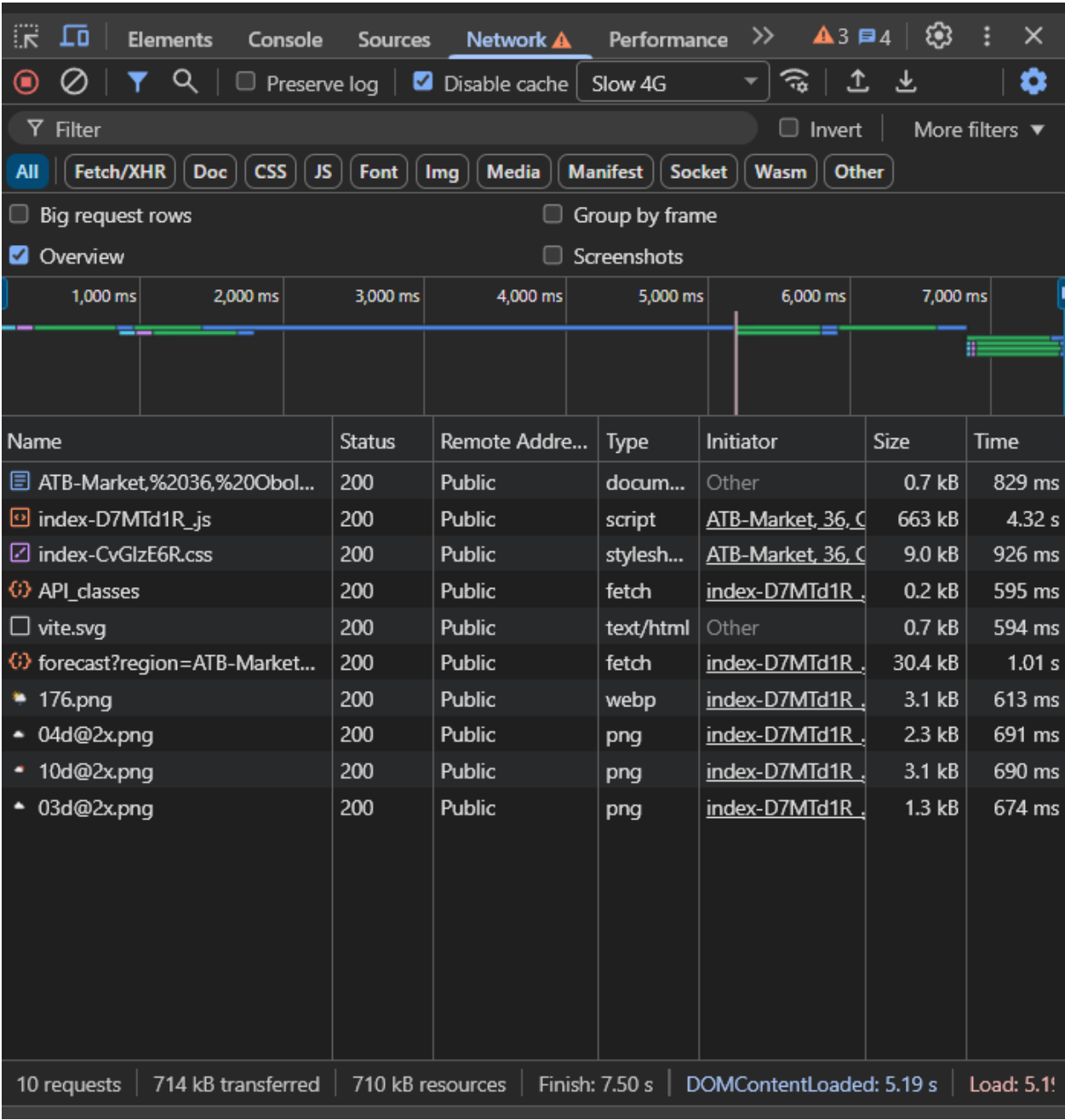


Рисунок 4.5 – Результати завантаження сторінки при повільному Інтернет з’єднанні

Наступним перевіримо веб-сторінку на доступність. Час за’явленої безвідомовної роботи серверу, на якому розгортається програмне забезпечення, 99.5%, що задовольняє ТЗ та нефункціональні вимоги. Отже, необхідно перевірити, що швидкість відновлення компонентів ПЗ достатня. Після перевірки за допомогою Postman, отримаємо результат, що приблизно дорівнює п’яти секундам, тобто відповідає необхідним умовам (рисунок 4.6).

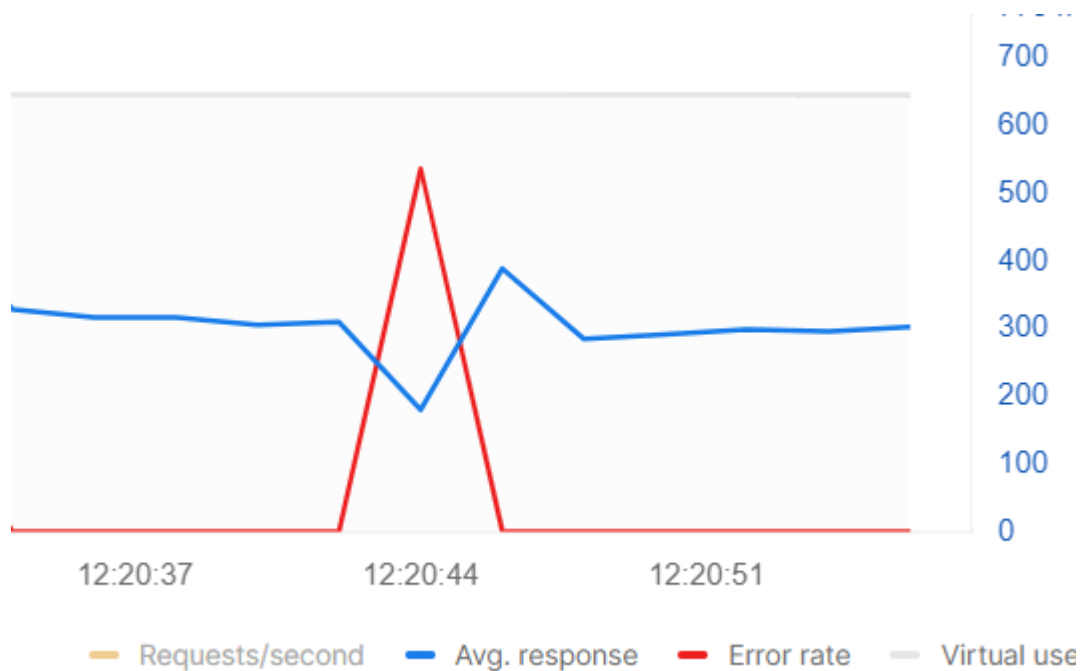


Рисунок 4.6 – Швидкість відновлення веб-сервісу

На останок перевіримо код на підтримуваність, що є важливим для подальшого вдосконалення ПЗ. Для цього застосуємо веб-застосунок SonarQube та отримаємо дуже хорошу загальну оцінку (рисунок 4.7), що підтверджує якість коду.

Maintainability

35 Open issues

A

Рисунок 4.7 – Оцінка підтримуваності коду SonarQube

4.2 Опис процесів тестування

Тестування програмного забезпечення є критично важливим етапом життєвого циклу розробки, метою якого є перевірка відповідності продукту заявленим вимогам та виявлення дефектів. Існує багато типів тестування, які класифікують за різними критеріями. За рівнем тестування виділяють

модульне, інтеграційне та системне. За метою – функціональне (перевірка відповідності функціональним вимогам) та нефункціональне (перевірка продуктивності, надійності, зручності використання, безпеки тощо). За ступенем автоматизації – мануальне та автоматизоване.

Як було детально продемонстровано у попередньому підрозділі (4.1 Аналіз якості ПЗ), оцінка ключових нефункціональних характеристик та аналіз якості коду вже були проведені з використанням спеціалізованих інструментів та аналізу сценаріїв. Ці дії фактично представляють собою результати нефункціонального тестування та статичного аналізу коду.

Даний підрозділ зосереджується на описі процесу функціонального тестування, спрямованого на перевірку коректності роботи всіх ключових функцій системи відповідно до сформованих функціональних вимог. Для даної курсової роботи було обрано саме мануальний підхід до функціонального тестування. Цей вибір обумовлений, по-перше, обсягом та складністю проєкту: розроблений веб-сервіс має чітко визначений та відносно невеликий набір основних функцій, що дозволяє ретельно перевірити кожен ключовий сценарій використання вручну. По-друге, академічний характер роботи передбачає демонстрацію розуміння процесу розробки та отримання працездатного прототипу, тоді як впровадження повноцінного автоматизованого тестування вимагало б значних додаткових зусиль. По-третє, на етапі розробки курсового проєкту частота змін функціоналу є помірною, що знижує потребу у частому регресійному тестуванні, де автоматизація найбільш ефективна.

Процес мануального функціонального тестування відбувався ітеративно паралельно з розробкою. Після реалізації кожної значної функціональної частини проводилося тестування відповідних сценаріїв. Для перевірки роботи кінцевих точок backend-частини використовувався інструмент Postman, а тестування frontend-частини здійснювалося безпосередньо у веб-браузері з використанням інструментів розробника.

В таблицях 4.1-4.7 наведено приклади ключових тест-кейсів, що були виконані для перевірки відповідності системи функціональним вимогам.

Таблиця 4.1 – Тест 1 Отримання прогнозу за геолокацією (FR-9)

Початковий стан системи	Користувач вперше відкриває веб-сервіс у браузері.
Вхідні дані	
Опис проведення тесту	1. відкрити головну сторінку веб-сервісу; 2. дочекатися запиту браузера на доступ до геолокації; 3. надати дозвіл натисканням «дозволити ...».
Очікуваний результат	Після надання дозволу система автоматично визначає поточне місто користувача, завантажує та відображає агрегований прогноз погоди для нього.
Фактичний результат	Збігається з очікуваним.

Таблиця 4.2 – Тест 2 Ручний пошук міста з підказками (FR-3, FR-8)

Початковий стан системи	Головна сторінка веб-сервісу відкрита.
Вхідні дані	Часткова (або повна) назва міста, як наприклад, «Kyiv».
Опис проведення тесту	1. у поле пошуку міста «City name» ввести назву існуючого міста; 2. дочекатися появи списку підказок під полем пошуку; 3. клікнути на відповідну підказку, яка найкраще підходить користувачу.
Очікуваний результат	Поле пошуку очищується. Система завантажує та відображає агрегований прогноз погоди для обраного міста.

Продовження таблиці 4.2

Фактичний результат	Збігається з очікуванням.
---------------------	---------------------------

Таблиця 4.3 – Тест 3 Вибір/скасування вибору API (FR-1, FR-2, FR-3)

Початковий стан системи	Відображено прогноз погоди для будь-якого міста. Усі доступні чекбокси API відмічені за замовчуванням.
Вхідні дані	
Опис проведення тесту	1. зняти позначку з одного із чекбоксів API; 2. зняти позначки з усіх API, крім одного; 3. спробувати зняти позначку з останнього;
Очікуваний результат	Після кроку 1 дані прогнозу оновлюються - у блоках зникають дані від деактивованого API, а також перераховуються усереднені результати, використовуючи результати активних API. Після кроку 2 відбувається той самий процес, що й після кроку 1. Крок 3 неможливий, оскільки система не дозволяє деактивувати усі чекбокси одночасно.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.4 – Тест 4 Перегляд узагальненого прогнозу (FR-4)

Початковий стан системи	Відображено прогноз погоди для будь-якого міста та обрано секцію «Forecast» за замовчуванням.
Вхідні дані	
Опис проведення тесту	1. Переглянути секцію сторінки, де відображається прогноз погоди по днях

Продовження таблиці 4.4

Очікуваний результат	На сторінці коректно відображаються блоки прогнозу для кожного доступного дня (згідно з даними API). Кожен блок містить основні денні показники (мін./макс. температура, вологість, швидкстія вітру, іконки погоди, середні значення тощо).
Фактичний результат	Збігається з очікуванням.

Таблиця 4.5 – Тест 5 Перегляд детального прогнозу на день (FR-5)

Початковий стан системи	Відображено прогноз погоди для будь-якого міста та обрано секцію «Forecast» за замовчуванням.
Вхідні дані	
Опис проведення тесту	1. клікнути на інтерактивний блок будь-якого дня з прогнозом.
Очікуваний результат	Відбувається перехід на нову сторінку. На екрані відображається детальний прогноз погоди для обраного дня, що містить показники, такі як температура, тиск, вологість, швидкість вітру тощо, для кожної години.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.6 – Тест 6 Перегляд та взаємодія з динамікою погоди (FR-6, FR-7)

Початковий стан системи	Відображено прогноз погоди для будь-якого міст, обрано секцію «Forecast» за замовчуванням та всі API сервіси.
Вхідні дані	

Продовження таблиці 4.6

Опис проведення тесту	1. клікнути на секцію «Dynamics»; 2. дочекатися завантаження сторінки з графіками; 3. клікнути на іконку для вибору іншого метеорологічного показника; 4. натиснути на початкову дату; 5. змінити початкову дату на наступну;
Очікуваний результат	Після кроку 1 відбувається перехід на сторінку динаміки та завантажується графік для температури, на якому зображені лінії усіх сервісів та усереднений варіант. Після кроку 3 відображається інший графік для відповідного показника. Після кроку 5 графік для будь-якого показника починається із відповідно обраної дати. Після кроку 7 графік будь-якого показника закінчується відповідно обраною датою.
Фактичний результат	Збігається з очікуванням.

Таблиця 4.7 – Тест 7 Обробка некоректного введення міста (FR-3, FR-8)

Початковий стан системи	Головна сторінка веб-сервісу відкрита.
Вхідні дані	Некоректна назва міста, як наприклад, «Test123».
Опис проведення тесту	1. у поле пошуку міста «City name» ввести назву неіснуючого міста; 2. очікувати пошук можливих варіантів; 3. натиснути клавішу «Enter».

Продовження таблиці 4.7

Очікуваний результат	Після кроку 1 з'являється повідомлення про завантаження варіантів «Loading...», після чого воно пропадає, а на його місці не з'являється жодного із варіантів. Після кроку 3 нічого не відбувається.
Фактичний результат	Збігається з очікуванням.

4.3 Опис контрольного прикладу

Даний підрозділ ілюструє типовий сценарій взаємодії користувача з розробленим веб-сервісом прогнозування погоди. Метою є покрокова візуалізація основного функціоналу системи на конкретному прикладі, включаючи опис можливих варіантів дій користувача та особливостей роботи інтерфейсу.

При відкритті веб-сервісу користувачем (рисунок 4.8), система може запропонувати визначити місцезнаходження автоматично за допомогою геолокації браузера. Якщо користувач надає дозвіл, система одразу переходить до відображення прогнозу для поточного міста (рисунок 4.9). Якщо ж користувач відмовляється від автоматичного визначення або бажає знайти погоду для іншого міста вручну. На екрані відображається поле для введення назви міста та перелік доступних для вибору API, усі з яких відмічені за замовчуванням (рисунок 4.10).

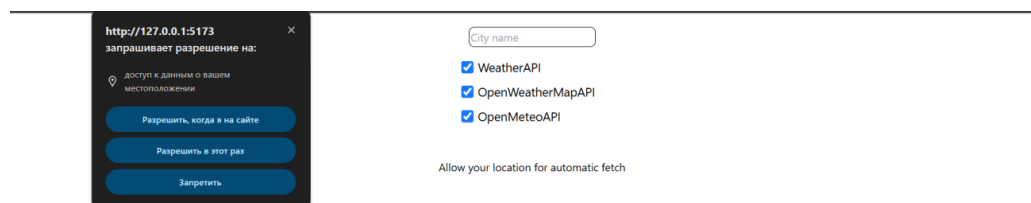


Рисунок 4.8 – Запит на геолокацію

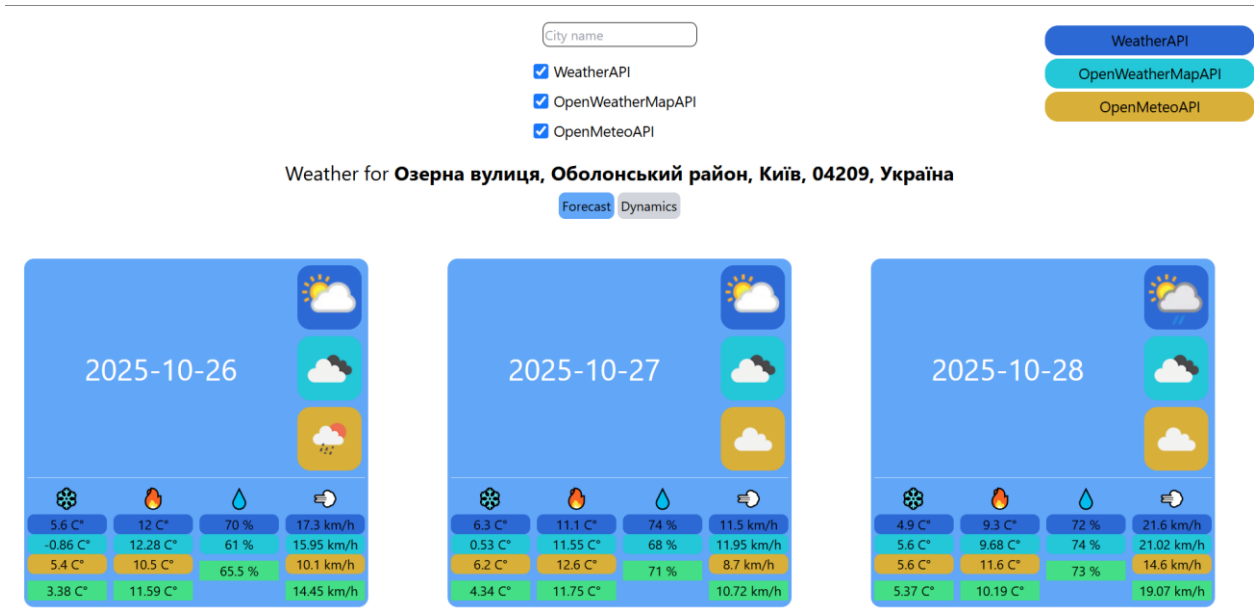


Рисунок 4.9 – Отримання прогнозу погоди при підтвердженні доступу до геолокації

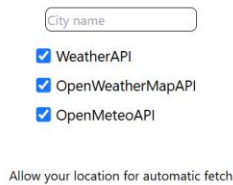


Рисунок 4.10 – Результат при відмові доступу до геолокації

Користувач може ввести назву міста у поле пошуку "City name". Система асинхронно надсилає запити до backend-сервісу для отримання варіантів автодоповнення. Під полем введення динамічно з'являється список підказок, що відповідають введеному тексту (рисунок 4.11). За умови введення некоректного міста, система не запропонує варіантів, таким чином, користувач не може отримати прогноз для некоректного міста (рисунок 4.12).

Київ, Україна

Київська область, Україна

☒ WeatherAPI

☒ OpenWeatherMapAPI

☒ OpenMeteoAPI

Allow your location for automatic fetch

Рисунок 4.11 – Отримання варіантів пошуку по частковому введенні

☒ WeatherAPI

☒ OpenWeatherMapAPI

☒ OpenMeteoAPI

WeatherAPI

OpenWeatherMapAPI

OpenMeteoAPI

Weather for **Озерна вулиця, Оболонський район, Київ, 04209, Україна**

Forecast Dynamics

2025-10-26

5.6 C°

12 C°

70 %

17.3 km/h

-0.86 C°

12.28 C°

61 %

15.95 km/h

5.4 C°

10.5 C°

65.5 %

10.1 km/h

3.38 C°

11.59 C°

14.45 km/h

2025-10-27

6.3 C°

11.1 C°

74 %

11.5 km/h

0.53 C°

11.55 C°

68 %

11.95 km/h

6.2 C°

12.6 C°

71 %

8.7 km/h

4.34 C°

11.75 C°

10.72 km/h

2025-10-28

4.9 C°

9.3 C°

72 %

21.6 km/h

5.6 C°

9.68 C°

74 %

21.02 km/h

5.6 C°

11.6 C°

73 %

14.6 km/h

5.37 C°

10.19 C°

19.07 km/h

Рисунок 4.12 – Результат при введенні некоректного міста

При обранні варіанту із запропонованих, система перенаправляє користувача на сторінку загального прогнозу по дням, видаляючи при цьому дані із поля «City name» (рисунок 4.13).

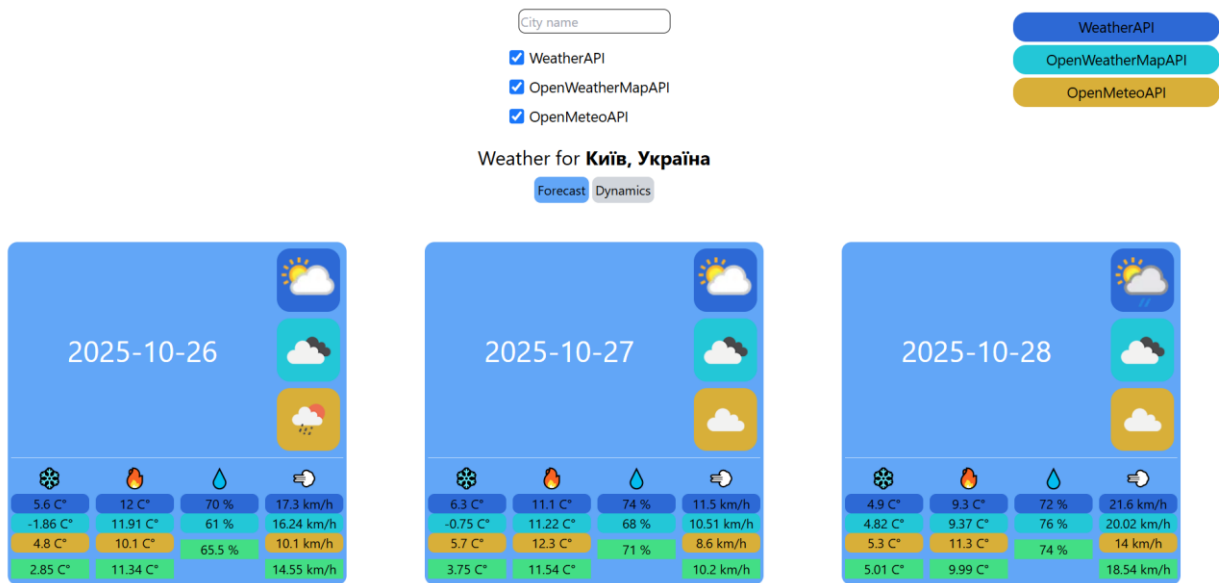


Рисунок 4.13 – Отримання прогнозу погоди після обрання варіанту

Для отримання інформації по годиннам, необхідно клікнути на будь-який денний блок, після чого відбудеться перенаправлення на відповідну сторінку із погодинної інформацією (рисунок 4.14). Для повернення на поденний прогноз можна використати відповідну кнопку «Back to daily forecast» в лівому верхньому куту застосунку.



Рисунок 4.14 – Результати для конкретного обраного дня

Якщо користувач бажає використовувати тільки деякі сервіси прогнозу для агрегованого результату, він має можливість деактивувати відповідні чекбокси. При цьому результуючий прогноз буде змінений (рисунок 4.15).

Вимикання усіх чекбоксів є неможливим, тому, при спробі вимкнути останній активний, система не дозволить це зробити.

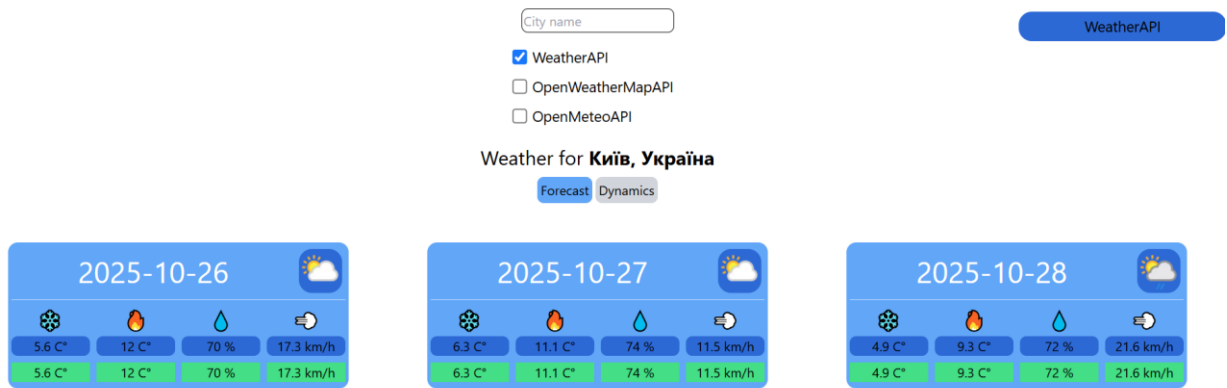


Рисунок 4.15 – Спроба деактивування усіх сервісів прогнозу

Для отримання погодинної динаміки параметрів прогнозу, необхідно вибрати секцію «Dynamics» натисканням на неї. Після чого відбудеться перехід на відповідну сторінку та рендеринг графіку із обраним індикатором за замовчуванням (рисунок 4.16). Для зміни індикатора можна натиснути на необхідну іконку справа і графік буде перебудовано (рисунок 4.17). Також можна змінити початкову та кінцеву дату. Для цього необхідно натиснути на відповідну дату під графіком (рисунок 4.18) та обрати необхідну, після чого графік буде перебудовано, починаючи та закінчуючи обраними датами (4.19-4.20)

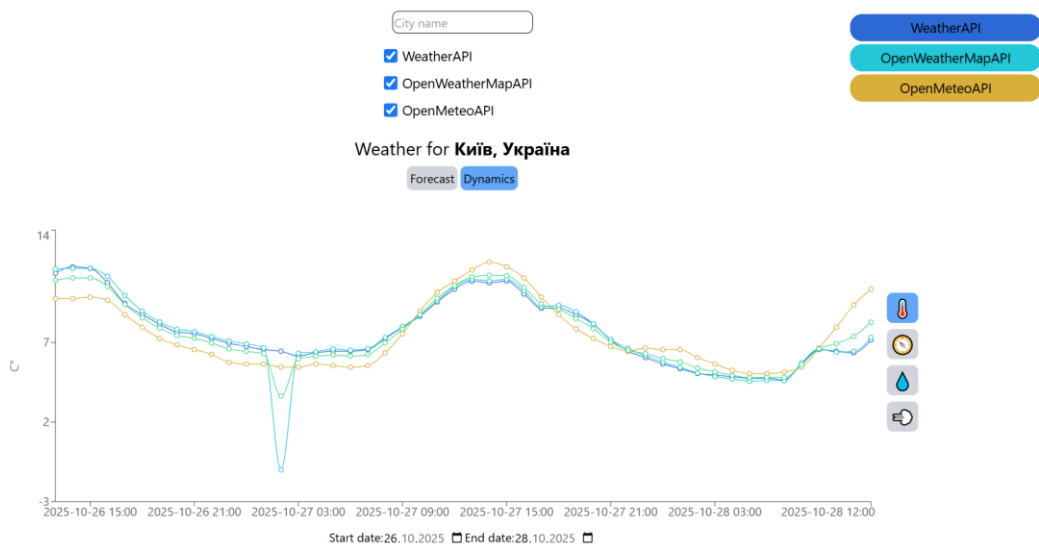


Рисунок 4.16 – Вибір секції динаміки прогнозу погоди

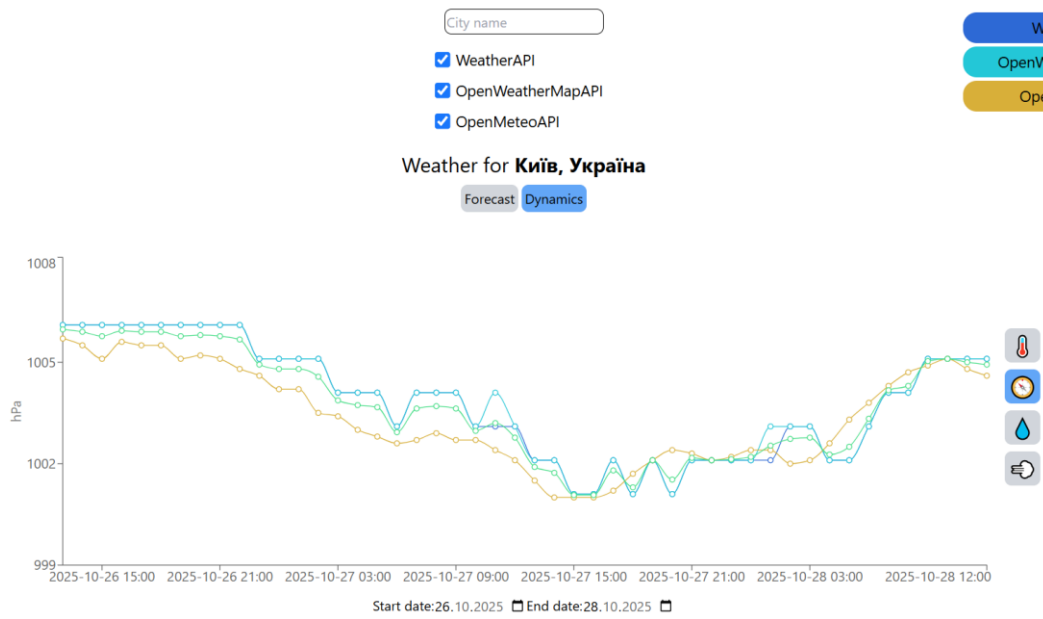


Рисунок 4.17 – Зміна індикатора погоди для динаміки

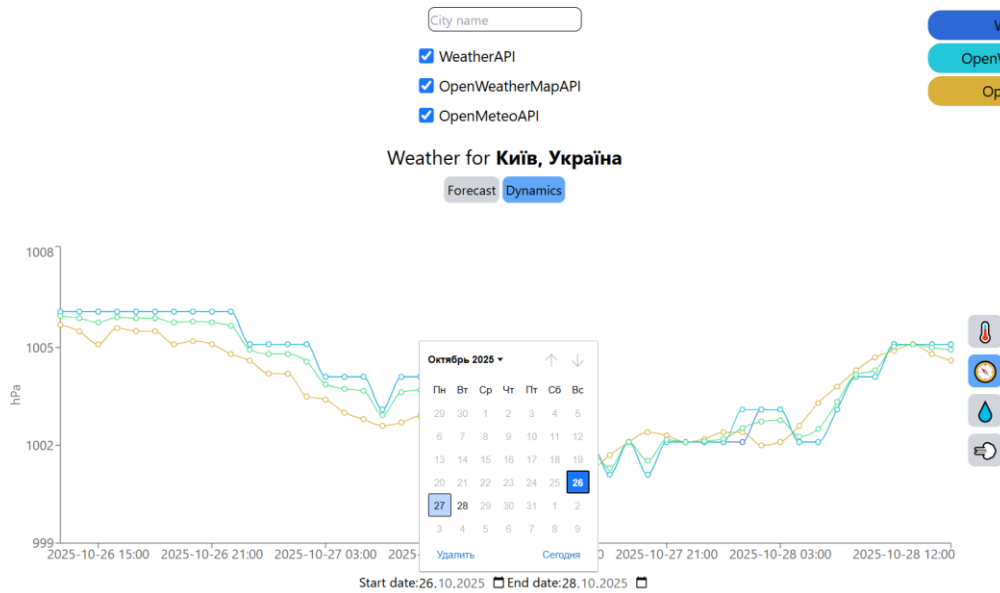


Рисунок 4.18 – Зміна початкової дати динаміки

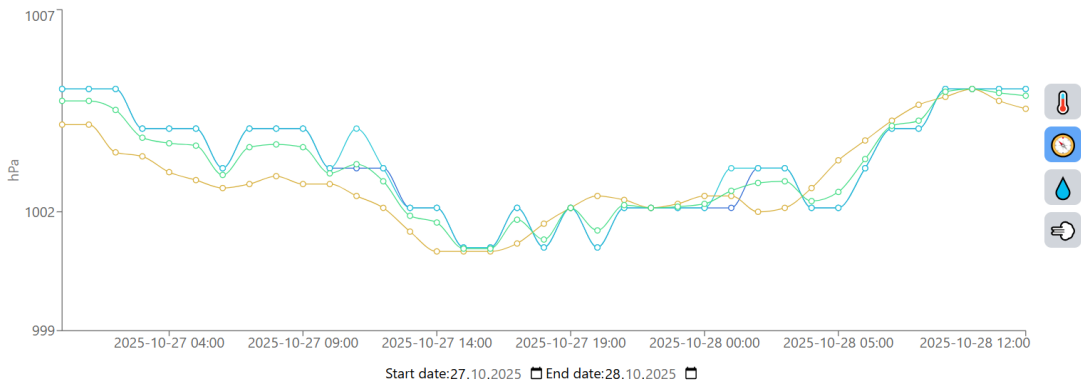


Рисунок 4.19 – Результат після зміни початкової дати

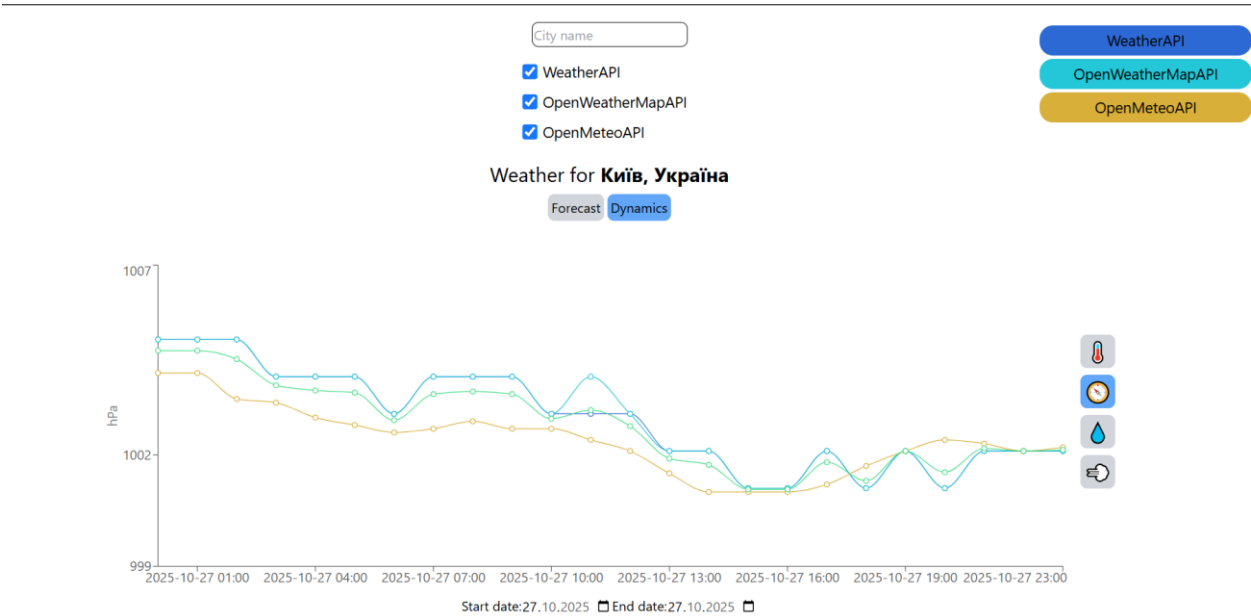


Рисунок 4.20 – Результат після зміни кінцевої дати динаміки

Висновки до розділу

У даному розділі було проведено аналіз якості розробленого програмного забезпечення та описано процеси його тестування, що є важливими етапами для підтвердження відповідності продукту встановленим вимогам.

Спочатку було виконано аналіз якості ПЗ за ключовими метриками: продуктивність, доступність, зручність використання та підтримуваність. Для кожної метрики було проведено оцінку з використанням відповідних

інструментів, таких як: Postman для вимірювання часу відповіді backend-частини, Google Lighthouse для оцінки продуктивності та доступності frontend-частини, а також було зімітовано навантаження та збої для перевірки роботи з великою кількістю одночасних користувачів та швидкості відновлення. Також було використано SonarQube для аналізу якості коду та його підтримуваності.

Далі було надано короткий огляд основних типів тестування та обґрунтовано вибір мануального функціонального тестування як основного підходу для даної курсової роботи. Також було описано методику проведення тестування та представлено набір ключових тест-кейсів у вигляді таблиць, що охоплюють перевірку всіх основних функціональних вимог.

На завершення розділу було наведено опис контрольного прикладу, який покроково, з ілюстраціями, демонструє типовий сценарій взаємодії користувача з веб-сервісом: від пошуку міста до отримання агрегованого прогнозу, перегляду детальної інформації та аналізу динаміки погоди. У прикладі також було показано основні альтернативні шляхи та особливості роботи інтерфейсу.

Таким чином, у цьому розділі було представлено докази якості розробленого програмного забезпечення через аналіз ключових метрик та демонстрації коректності роботи основного функціоналу за допомогою тест-кейсів та детального контрольного прикладу. Результати підтверджують, що створений веб-сервіс є працездатним, відповідає поставленим вимогам та готовий до використання.

5 ПОВНА ІНСТРУКЦІЯ КОРИСТУВАЧА

5.1 Встановлення програмного забезпечення

Розроблене програмне забезпечення є веб-сервісом, доступ до якого здійснюється через стандартний веб-браузер. Таким чином, для кінцевих користувачів не передбачено жодних специфічних кроків зі встановлення – достатньо мати підключення до мережі Інтернет та перейти за відповідною URL-адресою веб-сервісу, наданою адміністратором системи.

Опишемо далі процес розгортання веб-сервісу з вихідного коду за допомогою системи контейнеризації Docker. Цей процес призначений для розробників, тестувальників або системних адміністраторів, які бажають запустити локальну або приватну копію застосунку.

Перед початком розгортання необхідно переконатися, що на вашому комп'ютері або сервері встановлено наступне програмне забезпечення:

- система керування версіями Git [48];
- Docker Engine;
- Docker Compose;

Спочатку необхідно завантажити вихідний код проєкту з репозиторію. Відкрийте термінал або командний рядок та виконайте команду `git clone`, вказавши URL-адресу репозиторію: `git clone <URL-адреса_репозиторію>`.

Після успішного клонування перейдіть до створеної директорії проєкту: `cd <назва_директорії_проєкту>`.

Для взаємодії з зовнішніми API сервісу необхідно налаштувати ключі доступу. Створіть у кореневій директорії проєкту файл з назвою `.env`. Відкрийте цей файл у текстовому редакторі та додайте наступні рядки, замінивши `<your_key>` на ваші актуальні API-ключі, що можна отримати на відповідних сайтах сервісів Weatherapi, Openweathermap, Geocodemap:

```
WEATHER_API_KEY=<your_key>
```

```
OPEN_WEATHER_MAP_API_KEY=<your_key>
```

GEOCODE_API_KEY=<your_key>

Перебуваючи у кореневій директорії проєкту, виконайте в терміналі команду для збірки Docker-образів та запуску всіх необхідних контейнерів: `docker-compose up --build` (або `docker compose up --build`, в залежності від версії Docker Compose). Після встановлення усіх залежностей, відбудеться запуск контейнерів, backend-сервіс стане доступним локально за шляхом <http://localhost:8000>, а frontend-сервіс – за шляхом <http://localhost:5173>.

Для запуску системи на сервері, з можливістю будь-якого користувача отримати доступ, необхідно отримати домен [49], підключити його до відповідної публічної ір-адреси сервера [50] та додати додатковий запис у створений .env файл:

DOMAIN=<your_domain>

Тепер треба впевнитися, що порти (80, 443) сервера відкриті [51], а потім запуснути виконуваний файл «run_port.sh». Після цього frontend-сервіс стане доступним за посиланням https://your_domain, а backend-частина за посиланням https://your_domain/backend/.

5.2 Інструкція користувача

Цей підрозділ містить інструкції для користувачів щодо роботи з веб-сервісом прогнозування погоди. Описано основні кроки для отримання та аналізу метеорологічних даних.

Для початку роботи з веб-сервісом, відкрийте веб-браузер (рекомендовано Google Chrome, Firefox або інші сучасні варіанти), а потім перейдіть за посиланням, наданою адміністратором системи.

При першому відкритті сайту, браузер запросить дозвіл на доступ до вашого місцезнаходження (рисунок 5.1). Якщо надати доступ, сервіс автоматично визначить ваше поточне місцезнаходження та відобразить прогноз для нього, якщо відмовитися, то ви зможете перейти до введення міста вручну.

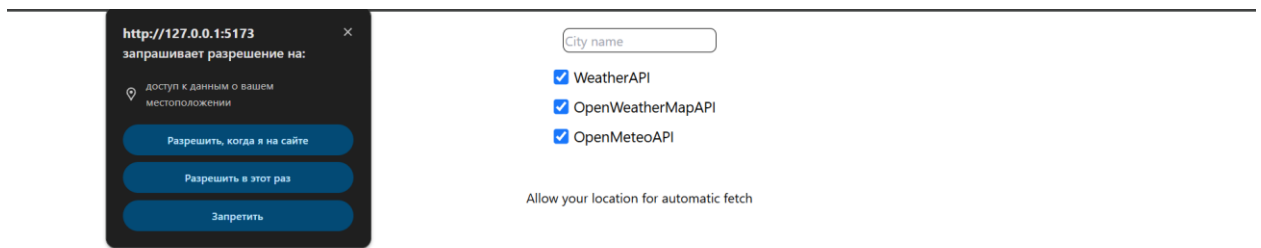


Рисунок 5.1 – Запит на геолокацію

Для пошуку прогнозу вручну, необхідно натиснути на поле зверху «City name» та ввести потрібне місто. Під полем з'являться можливі варіанти, які відповідають введенному місту (рисунок 5.2). Оберіть потрібний варіант натисканням на нього.

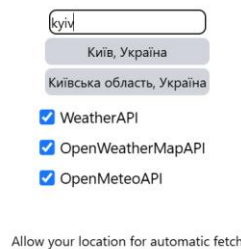


Рисунок 5.2 – Пошук прогнозу вручну

Після вибору одного із варіантів, система почне завантаження відповідного прогнозу. Через деякий час ви побачите узагальнений поденний прогноз (рисунок 5.3). Кожен день має окремий блок, у якому можна побачити результати різних сервісів прогнозування. В правому верхньому куту можна побачити кольори сервісів, а кожен блок має дані із ними. На рисунку 5.3, наприклад, для дати «2025-10-26», справа від неї можна побачити три іконки погоди. Наприклад, перша (синя) показує, що сервіс WeatherAPI (оскільки в верхньому правому куті він має також синій колір) прогнозує хмарний день, OpenWeatherMapAPI (бірюзовий) також показує теж саме, тільки використовуючи іншу іконку, і так далі. Нижня частина блоку показує різні числові параметри. Для денної інформації, можна побачити мінімальну

температуру за день (сніжинка), максимальну (вогонь), середню вологість (крапля), середню швидкість вітру (вітер). Для кожного параметру наведено результати відповідних сервісів та усереднений результат (зелений колір).

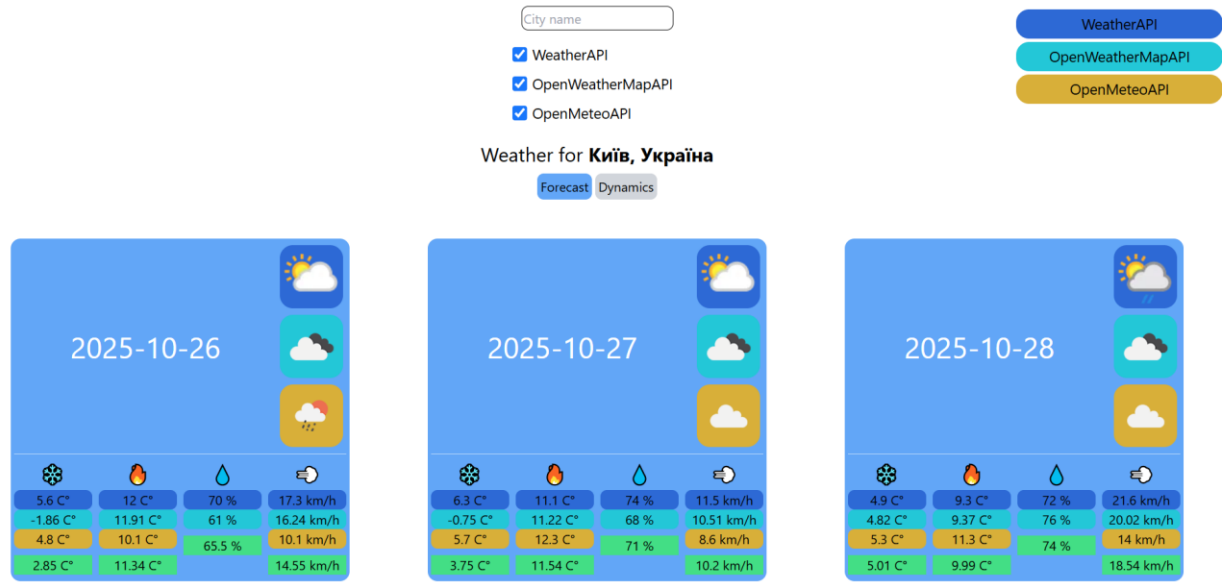


Рисунок 5.3 – Отримання прогнозу погоди

Для отримання погодинної інформації необхідно клікнути на денний блок. Після цього отримаємо схожі блоки, тільки вже для кожної години дня (рисунок 5.4). Показники для кожної години дещо відрізняються від денної інформації: температура (термометр), тиск (барометр), вологість (крапля), швидкість вітру (вітер).



Рисунок 5.4 – Результати для конкретного обраного дня

За необхідністю, можна відключити зайві сервіси прогнозування за допомогою ввимкнення відповідних чекбоксів, що знаходяться під полем «City name», після чого прогноз оновиться (рисунок 5.5).

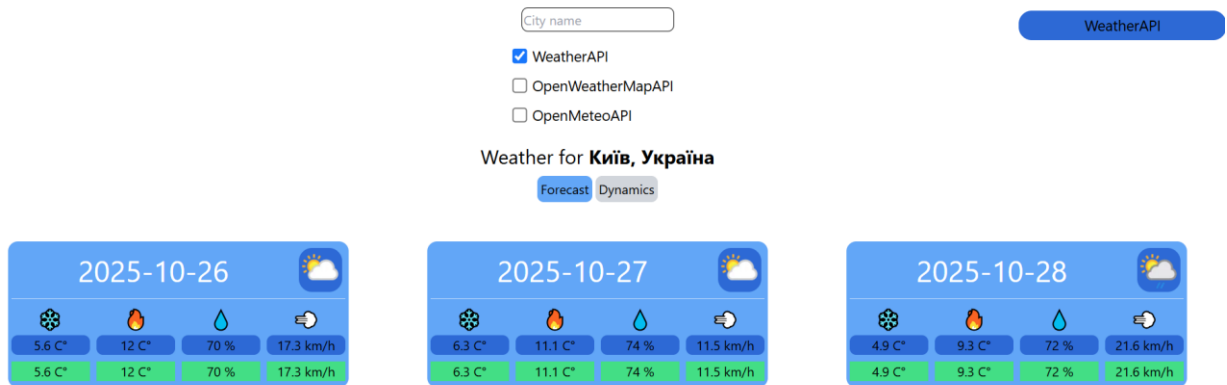


Рисунок 5.5 – Ввимкнення зайвих сервісів

Для отримання графіків метеорологічних показників у часі, необхідно натиснути на секцію «Dynamics», що знаходиться під чекбоксами. Після цього відбудеться процес відображення динамічних результатів для температури за замовчуванням (рисунок 5.6). Справа від графіку можна змінити досліджуваний показник натисканням на відповідну іконку. Під графіком можна обрати початкову та кінцеву дати динамічної картини із дозволених варіантів (рисунок 5.7).

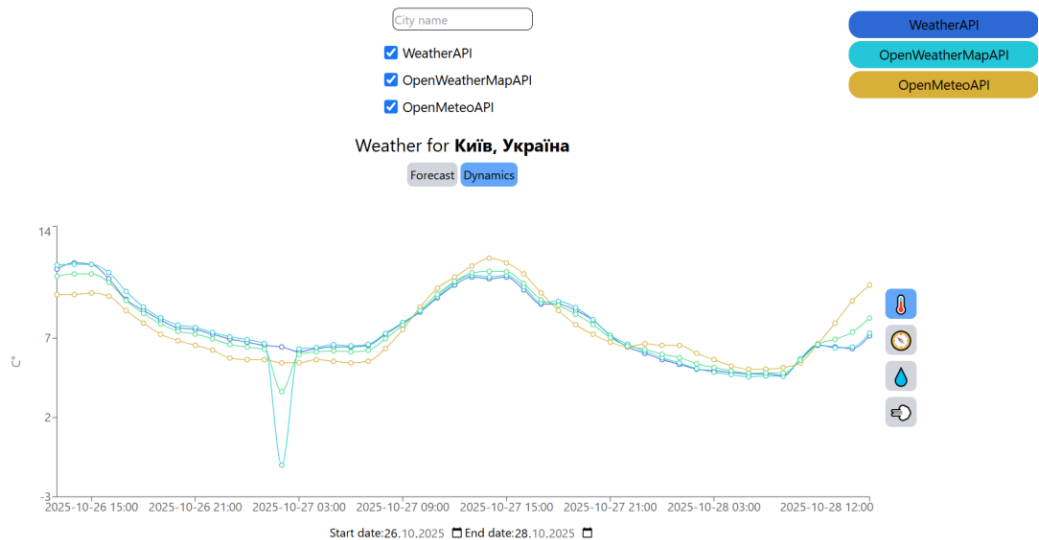


Рисунок 5.6 – Вибір секції динаміки прогнозу погоди

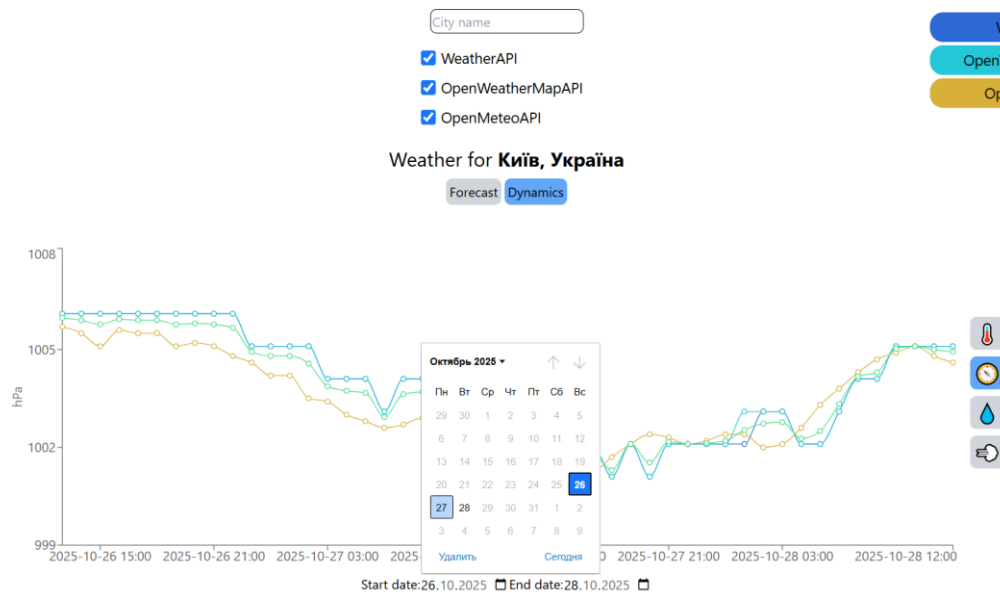


Рисунок 5.7 – Інтерфейс вибору дати

Висновки до розділу

У даному розділі було представлено повну інструкцію користувача для розробленого веб-сервісу прогнозування погоди, що охоплює як процес розгортання програмного забезпечення, так і безпосереднє використання його функціоналу.

Спочатку було роз'яснено, що для кінцевих користувачів не потрібні спеціальні дії зі встановлення, оскільки доступ до сервісу здійснюється через веб-браузер. Водночас, для розробників, тестувальників та адміністраторів було надано детальну покрокову інструкцію з розгортання застосунку з вихідного коду за допомогою системи контейнеризації Docker. Описано необхідні передумови, процес отримання коду, конфігурації змінних середовища з API-ключами, а також команди для збірки та запуску контейнерів як у локальному середовищі розробки, так і на віддаленому сервері з використанням домену.

Далі було надано вичерпний посібник з використання веб-сервісу. Описано основні кроки взаємодії з інтерфейсом: початок роботи з можливістю автоматичного визначення геолокації, ручний пошук міста з використанням

підказок, вибір джерел даних (API) для агрегації прогнозу. Детально проілюстровано процес перегляду узагальненого поденного прогнозу, пояснено значення елементів інтерфейсу, включаючи кольорове кодування джерел даних та розрахунок середніх значень. Також описано перехід до сторінки детального погодинного прогнозу та роботу зі сторінкою динаміки погодних показників, включаючи перемикання між індикаторами та використання фільтрів дат.

Таким чином, у цьому розділі зібрано всю необхідну інформацію, що дозволяє як розгорнути програмне забезпечення у відповідному середовищі, так і ефективно використовувати всі його основні функції кінцевим користувачем для отримання агрегованих даних прогнозу погоди.

ВИСНОВКИ

У ході виконання даної курсової роботи було успішно розроблено веб-сервіс прогнозування погоди шляхом агрегації даних з різних онлайн гідрометцентрів, що відповідає поставленій меті та задачам.

Практична значимість одержаних результатів полягає у створенні функціонального програмного продукту, який вирішує актуальну проблему отримання більш надійного та комплексного прогнозу погоди. Завдяки агрегації даних з кількох незалежних джерел, сервіс потенційно знижує залежність від точності одного джерела та надає користувачеві усереднені показники, що можуть бути більш репрезентативними. Розроблений веб-сервіс має інтуїтивно зрозумілий інтерфейс, реалізований за допомогою React та Tailwind CSS, що забезпечує зручний доступ до погодної інформації як для звичайних користувачів, так і може бути корисним для планування діяльності у сферах, залежних від погодних умов. Практичним результатом є готовий до розгортання веб-додаток, контейнеризований за допомогою Docker та налаштований для роботи з Nginx, що демонструє застосування сучасних підходів до розробки та розгортання ПЗ.

Розв'язання функціональних задач програмного забезпечення було досягнуто наступним чином: збір даних з різних джерел реалізовано шляхом створення асинхронних клієнтів у backend-частині для взаємодії з API трьох метеорологічних сервісів. Агрегація даних забезпечується модулем WeatherAggregator, який уніфікує отримані дані та розраховує середні значення. Отримання погоди за назвою міста та вибір API реалізовано через поле введення та чекбокси відповідно у frontend-частині, що потім взаємодіє backend-частиною, що отримує координати за допомогою сервісу Geocodemap та результату прогнозу за допомогою відповідних сервісів Openweathermap, Weatherapi та Open-meteo. Перегляд узагальненого прогнозу забезпечується React-компонентами, що відображають поденний прогноз.

Можливість перегляду детального прогнозу реалізовано через перехід на окрему сторінку погодинного прогнозу. Візуалізація динаміки та вибір періоду здійснюються на окремій сторінці з графіками Recharts та фільтрами дат. Надання підказок при введенні міста реалізовано через звернення до Geocodemapс та отримання можливих схожих варіантів. Автоматичне визначення місцезнаходження забезпечується запитом до Geolocation API браузера.

Таким чином, у процесі виконання курсової роботи було проведено аналіз предметної області, розроблено вимоги, спроектовано архітектуру, реалізовано програмне забезпечення з використанням сучасних технологій, проведено його тестування та підготовлено документацію. Усі поставлені функціональні задачі було успішно вирішено, а створений веб-сервіс демонструє працездатність та відповідає визначеним вимогам.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Web-application. URL: <https://www.javatpoint.com/web-application> (дата звернення 29.11.2025).
2. Weather. URL: <https://weather.com/> (дата звернення 29.11.2025).
3. Accuweather. URL: <https://www.accuweather.com/> (дата звернення 29.11.2025).
4. Worldweather. URL: <https://worldweather.wmo.int/> (дата звернення 29.11.2025).
5. Weather app market size. URL: <https://www.grandviewresearch.com/industry-analysis/weather-app-market-report> (дата звернення 29.11.2025).
6. Data-driven weather forecasting. URL: <https://www.mdpi.com/2073-4433/15/6/689> (дата звернення 29.11.2025).
7. Forecast process. URL: <https://www.weather.gov/about/forecast-process> (дата звернення 29.11.2025).
8. Introducing weather API. URL: <https://blog.weatherstack.com/blog/introducing-real-time-weather-data-api/> (дата звернення 29.11.2025).
9. Architecture patterns. URL: <https://www.geeksforgeeks.org/types-of-software-architecture-patterns/> (дата звернення 29.11.2025).
10. Web hosting. URL: <https://www.geeksforgeeks.org/types-of-web-hosting> (дата звернення 29.11.2025).
11. Python. URL: <https://www.python.org/> (дата звернення 29.11.2025).
12. Java. URL: <https://www.java.com/> (дата звернення 29.11.2025).
13. Go. URL: <https://go.dev/> (дата звернення 29.11.2025).
14. PHP. URL: <https://www.php.net/> (дата звернення 29.11.2025).
15. FastAPI. URL: <https://fastapi.tiangolo.com/> (дата звернення 29.11.2025).

16. Flask. URL: <https://flask.palletsprojects.com/en/stable/> (дата звернення 29.11.2025).
17. Django. URL: <https://www.djangoproject.com/> (дата звернення 29.11.2025).
18. JavaScript. URL: <https://developer.mozilla.org/ru/docs/Web/JavaScript> (дата звернення 29.11.2025).
19. React. URL: <https://react.dev/> (дата звернення 29.11.2025).
20. Vue. URL: <https://vuejs.org/> (дата звернення 29.11.2025).
21. Angular. URL: <https://vuejs.org/> (дата звернення 29.11.2025).
22. Tailwind. URL: <https://tailwindcss.com/> (дата звернення 29.11.2025).
23. Bootstrap. URL: <https://getbootstrap.com/> (дата звернення 29.11.2025).
24. Docker. URL: <https://docs.docker.com/> (дата звернення 29.11.2025).
25. Kubernetes. URL: <https://kubernetes.io/> (дата звернення 29.11.2025).
26. Nginx. URL: <https://nginx.org/> (дата звернення 29.11.2025).
27. Redis. URL: <https://redis.io/> (дата звернення 29.11.2025).
28. Memcached. URL: <https://memcached.org/> (дата звернення 29.11.2025).
29. MongoDB. URL: <https://www.mongodb.com/> (дата звернення 29.11.2025).
30. Openweathermap. URL: <https://openweathermap.org/> (дата звернення 29.11.2025).
31. Weatherapi. URL: <https://www.weatherapi.com/> (дата звернення 29.11.2025).
32. Open-meteo. URL: <https://open-meteo.com/> (дата звернення 29.11.2025).
33. Tomorrow. URL: <https://www.tomorrow.io/> (дата звернення 29.11.2025).

34. Weatherstack. URL: <https://weatherstack.com/> (дата звернення 29.11.2025).
35. Apache. URL: <https://httpd.apache.org/> (дата звернення 29.11.2025).
36. Visual Studio Code. URL: <https://code.visualstudio.com/> (дата звернення 29.11.2025).
37. PyCharm. URL: <https://www.jetbrains.com/pycharm/> (дата звернення 29.11.2025).
38. WebStorm. URL: <https://www.jetbrains.com/webstorm/> (дата звернення 29.11.2025).
39. BPMN. URL: https://en.wikipedia.org/wiki/Business_Process_Model_and_Notation (дата звернення 29.11.2025).
40. C4. URL: <https://c4model.com/> (дата звернення 29.11.2025).
41. Httpx Python library. URL: <https://www.python-httpx.org/> (дата звернення 29.11.2025).
42. Redis Python library. URL: <https://redis.io/docs/latest/develop/clients/redis-py/> (дата звернення 29.11.2025).
43. React Router JavaScript library. URL: <https://reactrouter.com/> (дата звернення 29.11.2025).
44. Recharts JavaScript library. URL: <https://recharts.github.io/> (дата звернення 29.11.2025).
45. Postman. URL: <https://www.postman.com/> (дата звернення 29.11.2025).
46. SonarQube. URL: <https://www.sonarsource.com/products/sonarqube/> (дата звернення 29.11.2025).
47. Geocodemap. URL: <https://geocode.maps.co/> (дата звернення 29.11.2025).
48. Git. URL: <https://git-scm.com/> (дата звернення 29.11.2025).

49. Domain name. URL:
https://uk.wikipedia.org/wiki/%D0%94%D0%BE%D0%BC%D0%B5%D0%BD%D0%BD%D0%B5_%D1%96%D0%BC%27%D1%8F (дата звернення
 29.11.2025).

50. IP address. URL: <https://uk.wikipedia.org/wiki/IP-%D0%B0%D0%B4%D1%80%D0%B5%D1%81%D0%B0> (дата звернення
 29.11.2025).

51. Port. URL:
https://uk.wikipedia.org/wiki/%D0%9F%D0%BE%D1%80%D1%82_%D0%BF%D1%80%D0%BE%D1%82%D0%BE%D0%BA%D0%BE%D0%BB%D1%83
 (дата звернення 29.11.2025).

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

керівник

_____ Максим ГОЛОВЧЕНКО

“29” листопада 2025 р.

**ВЕБ-СЕРВІС ПРОГНОЗУВАННЯ ПОГОДИ, ШЛЯХОМ АГРЕГАЦІЇ
ДАНИХ ОНЛАЙН ГІДРОМЕТЦЕНТРІВ**

Текст програми

КПІ.ІІ-3104.045440.03.12

“ПОГОДЖЕНО”

Керівник роботи:

____Максим ГОЛОВЧЕНКО____

Виконавець:

____Ілля ВІЖУТКІН____

Київ – 2025

Посилання на репозиторій з повним текстом програмного коду
<https://github.com/IlyaViz/coursework-4>

Файл backend/main.py

Реалізація функціональної задачі маршрутизації запитів та обробки основних кінцевих точок API.

```
import logging

from fastapi import FastAPI
from fastapi import Query
from fastapi import HTTPException
from typing import List
from fastapi.middleware.cors import CORSMiddleware
from .external_api.weather_api import WeatherAPI
from .external_api.open_weather_map_api import OpenWeatherMapAPI
from .external_api.open_meteo_api import OpenMeteoAPI
from .logic.weather_aggregator import WeatherAggregator
from .external_api.region_helper import RegionHelper
from .exceptions.external_api_exceptions import APIResponseException
from .types.weather_aggregator_types import AggregatedData


logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
    handlers=[logging.StreamHandler(), logging.FileHandler("app.log",
mode="w")],
)

logger = logging.getLogger(__name__)
```

```
app = FastAPI()
```

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
weather_API_classes = (WeatherAPI, OpenWeatherMapAPI, OpenMeteoAPI)
```

```
@app.get("/forecast")
async def get_weather_forecast(
    region: str, API_classes: List[str] = Query(None)
) -> AggregatedData:
    if not API_classes:
        logger.error(f"No API classes provided for region '{region}'")

        raise HTTPException(400, "API classes must be provided")

    selected_API_classes = [
        API_class
        for API_class in weather_API_classes
        if API_class.__name__ in API_classes
    ]

    if not selected_API_classes:
        logger.error(
```

```

        f"No valid API classes provided for region '{region}' with classes
        {API_classes}"
    )

```

```

    raise HTTPException(400, "No valid API classes provided")

```

```

try:

```

```

    result = await WeatherAggregator.get_aggregated_weather(
        region, selected_API_classes
    )

```

```

except APIResponseException as e:

```

```

    raise HTTPException(500, f"Weather aggregation failed: {e}")

```

```

logger.info(f"Weather forecast for region '{region}' retrieved successfully")

```

```

return result

```

```

@app.get("/partial_city_helper")

```

```

async def get_partial_city_helper(partial_city: str) -> dict[str, list[str]]:

```

```

    try:

```

```

        options = await RegionHelper.get_options(partial_city)

```

```

    except APIResponseException as e:

```

```

        raise HTTPException(500, f"Failed to fetch options for '{partial_city}': {e}")

```

```

logger.info(f"Options for partial city '{partial_city}' retrieved successfully")

```

```

return {"options": options}

```

```

@app.get("/convert_coordinates")
async def convert_coordinates(lat: float, lon: float) -> dict[str, str]:
    try:
        region = await RegionHelper.convert_to_region((lat, lon))
    except APIResponseException as e:
        raise HTTPException(500, f"Failed to convert coordinates: {e}")

    logger.info(
        f"Coordinates ({lat}, {lon}) converted to region '{region}' successfully"
    )

    return {"region": region}

@app.get("/API_classes")
def get_API_classes() -> dict[str, list[str]]:
    API_classes = [cls.__name__ for cls in weather_API_classes]

    return {"API_classes": API_classes}

```

Файл backend/logic/weather_aggregator.py

Реалізація функціональної задачі агрегації даних різних джерел.

```

import logging

from typing import Any

from fastapi import HTTPException

from ..enums.hour_result_key_enum import HourResultKeyEnum as hrke
from ..enums.day_result_key_enum import DayResultKeyEnum as drke
from ..enums.result_type_key_enum import ResultTypeKeyEnum as rtke
from ..external_api.weather_api_base import WeatherAPIBase
from ..external_api.region_helper import RegionHelper

```

```

from ..exceptions.external_api_exceptions import APIResponseException
from ..types.general_weather_api_types import ForecastData
from ..types.weather_aggregator_types import AggregatedData

```

```

logger = logging.getLogger(__name__)

```

```

class WeatherAggregator:

```

```

    @classmethod

```

```

    async def get_aggregated_weather(

```

```

        cls, region: str, API_classes: list[WeatherAPIBase]

```

```

    ) -> AggregatedData:

```

```

        try:

```

```

            coordinates = await RegionHelper.convert_to_coordinates(region)

```

```

        except APIResponseException as e:

```

```

            logger.error(f"Weather aggregation failed for region '{region}': {e}")

```

```

        raise

```

```

    results = {}

```

```

    for API_class in API_classes:

```

```

        try:

```

```

            result = await API_class.get_weather(coordinates)

```

```

            results[API_class.__name__] = result

```

```

        except APIResponseException as e:

```

```

            logger.error(

```

```

        f"Failed to fetch weather data from {API_class.__name__} for region
        '{region}': {e}"
    )

```

```

        continue

```

```

    if not results:

```

```

        logger.error(f"No valid weather data available for region '{region}'")

```

```

        raise APIResponseException(

```

```

            f"No valid weather data available for region '{region}'"

```

```

        )

```

```

    aggregated_result = cls._aggregate_weather(results)

```

```

    return aggregated_result

```

```

    @classmethod

```

```

    def _aggregate_weather(cls, data: dict[str, ForecastData]) -> AggregatedData:

```

```

        result = {}

```

```

        template_API_result = data[list(data.keys())[0]]

```

```

        for day in template_API_result[rtke.DAILY]:

```

```

            if not cls._check_dicts_have_time_key(

```

```

                [d[rtke.DAILY] for d in data.values()], day

```

```

            ):

```

```

                continue

```

```

        result[day] = {}

```

```
result[day]["indicators"] = {}
```

```
for key in drke:
```

```
    result[day]["indicators"][key] = {}
```

```
    for API_class in data:
```

```
        value = data[API_class][rtke.DAILY][day].get(key, None)
```

```
        if value is not None:
```

```
            result[day]["indicators"][key][API_class] = value
```

```
    if len(result[day]["indicators"][key]) == 0:
```

```
        del result[day]["indicators"][key]
```

```
    elif not isinstance(value, str):
```

```
        result[day]["indicators"][key]["average"] = round(
```

```
            sum(result[day]["indicators"][key].values())
```

```
            / len(result[day]["indicators"][key]),
```

```
            2,
```

```
        )
```

```
result[day]["hours"] = {}
```

```
for time in template_API_result[rtke.HOURLY]:
```

```
    date, hour = time.split(" ")
```

```
    if date != day:
```

```
        continue
```

```
    if not cls._check_dicts_have_time_key(
```

```
        [d[rtke.HOURLY] for d in data.values()], time
```


):

continue

result[day]["hours"][hour] = {}

for key in hrke:

result[day]["hours"][hour][key] = {}

for API_class in data:

value = data[API_class][rtke.HOURLY][time].get(key, None)

if value is not None:

result[day]["hours"][hour][key][API_class] = value

if (

not isinstance(value, str)

and len(result[day]["hours"][hour][key]) > 1

):

result[day]["hours"][hour][key]["average"] = round(

sum(result[day]["hours"][hour][key].values())

/ len(result[day]["hours"][hour][key]),

2,

)

return result

@staticmethod

def _check_dicts_have_time_key(dict: list[dict[Any, Any]], key: str) -> bool:

for d in dict:

if key not in d:

```
return False
```

```
return True
```

Файл backend/external_api/region_helper.py

Реалізація функціональної задачі геокодування та отримання підказок.

```
import os
```

```
import httpx
```

```
import json
```

```
import logging
```

```
from ..resources.async_client import async_client
```

```
from ..resources.async_redis import AsyncRedis
```

```
from ..constants.redis_cache import COORDINATES_CACHE_TIME,
PARTIAL_CITY_CACHE_TIME
```

```
from ..exceptions.external_api_exceptions import APIResponseException
```

```
logger = logging.getLogger(__name__)
```

```
API_KEY = os.environ["GEOCODE_API_KEY"]
```

```
class RegionHelper:
```

```
    @staticmethod
```

```
    async def convert_to_coordinates(region: str) -> tuple[float, float]:
```

```
        cached_coordinates = await AsyncRedis.safe_get(
            f"cache:region_helper:convert_to_coordinates:{region}"
        )
```

```
        if cached_coordinates:
```

```
return json.loads(cached_coordinates)
```

```
url = f"https://geocode.maps.co/search?q={region}&api_key={API_KEY}"
```

```
try:
```

```
    response = await async_client.get(url)
```

```
    if response.status_code != 200:
```

```
        logger.error(f"Failed to fetch coordinates for region '{region}'")
```

```
        raise APIResponseException(
```

```
            f"Failed to fetch coordinates for region '{region}'"
```

```
        )
```

```
    data = response.json()
```

```
    if not data:
```

```
        logger.error(f"No coordinates found for region '{region}'")
```

```
        raise APIResponseException(
```

```
            f"No coordinates found for region '{region}'"
```

```
        )
```

```
    lat = float(data[0]["lat"])
```

```
    lon = float(data[0]["lon"])
```

```
    coordinates = (lat, lon)
```

```
    await AsyncRedis.safe_set(
```

```
        f"cache:region_helper:convert_to_coordinates:{region}",
```

```

        json.dumps(coordinates),
        ex=COORDINATES_CACHE_TIME,
    )

    return (lat, lon)
except APIResponseException as e:
    raise
except httpx.ConnectError as e:
    logger.error(f"Connection error: {e}")

    raise APIResponseException(f"Connection error: {e}")
except httpx.TimeoutException as e:
    logger.error(f"Timeout error: {e}")

    raise APIResponseException(f"Timeout error: {e}")

@staticmethod
async def get_options(partial_city: str) -> list[str]:
    cached_options = await AsyncRedis.safe_get(
        f"cache:region_helper:get_options:{partial_city}"
    )

    if cached_options:
        return json.loads(cached_options)

    url
    =
    f"https://geocode.maps.co/search?q={partial_city}&api_key={API_KEY}"

    result = []

```

```

try:
    response = await async_client.get(url)

    if response.status_code != 200:
        logger.error(
            f"Failed to fetch options for partial city '{partial_city}'"
        )

        raise APIResponseException(
            f"Failed to fetch options for partial city '{partial_city}'"
        )

    data = response.json()

    for option in data:
        result.append(option["display_name"])

    await AsyncRedis.safe_set(
        f"cache:region_helper:get_options:{partial_city}",
        json.dumps(result),
        ex=PARTIAL_CITY_CACHE_TIME,
    )

    return result
except APIResponseException as e:
    raise
except httpx.ConnectError as e:
    logger.error(f"Connection error: {e}")

    raise APIResponseException(f"Connection error: {e}")

```

```
except httpx.TimeoutException as e:
```

```
    logger.error(f"Timeout error: {e}")
```

```
    raise APIResponseException(f"Timeout error: {e}")
```

```
@staticmethod
```

```
async def convert_to_region(coordinates: tuple[float, float]) -> str:
```

```
    cached_region = await AsyncRedis.safe_get(
```

```
        f"cache:region_helper:convert_to_region:{coordinates}"
```

```
    )
```

```
    if cached_region:
```

```
        return json.loads(cached_region)
```

```
    lat, lon = coordinates
```

```
    url
```

```
=
```

```
f"https://geocode.maps.co/reverse?lat={lat}&lon={lon}&api_key={API_KEY}"
```

```
    try:
```

```
        response = await async_client.get(url)
```

```
    if response.status_code != 200:
```

```
        logger.error(f"Failed to fetch region for coordinates '{coordinates}'")
```

```
        raise APIResponseException(
```

```
            f"Failed to fetch region for coordinates '{coordinates}'"
```

```
        )
```

```
    data = response.json()
```

```

if "display_name" not in data:
    logger.error(f"No display name found for coordinates '{coordinates}'")

    raise APIResponseException(
        f"No display name found for coordinates '{coordinates}'"
    )

await AsyncRedis.safe_set(
    f"cache:region_helper:convert_to_region:{coordinates}",
    json.dumps(data["display_name"]),
    ex=COORDINATES_CACHE_TIME,
)

return data["display_name"]
except APIResponseException as e:
    raise
except httpx.ConnectError as e:
    logger.error(f"Connection error: {e}")

    raise APIResponseException(f"Connection error: {e}")
except httpx.TimeoutException as e:
    logger.error(f"Timeout error: {e}")

    raise APIResponseException(f"Timeout error: {e}")

```

Файл backend/resources/async_redis.py

Реалізація функціональної задачі кешування даних.

```

import redis.asyncio as aioredis
import time
import logging

```

```

from ..constants.redis_cache import REDIS_TIMEOUT,
REDIS_RECONNECTION_ATTEMPT_TIMEOUT

```

```

logger = logging.getLogger(__name__)

```

```

class AsyncRedis:

```

```

    _redis = None

```

```

    _last_connection_attempt = time.time()

```

```

    _connection_is_active = False

```

```

    @classmethod

```

```

    async def _try_reconnect(cls) -> None:

```

```

        cls._connection_is_active = False

```

```

        if (

```

```

            time.time() - cls._last_connection_attempt

```

```

            > REDIS_RECONNECTION_ATTEMPT_TIMEOUT

```

```

        ):

```

```

            cls._last_connection_attempt = time.time()

```

```

        try:

```

```

            cls._redis = aioredis.Redis(

```

```

                host="redis",

```

```

                socket_connect_timeout=REDIS_TIMEOUT,

```

```

                socket_timeout=REDIS_TIMEOUT,

```

```

            )

```

```

            await cls._redis.ping()

```



```

        cls._connection_is_active = True
    except aioredis.ConnectionError as e:
        logger.error(f"Connection error: {e}")

```

```
@classmethod
```

```
async def safe_set(cls, key: str, value: str, ex: int) -> None:
```

```
    if cls._connection_is_active:
```

```
        try:
```

```
            await cls._redis.set(key, value, ex=ex)
```

```
        except aioredis.ConnectionError as e:
```

```
            logger.error(f"Failed to set: {e}")
```

```
            await cls._try_reconnect()
```

```
    else:
```

```
        await cls._try_reconnect()
```

```
@classmethod
```

```
async def safe_get(cls, key: str) -> str | None:
```

```
    if cls._connection_is_active:
```

```
        try:
```

```
            return await cls._redis.get(key)
```

```
        except aioredis.ConnectionError as e:
```

```
            logger.error(f"Failed to get: {e}")
```

```
            await cls._try_reconnect()
```

```
    else:
```

```
        await cls._try_reconnect()
```

Файл frontend/src/App.jsx

Реалізація функціональної задачі структури додатку та маршрутизації.

```
import { createBrowserRouter, RouterProvider } from "react-router";
import { ForecastProvider } from "../contexts/ForecastContext";
import DailyPage from "../pages/DailyPage";
import GeneralPage from "../pages/GeneralPage";
import DynamicsPage from "../pages/DynamicsPage";
import MainLayout from "../layouts/MainLayout";
import HomePage from "../pages/HomePage";
import "../index.css";
```

```
const router = createBrowserRouter([
  {
    path: "/",
    element: (
      <ForecastProvider>
        <MainLayout />
      </ForecastProvider>
    ),
    children: [
      { path: "", element: <HomePage /> },
      { path: ":city", element: <GeneralPage /> },
      { path: ":city/:date", element: <DailyPage /> },
      { path: ":city/dynamics", element: <DynamicsPage /> },
    ],
  },
]);
```

```
const App = () => {
  return <RouterProvider router={router} />;
};
```

```
};
```

```
export default App;
```

Файл frontend/src/contexts/ForecastContext.jsx

Реалізація функціональної задачі керування станом та отримання даних у frontend-частині.

```
import { createContext, useContext, useEffect, useState } from "react";
```

```
import useFetch from "../hooks/useFetch";
```

```
const ForecastContext = createContext();
```

```
const ForecastProvider = ({ children }) => {
```

```
  const [city, setCity] = useState("");
```

```
  const [availableAPIs, setAvailableAPIs] = useState([]);
```

```
  const [usedAPIs, setUsedAPIs] = useState([]);
```

```
  const [forecast, setForecast] = useState(null);
```

```
  const forecastParams = new URLSearchParams();
```

```
  forecastParams.append("region", city);
```

```
  usedAPIs.forEach((API) => {
```

```
    forecastParams.append("API_classes", API);
```

```
  });
```

```
  const forecastURL = `forecast?${forecastParams.toString()}`;
```

```
  const shouldFetchForecast = city.trim() && usedAPIs.length > 0;
```

```
  const {
```

```

    data: forecastData,
    loading: loadingForecast,
    error: errorForecast,
  } = useFetch(forecastURL, shouldFetchForecast);

```

```

const {
  data: APIClassesData,
  loading: loadingAPIClasses,
  error: errorAPIClasses,
} = useFetch(`API_classes`);

```

```

useEffect(() => {
  if (forecastData) {
    setForecast(forecastData);
  }
}, [forecastData]);

```

```

useEffect(() => {
  if (APIClassesData) {
    const APIClasses = APIClassesData.API_classes;

    setAvailableAPIs(APIClasses);
    setUsedAPIs(APIClasses);
  }
}, [APIClassesData]);

```

```

return (
  <ForecastContext.Provider
    value={{
      city,

```

```

    forecast,
    usedAPIs,
    availableAPIs,
    setCity,
    setUsedAPIs,
    loadingAPIClasses,
    errorAPIClasses,
    loadingForecast,
    errorForecast,
  }}
>
  {children}
</ForecastContext.Provider>
);
};

const useForecastContext = () => useContext(ForecastContext);

export { ForecastProvider, useForecastContext };

```

Файл frontend/src/components/ForecastBlock.jsx

Реалізація функціональної задачі відображення узагальненого та погодинного прогнозу.

```

import ConditionalLink from "../ConditionalLink";
import generateServiceColor from "../utils/serviceColorGenerator";
import INDICATOR_TO_ICON_CONSTANTS from
  "../constants/indicatorToIconConstants";
import UNITS from "../constants/units";

const ForecastBlock = ({ forecast, onClickBaseLink }) => {

```

```

return (
  <div className="grid grid-cols-1 gap-y-5 lg:grid-cols-2 xl:grid-cols-3 justify-
items-center">
    {Object.entries(forecast).map(([time, data]) => (
      <ConditionalLink
        to={`/${onClickBaseLink}/${time}`}
        condition={onClickBaseLink}
        key={time}
      >
        <div key={time} className="p-1 bg-blue-400 rounded-xl">
          <div className="flex items-center justify-between">
            <h1 className="w-full text-4xl text-center text-white">{time}</h1>

            <div className="flex flex-col">
              {Object.entries(
                data.indicators
                  ? data.indicators["condition icon"]
                  : data["condition icon"]
              ).map(([service, icon]) => (
                <img
                  key={service}
                  src={icon}
                  alt={service}
                  style={{ backgroundColor: generateServiceColor(service) }}
                  className="m-1 rounded-2xl"
                />
              ))}
            </div>
          </div>
        </div>
      </div>
    )}
  </div>
)

```

```
<hr className="my-1 border-blue-200" />
```

```
<div
```

```
  className={`grid grid-cols-2 gap-2 justify-items-center sm:grid-cols-${
    Object.keys(data.indicators ? data.indicators : data).length - 1
  }`}
```

```
>
```

```
{Object.entries(data.indicators ? data.indicators : data).map(
  ([indicator, services]) =>
```

```
    indicator !== "condition icon" && (
```

```
      <div key={indicator} className="flex flex-col w-24">
```

```
        <h1 className="m-1 text-2xl text-center">
```

```
          {INDICATOR_TO_ICON_CONSTANTS[indicator]}
```

```
        </h1>
```

```
      {Object.entries(services).map(
```

```
        ([service, value]) =>
```

```
          service !== "average" && (
```

```
            <div
```

```
              key={service}
```

```
              style={{
```

```
                backgroundColor: generateServiceColor(service),
```

```
              }}
```

```
              className="w-full text-center rounded-lg"
```

```
            >
```

```
              <p>
```

```
                {value} {UNITS[indicator]}
```

```
              </p>
```

```
            </div>
```

```
          )
```

```

    })

    <p className="w-full mt-2 mb-1 text-center bg-green-400">
      {services.average} {UNITS[indicator]}
    </p>
  </div>
)
})
</div>
</div>
</ConditionalLink>
)}}
</div>
);
};

```

```
export default ForecastBlock;
```

Файл frontend/src/pages/DynamicsPage.jsx

Реалізація функціональної задачі візуалізації динаміки прогнозу та вибору періоду.

```

import { LineChart, XAxis, Line, ResponsiveContainer, YAxis } from "recharts";
import { useEffect, useState } from "react";
import { useForecastContext } from "../contexts/ForecastContext";
import SelectButton from "../components/SelectButton";
import generateServiceColor from "../utils/serviceColorGenerator";
import INDICATOR_TO_ICON_CONSTANTS from
  "../constants/indicatorToIconConstants";
import UNITS from "../constants/units";

```



```

const getMinMaxDates = (data) => {
  const dates = Object.keys(data);

  return [dates[0], dates[dates.length - 1]];
};

const getIndicators = (data) => {
  const [firstDate] = getMinMaxDates(data);
  const firstHour = Object.keys(data[firstDate].hours)[0];

  return Object.keys(data[firstDate].hours[firstHour]);
};

const DynamicsPage = () => {
  const [dailyPeriod, setDailyPeriod] = useState([null, null]);
  const [indicator, setIndicator] = useState(null);

  const { city, forecast, loadingForecast, errorForecast } =
    useForecastContext();

  useEffect(() => {
    if (forecast) {
      const [minDate, maxDate] = getMinMaxDates(forecast);
      const indicators = getIndicators(forecast);

      setDailyPeriod([minDate, maxDate]);
      setIndicator(indicators[0]);
    }
  }, [forecast]);

```

```

if (loadingForecast) {
  return <h1 className="text-center">Loading...</h1>;
}

if (errorForecast) {
  return <h1 className="text-center">No data available for {city}</h1>;
}

if (!forecast || !dailyPeriod[0] || !dailyPeriod[1] || !indicator) {
  return <h1 className="text-center">Processing...</h1>;
}

const data = [];

Object.entries(forecast).forEach(([date, dateData]) => {
  if (date >= dailyPeriod[0] && date <= dailyPeriod[1]) {
    Object.entries(dateData.hours).forEach(([hour, hourData]) => {
      data.push({
        time: `${date} ${hour}`,
        ...hourData[indicator],
      });
    });
  }
});

const [minDate, maxDate] = getMinMaxDates(forecast);
const indicators = getIndicators(forecast);

return (
  <div className="flex flex-col">
    <div className="flex justify-center">

```

```

<ResponsiveContainer width="75%" height={400}>
  <LineChart data={data}>
    <XAxis dataKey="time" />
    <YAxis
      label={{ value: `${UNITS[indicator]}`, angle: -90, dx: -25 }}
      domain={['dataMin - 2', 'dataMax + 2']}
      tickFormatter={(value) => Math.round(value)}
    />

    {Object.keys(data[0])
      .filter((key) => key !== "time" && key !== "average")
      .map((key) => (
        <Line
          key={key}
          type="monotone"
          dataKey={key}
          stroke={` ${generateServiceColor(key)} `}
        />
      ))}

    <Line type="monotone" dataKey="average" stroke="#4ade80" />
  </LineChart>
</ResponsiveContainer>

<div className="flex flex-col justify-center gap-2 ml-4 text-2xl -translate-y-5">
  {indicators.map(
    (ind) =>
      ind !== "condition icon" && (
        <SelectButton

```

```

        key={ind}
        onClick={() => setIndicator(ind)}
        selected={indicator === ind}
      >
        {INDICATOR_TO_ICON_CONSTANTS[ind]}
      </SelectButton>
    )
  }}
</div>
</div>

<div className="flex justify-center">
  <label htmlFor="start-date">Start date:</label>
  <input
    type="date"
    id="start-date"
    value={dailyPeriod[0]}
    min={minDate}
    max={dailyPeriod[1]}
    onChange={(e) => setDailyPeriod([e.target.value, dailyPeriod[1]])}
  />

  <label htmlFor="end-date">End date:</label>
  <input
    type="date"
    id="end-date"
    value={dailyPeriod[1]}
    min={dailyPeriod[0]}
    max={maxDate}
    onChange={(e) => setDailyPeriod([dailyPeriod[0], e.target.value])}
  />

```

```

    />
  </div>
</div>
);
};

export default DynamicsPage;

```

Файл frontend/src/pages/HomePage.jsx

Реалізація функціональної задачі автоматичного визначення місцезнаходження.

```

import { useEffect, useState } from "react";
import { useNavigate } from "react-router";
import useFetch from "../hooks/useFetch";

```

```

const HomePage = () => {
  const [URL, setURL] = useState("");

  const navigate = useNavigate();

  let shouldFetchLocation = URL !== "";

  const {
    data: locationData,
    loading: loadingLocation,
    error: errorLocation,
  } = useFetch(URL, shouldFetchLocation);

  useEffect(() => {
    if (navigator.geolocation) {

```

```

navigator.geolocation.getCurrentPosition((position) => {
  const { latitude, longitude } = position.coords;

  setURL(`convert_coordinates?lat=${latitude}&lon=${longitude}`);
});
}
}, []);

useEffect(() => {
  if (locationData) {
    const { region } = locationData;

    navigate(`/${region}`);
  }
}, [locationData, navigate]);

if (loadingLocation) {
  return <h1 className="text-center">Loading you location...</h1>;
}

if (errorLocation) {
  return <h1 className="text-center">Error while getting your city</h1>;
}

return (
  <h1 className="text-center">Allow your location for automatic fetch</h1>
);
};

export default HomePage;

```

Файл frontend/src/components/Header.jsx

Реалізація функціональної задачі пошуку міста, отримання підказок та вибору API.

```
import { useLocation, useNavigate, useParams } from "react-router";
import { useState } from "react";
import { useForecastContext } from "../contexts/ForecastContext";
import SelectButton from "../SelectButton";
import generateServiceColor from "../utils/serviceColorGenerator";
import useFetch from "../hooks/useFetch";
import useDebounce from "../hooks/useDebounce";
```

```
const Header = () => {
  const [partialCity, setPartialCity] = useState("");

  const debouncedPartialCity = useDebounce(partialCity.trim());
  const shouldFetchPartialCity = debouncedPartialCity.trim();

  const {
    data: optionsData,
    loading: loadingCityOptions,
    error: errorCityOptions,
  } = useFetch(
    `partial_city_helper?partial_city=${debouncedPartialCity}`,
    shouldFetchPartialCity
  );

  const { city, setUsedAPIs, availableAPIs, usedAPIs, forecast } =
    useForecastContext();
```

```
const navigate = useNavigate();
```

```
const location = useLocation();
```

```
const { date } = useParams();
```

```
const isDayOpened = !!date;
```

```
const isDynamicsPage = location.pathname.includes("/dynamics");
```

```
let resultAPIs = [];
```

```
if (forecast) {
```

```
  const firstDay = Object.keys(forecast)[0];
```

```
  const indicators = forecast[firstDay].indicators;
```

```
  Object.values(indicators).forEach((APIGroup) => {
```

```
    Object.keys(APIGroup).forEach((API) => {
```

```
      if (!resultAPIs.includes(API)) {
```

```
        resultAPIs.push(API);
```

```
      }
```

```
    });
```

```
  });
```

```
}
```

```
const onOptionClick = (option) => {
```

```
  setPartialCity("");
```

```
  navigate(`/${option}`);
```

```
};
```

```
const OnAPIButtonClicked = (e) => {
```



```

    if (e.target.checked) {
      setUsedAPIs((prev) => [...prev, e.target.id]);
    } else if (usedAPIs.length > 1) {
      setUsedAPIs((prev) => prev.filter((API) => API !== e.target.id));
    }
  };

  return (
    <header className="flex justify-around gap-2 mt-5 mb-12">
      <div className="flex-1">
        {isDayOpened && (
          <SelectButton onClick={() => navigate(`/${city}`)}>
            <h1 className="w-12 text-xs sm:w-28 sm:text-sm lg:text-lg lg:w-32">
              Back to daily forecast
            </h1>
          </SelectButton>
        )}
      </div>

      <div className="flex flex-col items-center flex-[4]">
        <input
          className="p-0.5 border border-black rounded-lg w-2/3 sm:w-auto"
          type="text"
          placeholder="City name"
          onChange={(e) => setPartialCity(e.target.value)}
          value={partialCity}
        />

        <div className="flex flex-col gap-1 mt-1 text-center">
          {loadingCityOptions && <div>Loading...</div>}
        </div>
      </div>
    </header>
  );
}

```

```
{errorCityOptions && <div>Error: {errorCityOptions}</div>}
```

```
{partialCity.trim() &&
```

```
[...new Set(optionsData?.options)].map((option) => (
```

```
<SelectButton key={option} onClick={() => onOptionClick(option)}>
```

```
  {option}
```

```
</SelectButton>
```

```
)))
```

```
</div>
```

```
<div className="flex flex-col gap-2 mt-3">
```

```
{availableAPIs.length > 0 ? (
```

```
  availableAPIs.map((API) => (
```

```
    <div key={API}>
```

```
      <input
```

```
        type="checkbox"
```

```
        id={API}
```

```
        className="scale-125 sm:scale-150"
```

```
        checked={usedAPIs.includes(API)}
```

```
        onChange={OnAPIButtonClicked}
```

```
      />
```

```
      <label htmlFor={API} className="m-2 text-xs sm:text-lg">
```

```
        {API}
```

```
      </label>
```

```
    </div>
```

```
  ))
```

```
): (
```

```
  <div>No APIs available</div>
```

```
))
```

```
</div>
```

```
{city.trim()} && (
```

```
<div>
```

```
<h1 className="mt-5 text-2xl text-center">
```

```
Weather for <span className="font-bold">{city}</span>
```

```
</h1>
```

```
<div className="flex justify-center gap-1 mt-2">
```

```
<SelectButton
```

```
onClick={() => navigate(`/${city}`)}
```

```
selected={!isDynamicsPage}
```

```
>
```

```
Forecast
```

```
</SelectButton>
```

```
<SelectButton
```

```
onClick={() => navigate(`/${city}/dynamics`)}
```

```
selected={isDynamicsPage}
```

```
>
```

```
Dynamics
```

```
</SelectButton>
```

```
</div>
```

```
</div>
```

```
)}
```

```
</div>
```

```
<div className="flex-1">
```

```
{resultAPIs.map(
```

```
(API) =>
```

```

    API !== "average" && (
      <h1
        key={API}
        style={{ backgroundColor: generateServiceColor(API) }}
        className="w-20 p-1 mt-1 overflow-scroll text-xs text-center rounded-
2xl no-scrollbar sm:text-sm sm:w-auto lg:text-lg"
      >
        {API}
      </h1>
    )
  )}
</div>
</header>
);
};

export default Header;

```